DO LLM RED-TEAM AGENTS GENERATE REALISTIC HONEYPOT TRAFFIC? A TRANSFER STUDY AGAINST REAL COWRIE SESSIONS

A PROJECT REPORT

Submitted by

KESHAV ([UID])

ANSHUL SHARMA ([UID])

PARTH KANSAL ([UID])

VIRAIN NAYAR ([UID])

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Chandigarh University

[MONTH YEAR]

BONAFIDE CERTIFICATE

Certified that this project report "**Do LLM Red-Team Agents Generate Realistic Honeypot Traffic? A Transfer Study Against Real Cowrie Sessions**" is the bonafide work of "**Keshav, Anshul Sharma, Parth Kansal, Virain Nayar**" who carried out the project work under my/our supervision.

SIGNATURE

[Name of the Head of the Department]

HEAD OF THE DEPARTMENT

[Department of Computer Science and Engineering]

SIGNATURE

[Name of the Supervisor]

SUPERVISOR

[Academic Designation]

[Department of Computer Science and Engineering]

Submitted for the project viva-voce examination held on

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

This project would not have been possible without the help of many people, and we are glad to thank them here.

We are deeply grateful to our supervisor, [Name of the Supervisor], for steady guidance, honest feedback and for always asking the uncomfortable question "how do you know that number is right?". That single question shaped the whole method of this work. We also thank [Name of the Head of the Department], Head of the Department, and all faculty members of the [Department of Computer Science and Engineering], Chandigarh University, for providing the academic environment, the laboratory time and the encouragement we needed.

We thank the maintainers of the Cowrie honeypot and the authors of the CyberLab honeynet dataset. Their decision to release software and data openly made this study possible. Our thanks also go to the many open-source communities whose tools we used every day: Python, scikit-learn, SciPy, NumPy, Matplotlib, Paramiko, Docker, LaTeX and draw.io.

Finally, we thank our families and friends for their patience during the long evenings spent watching log files scroll past, and each other, for the arguments that made the work better.

Keshav, Anshul Sharma, Parth Kansal and Virain Nayar

TABLE OF CONTENTS

ACKNOWLEDGEMENT iii

LIST OF FIGURES xi

LIST OF TABLES xiv

ABSTRACT xvii

GRAPHICAL ABSTRACT xviii

ABBREVIATIONS xix

SYMBOLS xxi

CHAPTER 1 INTRODUCTION 1

1.1 Background and Motivation 1

1.1.1 Why defenders watch attackers 1

1.1.2 From logs to learning 1

1.1.3 The supply problem 1

1.1.4 A tempting shortcut: LLM red-team agents 2

1.2 Identification of the Client and the Need 2

1.2.1 Who needs this work 2

1.2.2 Evidence that the need is real 3

1.3 Relevant Contemporary Issues 3

1.4 Problem Identification 3

1.5 Task Identification 4

1.6 Research Questions and Hypotheses 6

1.7 Timeline 7

1.8 Contributions 8

1.9 Organisation of the Report 8

CHAPTER 2 LITERATURE SURVEY 10

2.1 Timeline of the Reported Problem 10

2.2 Honeypots: Concepts and Taxonomy 13

2.2.1 What a honeypot is 13
2.2.2 Notable honeypot systems 13
2.2.3 Honeypots under scrutiny 14
2.3 What Real Honeypot Data Looks Like 14
2.3.1 Public datasets 14
2.3.2 Repetition and skew 15
2.4 The Threat Landscape Inside Honeypot Traffic 15
2.5 Datasets and Machine Learning for Intrusion Detection 16
2.5.1 The long shadow of synthetic benchmarks 16
2.5.2 Why ML-based detection is hard 16
2.5.3 Methods we rely on 17
2.6 Synthetic Data for Security 17
2.7 Large Language Models and Agents 18
2.7.1 The models 18
2.7.2 Agents, reasoning and their failure modes 18
2.7.3 Surveys of LLMs in security 18
2.8 LLM Agents for Offensive Security 19
2.9 LLM-Based Honeypots 19
2.10 Dataset Shift and Two-Sample Testing 20
2.11 LLMs as Annotators and Measuring Agreement 21
2.12 Reproducible and Responsible Research 21
2.13 Summary of Solutions Proposed by Different Researchers 22
2.14 Research Gap Analysis 23
2.15 Summary Linking the Literature to This Project 24
2.16 Problem Definition 24
2.17 Goals and Objectives 25
CHAPTER 3 DESIGN FLOW AND PROCESS 27

3.1 Concept Generation 27
3.2 Evaluation and Selection of Specifications and Features 29
3.2.1 Functional requirements 29
3.2.2 Non-functional requirements 29
3.2.3 Weighted decision matrix 30
3.3 Design Constraints 30
3.4 Analysis and Feature Finalisation Subject to Constraints 31
3.5 Design Flow: Alternative Designs 32
3.6 Best Design Selection 34
3.7 Implementation Plan 35
3.7.1 Overall architecture and method 35
3.7.2 Testbed configuration 37
3.7.3 Harness structure 38
3.7.4 The attacker session loop 40
3.7.5 Reading the honeypot's reply 42
3.7.6 Surviving a frozen honeypot 44
3.7.7 Parsing the real dataset 45
3.7.8 Normalisation and behaviour templates 47
3.7.9 Tactic tagging 48
3.7.10 Label-free measures 49
3.7.11 Annotation protocol 51
3.7.12 Four-quadrant transfer experiment 55
3.7.13 Data model 57
3.7.14 Risk register 59
3.8 Software Design and Engineering 59
3.8.1 Development life cycle 59
3.8.2 Use cases 60

3.8.3 Data-flow diagrams 62
3.8.4 Components and technology stack 65
3.8.5 Class and module design 67
3.8.6 Interaction and state views 69
3.8.7 Algorithmic complexity 71
3.8.8 Engineering practices 72
CHAPTER 4 RESULTS ANALYSIS AND VALIDATION 73
4.1 Implementation Using Modern Engineering Tools 73
4.2 Design Drawings and Schematics 74
4.3 Characterising the Real Data 76
4.3.1 Overall profile 76
4.3.2 Where the traffic came from, and what it tried 77
4.3.3 A few scripts dominate 78
4.4 How the LLM Attackers Behaved (RQ1, RQ4) 80
4.4.1 Overview 80
4.4.2 Effect of the LLM-backed honeypot 83
4.4.3 How sessions ended 83
4.5 A Contamination Finding: Chat-Format Token Leakage 84
4.6 Synthetic Versus Real Without Labels (RQ2, RQ4) 85
4.6.1 Corpus structure 85
4.6.2 Tactic profiles 85
4.6.3 Command verbs 87
4.6.4 Classifier two-sample test 87
4.6.5 Which synthetic cell is closest to real? 88
4.7 Annotation Results 89
4.7.1 Agreement 89
4.7.2 Final labels 91

4.8 The Transfer Experiment (RQ3) 92
4.8.1 Main results 92
4.8.2 Why transfer fails 95
4.9 Sensitivity and Robustness Checks 95
4.9.1 Does the dota label drive the result? 95
4.9.2 The keyword-rule plumbing check 96
4.9.3 Other robustness choices 96
4.10 Answers to the Research Questions 97
4.11 Discussion 97
4.11.1 The gap is about intent 97
4.11.2 What the LLM-backed honeypot changed 97
4.11.3 What this means for practitioners 98
4.12 Threats to Validity 98
4.13 Testing, Characterisation and Data Validation 98
4.13.1 Software testing strategy 100
4.13.2 Test cases 101
4.14 Report Preparation, Project Management and Communication 101
CHAPTER 5 CONCLUSION AND FUTURE WORK 103
5.1 Conclusion 103
5.2 Deviation from Expected Results 104
5.3 Lessons Learned 104
5.4 Future Work 104
5.5 Final Remarks 105
REFERENCES 106
APPENDICES 114
Appendix 1: User Manual 115
A1.1 What you need 115

A1.2 Step 1 — Get the code 115
A1.3 Step 2 — Get the real dataset 115
A1.4 Step 3 — (Optional) Run the live honeypot 116
A1.5 Step 4 — (Optional) Generate synthetic sessions 116
A1.6 Step 5 — Parse and normalise the real data 116
A1.7 Step 6 — Run the analysis 117
A1.8 Step 7 — (Optional) Reproduce the labelling 117
A1.9 Step 8 — Build this report 117
A1.10 Where every output lives 118
A1.11 Troubleshooting 118
Appendix 2: Annotation Rubric and Protocol 119
A2.1 Instructions to annotators 119
A2.2 The eight labels 119
A2.3 Tie-break order 119
A2.4 Worked rubric examples 120
A2.5 The adjudication record 120
Appendix 3: Data Schemas 121
A3.1 Raw synthetic session log 121
A3.2 Normalised session (both corpora) 121
A3.3 Annotation files 122
A3.4 Result files 122
Appendix 4: List of Scripts 123
Appendix 5: Glossary 124
Appendix 6: Catalogue of the Real Behaviour Templates 125
Appendix 7: Adjudicated Annotation Disagreements 127
Appendix 8: Summary of All Synthetic Sessions 130
DeepSeek-V3.2 against Dynamic Qwen-32B 130

DeepSeek-V3.2 against Static Cowrie 130
GPT-OSS-120B against Dynamic Qwen-32B 131
GPT-OSS-120B against Static Cowrie 132
Gemma-3-4B against Dynamic Qwen-32B 132
Gemma-3-4B against Static Cowrie 133
Appendix 9: Configuration Files 135
Cowrie honeypot configuration (cowrie_etc/cowrie.cfg) 135
Docker Compose file (docker-compose.yml) 135
Analysis container (Dockerfile.attacker) 136
Python dependencies (requirements.txt) 137
Appendix 10: Source Code of the Analysis Scripts 138
Normalisation, verbs, tactic tags and templates (session_features.py) 138
CyberLab parser (parse_cyberlab.py) 140
Label-free comparison (domain_shift.py) 141
Blind annotation sheets (make_annotation_sheet.py) 145
Agreement and final labels (agreement.py) 146
Four-quadrant transfer experiment (train_eval_classifier.py) 149
Appendix 11: Achievements 153
Paper status 153
Authors 153

LIST OF FIGURES

Figure 1.1: Work breakdown structure of the project. 5
Figure 1.2: Project timeline (Gantt chart). Blue bars come from log and file timestamps; grey bars are approximate planned windows. 7
Figure 2.1: Timeline of work on honeypots, attack data and LLM agents, from 1989 to this project. 11
Figure 3.1: Concept generation: six ways to obtain labelled attack sessions. 27
Figure 3.2: Three alternative designs. Design A was our first prototype; Design B is the final design; Design C is the ideal but unaffordable option. 33
Figure 3.3: System architecture: synthetic data generation, data preparation and analysis. 36
Figure 3.4: Research methodology in nine stages. 37
Figure 3.5: Module structure of the attacker harness. 39
Figure 3.6: Attacker session loop: health check, connection, the decisions taken in every turn, and the status saved at each exit. 41
Figure 3.7: Idle-timeout reading of the honeypot's reply. 43
Figure 3.8: Session retry and recovery logic. 44
Figure 3.9: Streaming parser for the CyberLab export. 46
Figure 3.10: Command normalisation in six steps, with three worked examples produced by the project's own code. 47
Figure 3.11: Label-free domain-shift analysis. 49
Figure 3.12: Blind dual-LLM annotation, adjudication, author verification and template broadcast. 52
Figure 3.13: The rubric's tie-break order, read from top to bottom. 54
Figure 3.14: The four quadrants of the transfer experiment, with the main results. 56
Figure 3.15: Data model of the project's main records. 58
Figure 3.16: Iterative development life cycle, with the main design changes made in this project. 60
Figure 3.17: Use case diagram of the research pipeline. 61
Figure 3.18: Data-flow diagram, level 0 (context diagram). 63
Figure 3.19: Data-flow diagram, level 1 64
Figure 3.20: Component diagram. 66
Figure 3.21: Class and module diagram. 68
Figure 3.22: Sequence diagram of one session. 70

Figure 3.23: State machine of a session and its possible outcomes. 71
Figure 4.1: Starting the Cowrie honeypot in Docker and checking that it answers with an SSH banner (typical console output). 74
Figure 4.2: Running one attacker batch; each line reports the outcome of one session (example from the DeepSeek-V3.2 static batch). 74
Figure 4.3: Docker deployment of the testbed. 75
Figure 4.4: From raw sessions to behaviours (log scale), and the protocol split. 77
Figure 4.5: Top source countries for all sessions and for interactive sessions. 78
Figure 4.6: Most frequent usernames and passwords in login attempts (log scale). 78
Figure 4.7: Number of sessions in each of the 22 real behaviour templates. 79
Figure 4.8: Output of the CyberLab parser. 80
Figure 4.9: Share of commands matching each keyword category, per attacker and honeypot backend. 81
Figure 4.10: Total and unique commands per cell. 81
Figure 4.11: The first command of each synthetic session. 82
Figure 4.12: The first three turns of a DeepSeek-V3.2 session against the static honeypot (output shortened). 82
Figure 4.13: Mean and median latency per command. 83
Figure 4.14: How the 20 sessions in each cell ended. 84
Figure 4.15: Corpus structure: synthetic versus real. 85
Figure 4.16: Share of sessions touching each tactic at least once. 86
Figure 4.17: The ten most frequent verbs in each corpus. 87
Figure 4.18: Beginning of the label-free analysis output. 88
Figure 4.19: Verb JSD between each attacker × backend cell and the real templates, with 95% bootstrap confidence intervals. 89
Figure 4.20: Confusion between the two annotators (rows: annotator A; columns: annotator B). 90
Figure 4.21: Output of the agreement script. 91
Figure 4.22: Final label distribution as a share of each corpus (counts on top of bars). 91
Figure 4.23: Macro-F1 of the classifier and of the majority-class baseline in each quadrant. 93
Figure 4.24: True labels of the real sessions and the labels predicted by the synthetic-trained model. 94
Figure 4.25: Console output of the transfer experiment. 95

Figure 4.26: Macro-F1 per quadrant under three label sources. The keyword-rule labels are a plumbing check only and are not a result. 96
Figure 4.27: Validation checks used throughout the project. 99

LIST OF TABLES

Table 1.1: Stakeholders and their needs. 2
Table 1.2: Contemporary issues connected to the project. 3
Table 1.3: Task identification, deliverables and verification. 6
Table 1.4: Research questions and hypotheses. 7
Table 1.5: Project schedule and milestones. 8
Table 2.1: Milestones in the history of the problem. 12
Table 2.2: Honeypot interaction levels. 13
Table 2.3: Comparison of honeypot systems. 14
Table 2.4: Intrusion datasets and known issues. 16
Table 2.5: LLM agents for offensive security. 19
Table 2.6: LLM-based honeypots. 20
Table 2.7: Methods for detecting dataset shift. 21
Table 2.8: Proposed solutions by different researchers and their limitations for this problem. 23
Table 2.9: Research gap matrix (✓ = provided, – = not provided). 24
Table 2.10: Objectives with their Bloom level, research questions and where they are reported. 26
Table 3.1: Concepts considered and their trade-offs. 28
Table 3.2: Functional requirements of the research system. 29
Table 3.3: Non-functional requirements. 29
Table 3.4: Weighted decision matrix for the six concepts (higher is better). 30
Table 3.5: Design constraints and how the design responds to them. 31
Table 3.6: Feature finalisation. 32
Table 3.7: Comparison of the alternative designs. 34
Table 3.8: Cowrie honeypot configuration (from cowrie_etc/cowrie.cfg; secrets omitted). 37
Table 3.9: Language models used in the project. 38
Table 3.10: Main harness parameters (defaults in the code; batch files can override them). 39
Table 3.11: Session outcomes recorded by the harness. 42
Table 3.12: Values masked during normalisation (session_features.py). 48
Table 3.13: The nine tactic categories and example patterns. 49

Table 3.14: Annotation rubric: one primary intent per session (annotation/RUBRIC.md). 53
Table 3.15: Classifier and evaluation settings. 56
Table 3.16: Risks identified during design and how they were handled. 59
Table 3.17: Use case summary. 62
Table 3.18: Data dictionary of the main stores and flows. 65
Table 3.19: Technology stack. 67
Table 3.20: Main analysis interfaces. 69
Table 3.21: Time and memory cost of the main analysis steps. 72
Table 3.22: Engineering practices used in the project. 72
Table 4.1: Engineering tools used in the project. 73
Table 4.2: Repository layout. 76
Table 4.3: Profile of the real CyberLab data (15 November 2019). 77
Table 4.4: The largest real behaviour templates (normalised commands; first steps only). 79
Table 4.5: Attacker behaviour per cell (20 sessions each). Recon and credential values are percentages of commands and can overlap. Latency is the mean time until output goes quiet; change is relative to the static backend. 80
Table 4.6: Session status counts per cell (C completed, M turn limit, L connection lost, E LLM error, R refused, T timed out). 84
Table 4.7: Corpus structure (results/domain_shift.md, Table 4a). 85
Table 4.8: Share of sessions touching each tactic (results/domain_shift.md, Table 4b). 86
Table 4.9: Verb-distribution statistics (results/domain_shift.md, Table 4c). 87
Table 4.10: Classifier two-sample test, synthetic versus real templates (results/domain_shift.md, Table 4d). 88
Table 4.11: Distance of each cell to the real templates (results/domain_shift.md, Table 4e). 89
Table 4.12: Inter-annotator agreement (results/annotation_agreement.md). 89
Table 4.13: Final label counts. Real labels are copied from 22 templates, so the effective real sample size is 22. 92
Table 4.14: Transfer results (results/table3_transfer.md). For Q1, Q2 and Q4, n_train is the mean training size per fold and n_test counts pooled out-of-fold predictions. F1 is macro-F1 over classes present in the test set. 92
Table 4.15: Per-class F1 of the synthetic-trained model on real sessions (Q3). 94
Table 4.16: Number of sessions predicted in each class per quadrant (results/table3_transfer.md). 94

Table 4.17: Main labels versus the dota sensitivity run (macro-F1 / accuracy). 96
Table 4.18: Research questions, hypotheses and outcomes. 97
Table 4.19: Threats to validity and how they were handled. 98
Table 4.20: Testing and validation checks. 100
Table 4.21: Testing levels. 100
Table 4.22: Test cases with expected and observed results. 101
Table 5.1: Expected versus actual results. 104
Table 5.2: Future work. 105
Table A1.1: Prerequisites for running the project. 115
Table A1.2: Where to find each output. 118
Table A1.3: Common problems and fixes. 118
Table A2.1: Example sessions and their labels under the rubric. 120
Table A3.1: Fields of a raw synthetic session log. 121
Table A3.2: Fields of a normalised session. 122
Table A3.3: Annotation files and their columns. 122
Table A4.1: The project's scripts and what they do. 123
Table A5.1: Glossary of key terms. 124
Table A6.1: The real behaviour templates, largest first. 125
Table A7.1: Every disagreement, the final label and the adjudicator's reason. 128
Table A8.1: Synthetic sessions of DeepSeek-V3.2 against Dynamic Qwen-32B. 130
Table A8.2: Synthetic sessions of DeepSeek-V3.2 against Static Cowrie. 130
Table A8.3: Synthetic sessions of GPT-OSS-120B against Dynamic Qwen-32B. 131
Table A8.4: Synthetic sessions of GPT-OSS-120B against Static Cowrie. 132
Table A8.5: Synthetic sessions of Gemma-3-4B against Dynamic Qwen-32B. 132
Table A8.6: Synthetic sessions of Gemma-3-4B against Static Cowrie. 133
Table A11.1: Project achievements and outputs. 153

ABSTRACT

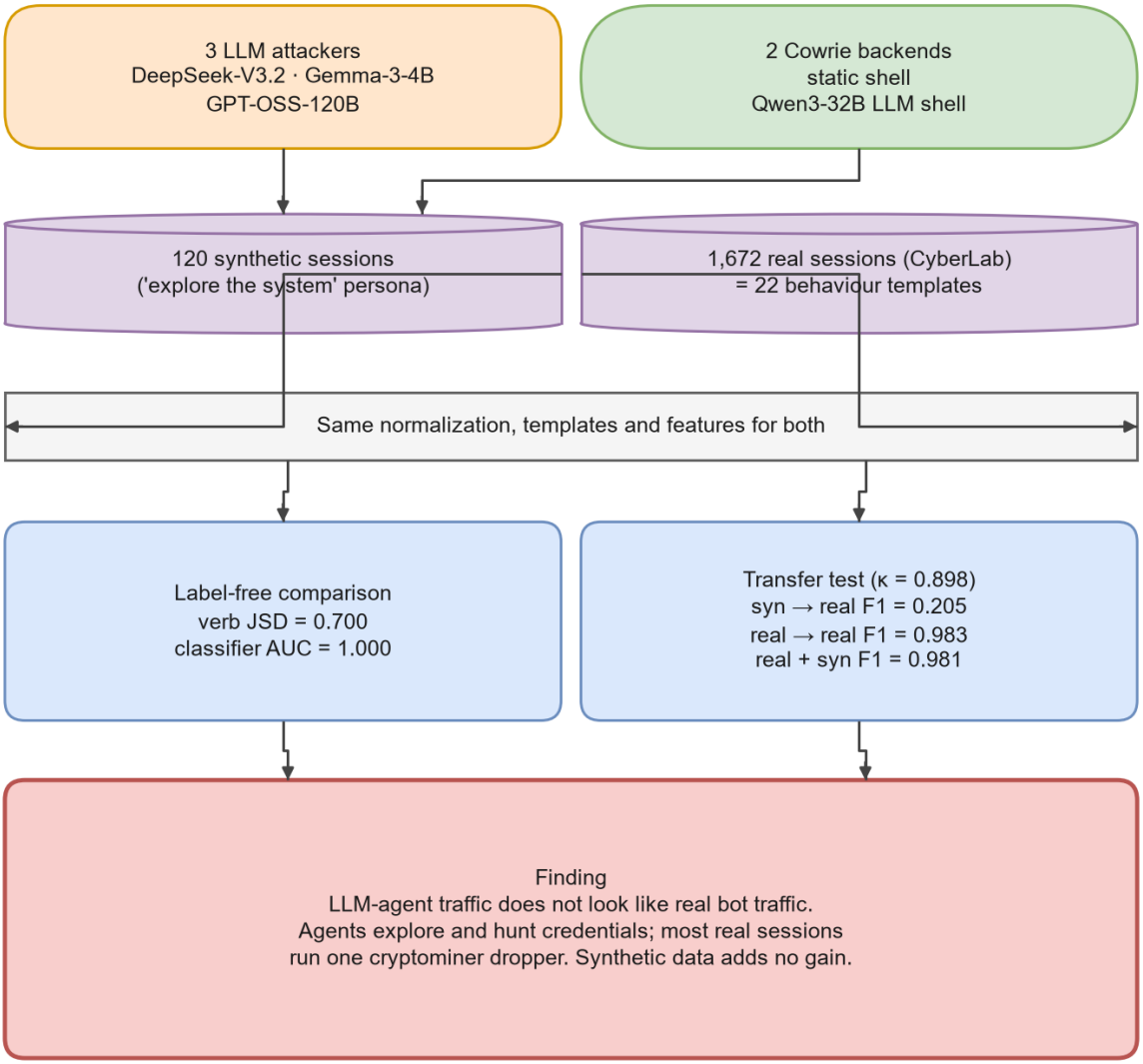
Labelled attack data is scarce, so it is tempting to let LLM "red-team" agents attack a honeypot we own and train on what they do. We tested whether this works. Three LLM attackers (DeepSeek-V3.2, Gemma-3-4B, GPT-OSS-120B) each ran 20 sessions against a static Cowrie honeypot and one whose replies were written by Qwen3-32B. We compared these 120 sessions with 1,672 real interactive sessions (22 behaviour templates) from one day of the public CyberLab honeynet.

Without labels, the two sets are almost perfectly separable: verb distributions diverge strongly (Jensen-Shannon divergence 0.700) and a simple classifier tells them apart (AUC 1.000). The reason is intent: the agents explored files and hunted credentials, while 1,038 real sessions ran one cryptominer dropper. With blind dual-LLM intent labels (Cohen's κ 0.898), a classifier trained on synthetic sessions scored a macro-F1 of 0.205 on real sessions, against 0.983 when trained on real data; adding synthetic data gave no gain (0.981). GPT-OSS-120B also leaked chat-format tokens into the shell in 18 of 40 sessions.

With a single "explore the system" persona, LLM-agent sessions are not a substitute for real honeypot traffic.

Keywords: honeypot; Cowrie; large language models; red teaming; synthetic data; dataset shift; intrusion detection

GRAPHICAL ABSTRACT



Graphical abstract. Two attacker conditions produce 120 synthetic sessions that are compared with 1,672 real CyberLab sessions, first without labels and then with a four-quadrant transfer test.

ABBREVIATIONS

List of abbreviations used in this report.

Abbreviation	Full form
ACM	Association for Computing Machinery
AI	Artificial Intelligence
API	Application Programming Interface
ATT&CK	Adversarial Tactics, Techniques and Common Knowledge (MITRE knowledge base)
AUC	Area Under the (ROC) Curve
AWS	Amazon Web Services
C2	Command and Control
C2ST	Classifier Two-Sample Test
CI	Confidence Interval
CLI	Command-Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
CTF	Capture The Flag
CV	Cross-Validation
CVD	Colour Vision Deficiency
DPDP	Digital Personal Data Protection (Act, India, 2023)
DOI	Digital Object Identifier
F1	Harmonic mean of precision and recall
GAN	Generative Adversarial Network
HTTPS	Hypertext Transfer Protocol Secure
IDS	Intrusion Detection System
IoT	Internet of Things
IP	Internet Protocol
JSD	Jensen-Shannon Divergence
JSON	JavaScript Object Notation
JSONL	JSON Lines (one JSON object per line)
KL	Kullback-Leibler (divergence)
LLM	Large Language Model
LNNS	Lecture Notes in Networks and Systems (Springer)
ML	Machine Learning
NIST	National Institute of Standards and Technology
PNG	Portable Network Graphics
Q1–Q4	Quadrants 1 to 4 of the transfer experiment
RF	Random Forest
ROC	Receiver Operating Characteristic
SOC	Security Operations Centre

Abbreviation	Full form
SSH	Secure Shell
SUID	Set User ID (Unix permission bit)
TF-IDF	Term Frequency – Inverse Document Frequency
TTP	Tactics, Techniques and Procedures
UTC	Coordinated Universal Time
VM	Virtual Machine

SYMBOLS

List of symbols used in equations and tables.

Symbol	Meaning
P, Q	Two probability distributions (for example, synthetic and real verb frequencies)
M	Mixture distribution, M = ½(P + Q)
D_KL(P‖Q)	Kullback-Leibler divergence of P from Q
JSD(P‖Q)	Jensen-Shannon divergence between P and Q (base 2, range 0 to 1)
H(P)	Shannon entropy of distribution P
J(A, B)	Jaccard index of sets A and B
A ∩ B, A ∪ B	Intersection and union of sets
|A|	Number of elements in set A
N	Number of documents (sessions)
tf(t, d)	Frequency of token t in document d
df(t)	Number of documents that contain token t
w(t, d)	TF-IDF weight of token t in document d
κ	Cohen's kappa (chance-corrected agreement)
p_o, p_e	Observed and chance-expected agreement
P_c, R_c	Precision and recall for class c
F1_c	F1 score for class c
C_test	Set of classes present in the test set
B	Number of bootstrap resamples (500 in this work)
n_train, n_test	Training and test set sizes
Δ	Relative change, for example in latency
$\hat{\theta}$	An estimated statistic (for example a JSD value)
1[·]	Indicator function (1 if the condition is true, else 0)
s_i	Classifier score for item i

CHAPTER 1
INTRODUCTION

1.1 Background and Motivation

1.1.1 Why defenders watch attackers

Every machine with a public IP address is probed within minutes of going online. Most of this traffic is automated: scanners sweep the address space, bots try default passwords, and successful logins are followed by short, scripted command sequences [3, 27]. Defenders who want to understand this activity need to see it up close, and the classic way to do that is a honeypot: a system that has no production value, so any interaction with it is suspicious by definition [100].

The idea is old. Clifford Stoll followed an intruder through Lawrence Berkeley Laboratory's systems in the 1980s [102], and Bill Cheswick built a jail to watch a cracker he named "Berferd" [20]. The Honeynet Project later turned these one-off efforts into a research discipline [101], and low-cost virtual honeypots made large deployments practical [86, 87]. Today the most widely used SSH and Telnet honeypot is Cowrie, a medium-interaction honeypot that emulates a Linux shell and logs every login attempt, every command and every downloaded file [78].

1.1.2 From logs to learning

Honeypot logs are valuable on their own, but their real promise is as training data. If we can label each session with what the intruder was trying to achieve (looking around, stealing credentials, installing a backdoor, mining cryptocurrency), we can train a classifier that does this labelling automatically for new traffic. Such labels map naturally onto the tactics of the MITRE ATT&CK knowledge base [103] and onto the stages of the intrusion kill chain [54]. A security operations centre (SOC) can then prioritise the rare session that tries to persist over the thousands that only probe.

1.1.3 The supply problem

The difficulty is supply. Real interactive sessions are rare and highly repetitive. In the single day of public CyberLab data used in this project [92], only 1,672 of 76,785 sessions (2.18%) typed even one command, and those 1,672 sessions collapse to just 22 distinct behaviours once IP addresses, URLs and random passwords are masked. One botnet script

alone accounts for 1,038 sessions. Machine learning on such data runs straight into the problems that Sommer and Paxson [98] and Arp et al. [4] warned about: small samples, sampling bias and evaluation that quietly rewards memorisation.

1.1.4 A tempting shortcut: LLM red-team agents

Large language models (LLMs) changed the picture in 2023 and 2024. Research prototypes showed that LLM agents can plan and carry out multi-step penetration tests [24, 48], escalate privileges on Linux machines [49] and even attack web applications without human help [34]. If an agent can behave like an intruder, it can be pointed at a honeypot we own and asked to "attack" it as many times as we like. Every session would be cheap, fully logged and free of privacy concerns. That is the shortcut this project puts to the test.

> **Key idea.** Synthetic sessions are only useful as training data if a model trained on them still works on real traffic. This report measures exactly that, instead of assuming it.

1.2 Identification of the Client and the Need

1.2.1 Who needs this work

The "client" of a research project is the group of people whose problem it solves. **Table 1.**1 lists the main stakeholders we identified and what each of them needs from the answer.

Table 1.1: Stakeholders and their needs.

Stakeholder	Typical situation	What they need from this project
SOC analysts and blue teams	Run honeypots or sensors, drown in repetitive alerts	To know whether synthetic data can train intent classifiers they can trust
Honeypot operators and researchers	Collect data slowly, have few labelled sessions	A tested pipeline and an honest estimate of how far synthetic data can go
ML-for-security researchers	Publish detectors trained on public or synthetic data	Evidence about dataset shift between agent and real traffic, plus reusable code
Builders of LLM-based honeypots	Use an LLM to imitate a shell [96, 112]	To learn whether an adaptive backend changes attacker behaviour
Educators and students	Teach intrusion analysis with realistic examples	Labelled, documented, ethically sourced example sessions
AI safety and policy community	Track what offensive AI agents actually do [115]	A measured description of agent behaviour, including failure modes such as token leakage

1.2.2 Evidence that the need is real

Three observations justify the need. First, real interactive data is scarce: the 2.18% interaction rate above is typical of internet-facing honeypots, where most connections are scans or failed logins [8, 110]. Second, the scarce data is skewed: a handful of botnet families, such as Mirai and its descendants [3, 60] or the Outlaw cryptomining group [108], produce most of it. Third, practitioners are already experimenting with LLMs in security work, both for defence and for attack simulation [117, 121]. If synthetic agent data does not transfer, people need to know that before they build on it.

1.3 Relevant Contemporary Issues

The project sits where several current debates meet. **Table 1.**2 summarises them and shows how each one touches our work.

Table 1.2: Contemporary issues connected to the project.

Issue	What is happening	Connection to this project
Offensive use of LLMs	Agents that plan and run attacks are now research tools [24, 33]	We study their footprints, not their success rate, and keep them inside our own testbed
LLMs that imitate systems	LLMs are used as fake shells to keep attackers busy [69, 79]	One of our two honeypot backends is exactly such an LLM shell
Training on synthetic data	Models trained on generated data can drift away from reality [95]	Our transfer test is a direct, domain-specific check of this risk
Botnets and cryptojacking	IoT and Linux botnets keep growing [22, 52]	The dominant real behaviour in our data is a cryptominer dropper
Data protection	India's DPDP Act 2023 and the IT Act 2000 govern personal data [43, 44]	The real dataset uses hashed IPs; we never try to re-identify anyone
Incident reporting	CERT-In requires cyber incidents to be reported within six hours [18]	Better automatic triage of honeypot sessions supports faster reporting
AI regulation and risk management	The EU AI Act and NIST AI RMF ask for documented, tested AI systems [31, 75]	We document data provenance, labelling and limitations in detail
Reproducibility of ML research	Leakage and weak evaluation inflate results [58]	We group folds by template, de-duplicate and publish all scripts

1.4 Problem Identification

Putting the background together, the problem can be stated in plain words:

Defenders want to train intent classifiers for honeypot sessions, but real labelled sessions are scarce and repetitive. LLM agents can generate unlimited synthetic sessions against a honeypot we control, but nobody has measured whether a model trained on those sessions works on real traffic, or how the two kinds of traffic differ.

During the early phase of the project we also found three practical problems in our own first prototype, which shaped the final design:

1. **Circular labels.** Our first classifier used keyword rules (for example, "contains `wget`" means *download*) both to create labels and, indirectly, as features. A model trained this way only re-learns the rules, so its scores look high but mean little [4].
2. **Fragile terminal input and output.** Cowrie's LLM backend sometimes replied slowly, so a naive reader cut replies short or mixed the tail of one reply into the next turn. A long-running command such as a recursive `find /` could also freeze the honeypot for minutes.
3. **A single persona.** All agents received the same "explore the system" instruction. This keeps the comparison clean, but it also means the synthetic data may simply be about a different kind of intruder than the bots that dominate real logs.

1.5 Task Identification

We broke the work into seven work packages. **Figure 1.**1 shows the work breakdown structure and **Table 1.**3 lists the tasks, their outputs and how we checked each output.

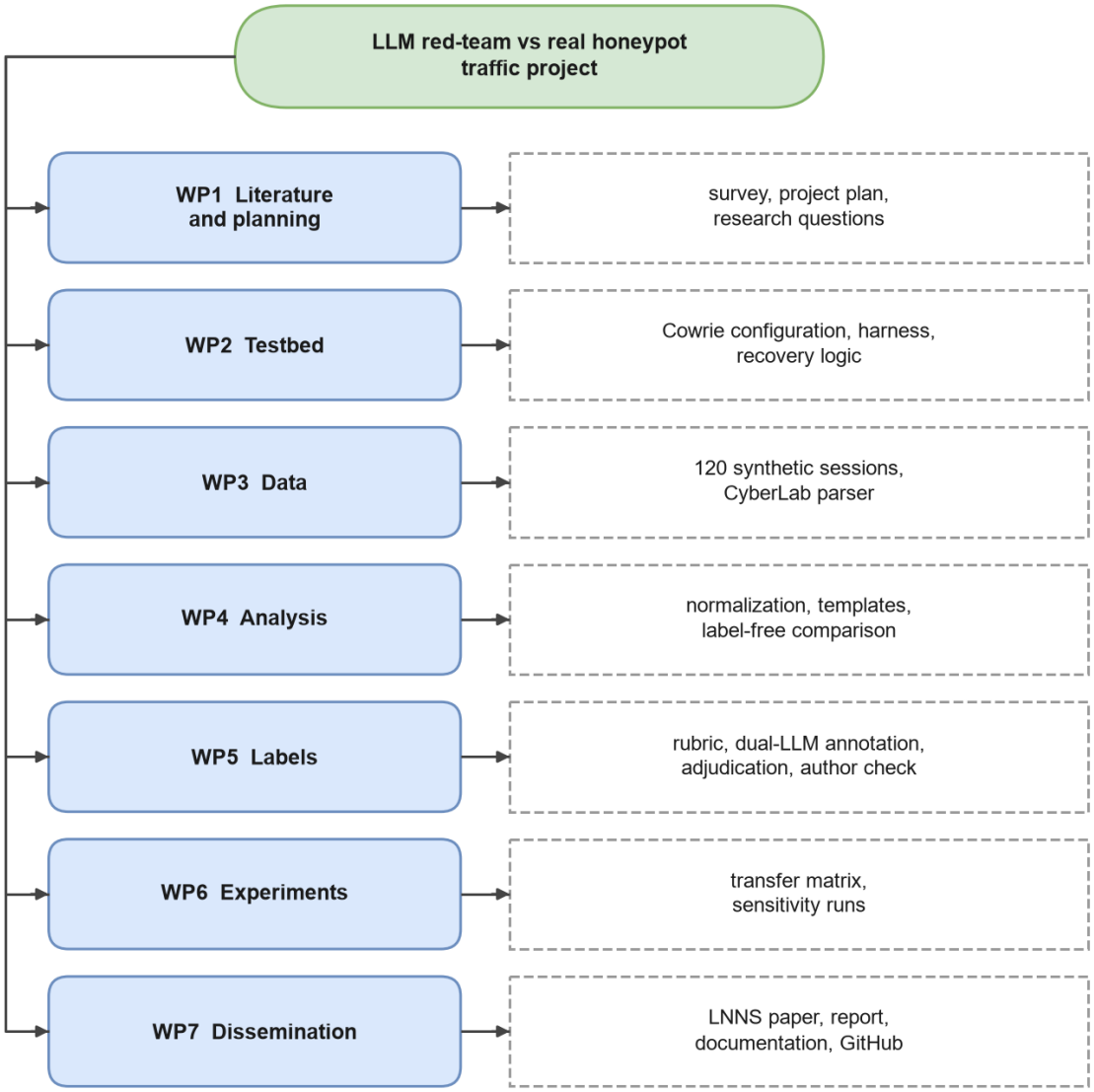


Figure 1.1: Work breakdown structure of the project.

Table 1.3: Task identification, deliverables and verification.

Work package	Main tasks	Deliverable	How it was verified
WP1 Literature and planning	Survey honeypots, IDS datasets, LLM agents, dataset shift; write research questions	Literature review, project plan	Supervisor review; gap analysis (Chapter 2)
WP2 Testbed	Configure Cowrie (static and LLM backends), build attacker harness, add recovery logic	`cowrie.cfg`, `better_ai_attacker.py`, Docker files	Banner probes, test sessions, infra event log
WP3 Data	Run 6 × 20 synthetic sessions; parse CyberLab	120 session logs; `real_organic.jsonl`	Session counts, status counts, parser cross-check (76,785 sessions)
WP4 Analysis	Normalise commands, build templates, label-free comparison	`domain_shift.md/.json`	Bootstrap confidence intervals; verb-only control
WP5 Labels	Rubric, blind sheets, two LLM annotators, adjudication, author check	`labels_final.csv`	Cohen's kappa; disagreement review
WP6 Experiments	Four-quadrant transfer test, sensitivity runs, plumbing check	`table3_transfer.md/.json`	Grouped cross-validation, de-duplicated test, majority baselines
WP7 Dissemination	Paper (IEEE, then Springer LNNS), this report, documentation, open repository	Paper, report, docs, GitHub	Number-by-number check against `results/`

1.6 Research Questions and Hypotheses

We framed the study around four research questions (RQ) and three testable hypotheses (H). Writing them down before running the final experiments helped us avoid moving the goalposts later.

Table 1.4: Research questions and hypotheses.

ID	Question or hypothesis	Where it is answered
RQ1	How do different LLM attackers behave against the same honeypot and prompt?	Section 4.4
RQ2	Without using any labels, how different are synthetic and real sessions?	Section 4.6
RQ3	Does an intent classifier trained on synthetic sessions transfer to real sessions? Does adding synthetic data help a model trained on real data?	Section 4.8
RQ4	Does an LLM-backed (adaptive) honeypot make agent traffic look more realistic?	Sections 4.4 and 4.6
H1	Synthetic and real command distributions are clearly different (JSD well above 0.5; C2ST AUC well above 0.5)	Supported (Section 4.10)
H2	A synthetic-trained classifier performs much worse on real data than a real-trained one	Supported
H3	The LLM backend reduces the distance between synthetic and real traffic for every attacker	Not supported (only for Gemma-3-4B)

1.7 Timeline

The project ran from August to October 2026. **Figure 1.**2 shows the schedule. The bars in blue are taken from the timestamps in our own log and result files (for example, all synthetic sessions were recorded between 24 and 29 September 2026); the grey bars are the planned windows for activities that leave no timestamped trace, such as reading papers.

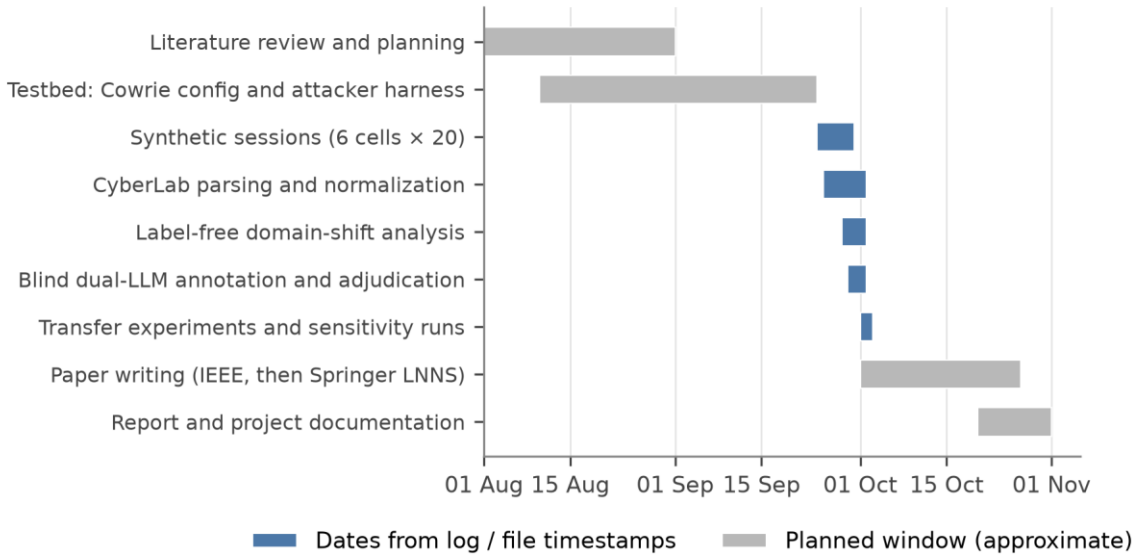


Figure 1.2: Project timeline (Gantt chart). Blue bars come from log and file timestamps; grey bars are approximate planned windows.

Table 1.5: Project schedule and milestones.

Phase	Period (2026)	Milestone
Literature review and planning	1–31 August	Research questions fixed; risks listed
Testbed development	10 August – 23 September	Harness with recovery logic; both Cowrie backends working
Synthetic data collection	24–29 September	120 sessions across 6 attacker × backend cells
Real data parsing and normalisation	25 September – 1 October	76,785 sessions parsed; 22 templates found
Label-free analysis	28 September – 1 October	`domain_shift.md` written
Annotation and adjudication	29 September – 1 October	$\kappa = 0.898$; `labels_final.csv`
Transfer and sensitivity experiments	1–2 October	`table3_transfer.md`, sensitivity and plumbing runs
Paper writing	1–26 October	Springer LNNS version prepared for ICICCD 2026
Report and documentation	20–31 October	This report and the project documentation

1.8 Contributions

The main contributions of this project are:

1. **An open, documented pipeline** that runs LLM attackers against Cowrie, records every turn with timing information, recovers from honeypot freezes, parses the CyberLab export correctly and regenerates every table and figure from scripts.
2. **A label-free comparison** of agent and real honeypot traffic, using verb-distribution divergence with bootstrap confidence intervals and a classifier two-sample test [67], which shows that the two are almost perfectly separable and explains why.
3. **A four-quadrant transfer experiment** with blind dual-LLM labels, template-grouped cross-validation and sensitivity checks, which shows that the synthetic sessions do not transfer and do not help.
4. **A practical warning** about chat-format token leakage from one attacker model, with a simple way to detect and strip it.
5. **A parsing pitfall** that affects anyone who reuses the CyberLab data: a parser written for Cowrie's native logs silently reads zero sessions.

1.9 Organisation of the Report

The rest of the report is organised as follows. Chapter 2 reviews the literature on honeypots, attack datasets, botnets, synthetic data, dataset shift, LLM agents, LLM

honeypots and LLM-based annotation, and ends with the problem definition and objectives. Chapter 3 describes the design process: the concepts we generated, the specifications and constraints, three alternative designs and the reasons we chose one of them, and the detailed implementation plan with flowcharts and algorithms. Chapter 4 presents the implementation, the data, all results and the validation steps. Chapter 5 concludes, discusses where results differed from our expectations and lists future work. The appendices contain a step-by-step user manual, the annotation rubric and data schemas, and the project's achievements.

CHAPTER 2
LITERATURE SURVEY

This chapter reviews the work that this project builds on. We start with a timeline of the problem and then go theme by theme: honeypots, the data they produce, the threats that dominate it, machine learning on intrusion data, synthetic data, LLMs and LLM agents, LLM-based honeypots, dataset shift, and LLM-based annotation. The chapter ends with a summary table of the solutions proposed by other researchers, a gap analysis, the formal problem definition and our objectives.

2.1 Timeline of the Reported Problem

The question "how do we get realistic attack data?" is almost as old as networked computing. **Figure 2.**1 places the key works on one time axis and **Table 2.**1 describes what each one changed.

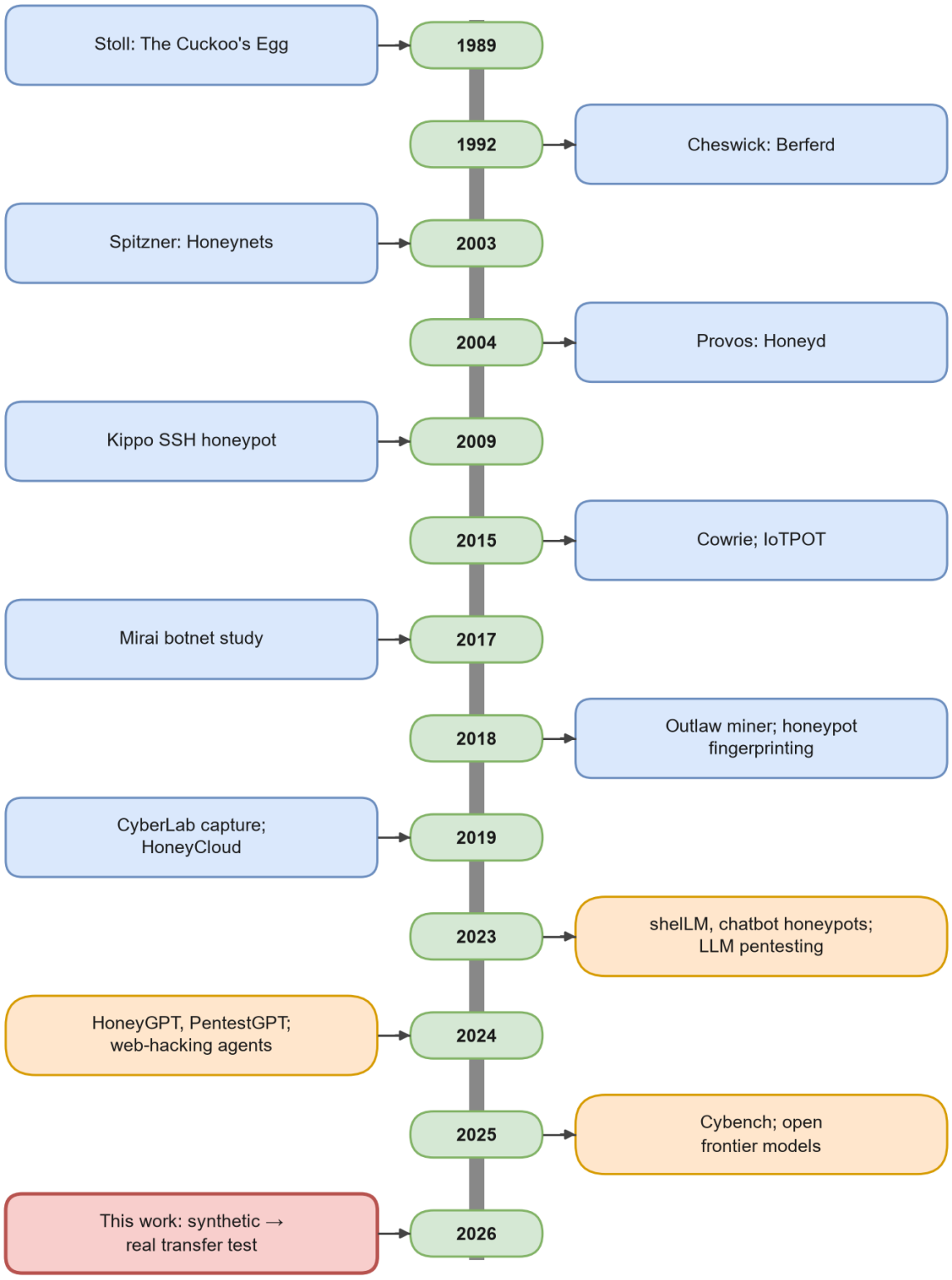


Figure 2.1: Timeline of work on honeypots, attack data and LLM agents, from 1989 to this project.

Table 2.1: Milestones in the history of the problem.

Year	Work	What changed
1989	The Cuckoo's Egg [102]	Showed that watching a live intruder reveals goals and methods that logs alone hide
1992	An Evening with Berferd [20]	First deliberate "jail" built to keep an attacker busy and observed
2000	DARPA/KDD critique [68]	Warned that synthetic intrusion benchmarks may not resemble real traffic
2003	Honeynet Project; Honeypots: Tracking Hackers [100, 101]	Gave honeypots a vocabulary, a taxonomy and an ethics discussion
2004	Honeyd [86]; Honeycomb [61]	Cheap virtual honeypots at scale; automatic signatures from honeypot traffic
2005	Internet Motion Sensor [7]	Large darknet monitoring; showed how much of the internet's noise is automated
2009	Kippo SSH honeypot [104]	Medium-interaction SSH shell emulation; ancestor of Cowrie
2010	Outside the Closed World [98]	Explained why ML-based intrusion detection fails outside the lab
2015	Cowrie [78]; IoTPOT [81]	Modern SSH/Telnet honeypot; first large IoT honeypot study
2017	Mirai [3]	Botnets built from IoT devices with default passwords dominate honeypot logs
2018	Outlaw [108]; Bitter Harvest [110]	Cryptomining botnets on Linux; honeypots can be fingerprinted at internet scale
2019	CyberLab capture [92]; HoneyCloud [22]	Public honeynet dataset; fileless attacks on Linux IoT devices
2022	Dos and Don'ts of ML in security [4]	Catalogue of pitfalls such as sampling bias and data snooping
2023	shelLM [96]; chatbot honeypots [69]; LLM pentesting [48]	LLMs used both to fake a shell and to attack one
2024	PentestGPT [24]; HoneyGPT [112]; web-hacking agents [34]	LLM agents and LLM honeypots become research tools
2025	Cybench [122]; open frontier models [23, 119]	Standard benchmarks for offensive agents; strong open-weight models
2026	This project	First measurement of whether agent-generated honeypot sessions transfer to real traffic

Two threads run through this history. One is about **collecting** better data (from Stoll's notebook to public honeynet datasets). The other is about **simulating** data, and every generation of simulated data has eventually been criticised for not looking like the real thing [30, 68, 105]. LLM agents are the newest entry in the simulation thread, and this project asks the same old question of them.

2.2 Honeypots: Concepts and Taxonomy

2.2.1 What a honeypot is

Spitzner [100] defines a honeypot as an information system resource whose value lies in unauthorised or illicit use of that resource. Because a honeypot has no legitimate users, it produces very little noise compared with a normal intrusion detection system: almost everything it sees is worth looking at. Surveys group honeypots by how much an attacker can do inside them, by their purpose (research or production) and by where they are deployed [14, 32, 37, 74].

Table 2.2: Honeypot interaction levels.

Level	What the attacker gets	Strengths	Weaknesses
Low interaction	A few emulated services or banners (for example Honeyd)	Safe, cheap, scales to thousands of IPs	Little detail on post-login behaviour; easy to fingerprint
Medium interaction	An emulated shell and file system (Kippo, Cowrie)	Records full command sequences and downloads with low risk	Emulation gaps can be detected [110]
High interaction	A real operating system, usually a VM	Most realistic behaviour	Expensive; risk of the honeypot being used to attack others
LLM-backed (adaptive)	A shell whose replies are written by an LLM	Can answer any command, adapts on the fly	Hallucinated state, latency, cost [96, 112]

2.2.2 Notable honeypot systems

Table 2.3 compares the systems most relevant to our work. Cowrie is the natural choice for SSH research: it is mature, widely deployed, logs to structured JSON and, in recent versions, can delegate command output to an LLM.

Table 2.3: Comparison of honeypot systems.

System	Type	Protocols	Notes relevant to this project
Honeyd [86]	Low interaction, virtual	Many (TCP/UDP personalities)	Showed that many virtual hosts can run on one machine
Honeycomb [61]	Signature generator	Network	Derived IDS signatures automatically from honeypot traffic
Kippo [104]	Medium interaction	SSH	Fake file system, session replay; development moved to Cowrie
Cowrie [78]	Medium interaction	SSH, Telnet	JSON event logs, file download capture, optional LLM backend; used here
IoTPOT [81]	IoT honeypot + sandbox	Telnet	Captured many DDoS malware families across CPU architectures
HoneyCloud [22]	Hardware + software IoT honeypots	SSH, Telnet	Studied fileless attacks over twelve months
T-Pot [25]	Multi-honeypot platform	Many	Bundles Cowrie and others with dashboards; popular in practice
shelLM [96]	LLM-generated shell	SSH	LLM writes responses; humans often could not tell it was fake
HoneyGPT [112]	LLM-generated shell	SSH	Targets the trade-off between flexibility, interaction depth and deception
LLM Honeypot [79]	Fine-tuned LLM shell	SSH	Fine-tuned an open model on attacker commands and responses

2.2.3 Honeypots under scrutiny

Honeypots are not invisible. Vetterl and Clayton [110] showed that low- and medium-interaction honeypots can be fingerprinted with a single packet because their network libraries differ subtly from real servers, and they found thousands of honeypot instances on the internet, many of them badly maintained. Barron and Nikiforakis [8] ran 102 honeypots for four months and found that bots behave the same way regardless of the environment, while human attackers react to what they find, for example running more commands when the file system looks realistic. Both findings matter here: if real traffic is mostly bots that ignore their surroundings, then making an agent's environment more realistic (our LLM backend) may not make the agent behave more like those bots.

2.3 What Real Honeypot Data Looks Like

2.3.1 Public datasets

Few honeypot datasets with full command sequences are public. The CyberLab honeynet dataset [92] is one of the best: it contains daily Cowrie logs from a research honeynet, with

source IPs replaced by hashed identifiers and released under a CC BY 4.0 licence. The HoneyCloud authors also published their SSH and Telnet logs [22]. Darknet sensors such as the Internet Motion Sensor [7] see much more traffic, but only connection attempts, not what happens after login.

2.3.2 Repetition and skew

A recurring observation is that honeypot traffic is dominated by a few automated campaigns. IoTPOT saw a handful of malware families account for most Telnet compromises [81]. Mirai alone infected hundreds of thousands of devices by trying default credentials [3], and its successors such as Hajime keep evolving their exploits [52]. This skew is good news for a defender who wants to block the common case, but it is a trap for a machine learning researcher: a random train/test split puts copies of the same bot script on both sides, and the classifier looks far better than it is. We return to this point in Chapter 3, where we group cross-validation folds by behaviour template.

2.4 The Threat Landscape Inside Honeypot Traffic

Understanding *who* produces real honeypot traffic explains much of our final result.

- **Scanners and brute-forcers.** Fast internet-wide scanners make it cheap to find every open SSH port [27]. Dictionary attacks with default credentials follow.
- **IoT botnets.** Mirai and its relatives log in with default passwords, fingerprint the device (often with `/bin/busybox` probes) and download a binary for the right CPU architecture [3, 52, 60].
- **Cryptomining groups.** The Outlaw group brute-forces SSH, changes the password, installs a miner and an SSH scanner and moves on [108]. As we show in Chapter 4, one Outlaw-style "dota" template accounts for 62% of all interactive sessions in our real data.
- **Fileless and living-off-the-land attacks.** Some attacks avoid writing files entirely and use built-in tools [22].
- **Rare human intruders.** Skilled humans and targeted attacks are rare, but they are what defenders fear most; zero-day attacks, for example, can stay undetected for months [10].

Frameworks such as the intrusion kill chain [54] and MITRE ATT&CK [103] give a shared language for these behaviours. We use ATT&CK tactics such as *discovery*, *credential access*, *persistence* and *resource hijacking* to describe sessions in this project.

2.5 Datasets and Machine Learning for Intrusion Detection

2.5.1 The long shadow of synthetic benchmarks

The intrusion detection community has been burned by synthetic data before. The DARPA 1998/1999 evaluations generated background traffic synthetically, and McHugh [68] showed that the result did not match real networks. The derived KDD Cup 99 dataset was later found to contain huge numbers of duplicates and other artefacts [105]. Newer datasets such as UNSW-NB15 [73] and CIC-IDS2017 [94] improved realism, yet CIC-IDS2017 itself turned out to contain labelling and feature-extraction errors [30]. The lesson is that a dataset is only as good as the process that produced it, and synthetic processes tend to leave fingerprints.

Table 2.4: Intrusion datasets and known issues.

Dataset	How it was produced	Known issues
DARPA 1998/99 [68]	Synthetic background + scripted attacks	Traffic statistics unlike real networks
KDD Cup 99 / NSL-KDD [105]	Features from DARPA data	Massive redundancy; biased evaluation
UNSW-NB15 [73]	Testbed with traffic generator	Still a lab environment
CIC-IDS2017 [94]	Testbed with profiled users	Labelling and flow-extraction errors [30]
CyberLab [92]	Real Cowrie honeynet	Small, repetitive; one network; hashed IPs

2.5.2 Why ML-based detection is hard

Sommer and Paxson [98] gave several reasons why machine learning works well for spam or image recognition but struggles in intrusion detection: attacks are rare, errors are expensive, the gap between what the model outputs and what an operator can act on is wide, and evaluation is hard because good data is scarce. Buczak and Guven [16] surveyed hundreds of ML methods for intrusion detection and noted the same dependence on a few old datasets. Arp et al. [4] went further and listed ten common pitfalls in security ML papers, including sampling bias, label inaccuracy, data snooping and inappropriate

baselines. Kapoor and Narayanan [58] showed that data leakage of this kind has caused a reproducibility crisis across ML-based science.

Two related problems from malware classification are directly relevant. Pendlebury et al. [83] showed that random splits that ignore time inflate scores, and proposed evaluating "across space and time". Jordaney et al. [57] showed that real data drifts and that detectors need ways to notice when inputs stop looking like the training data. Our use of template-grouped folds and label-free shift measures follows from these papers.

2.5.3 Methods we rely on

Our classifiers are deliberately simple and well understood. Text features come from TF-IDF weighting [91, 99], and the main classifier is a random forest [13] with balanced class weights, which is a common and robust response to class imbalance [51]; resampling methods such as SMOTE [19] are an alternative we did not need. We report macro-averaged F1, which treats rare classes as seriously as common ones [97], together with accuracy and majority-class baselines. All of this is implemented with scikit-learn [82].

2.6 Synthetic Data for Security

Generative models have been used to create network traffic for a decade. Generative adversarial networks, or GANs [42], can produce realistic-looking flow records [90], and IDSGAN modified attack records so that they slipped past detectors [65]. These methods need a lot of real data to learn from, which is exactly what is missing in the honeypot setting.

Robotics offers a different idea. Domain randomisation trains a model on many randomised simulations so that the real world looks like just another variation [106]. This only works if the simulations cover the real behaviour; a simulator that always produces the same kind of scene will not transfer. Our single-persona design is, in this sense, a simulator with no randomisation, and our results show the consequence.

A newer concern is what happens when models are trained on model-generated data. Shumailov et al. [95] showed that models trained recursively on generated data gradually lose the tails of the real distribution ("model collapse"). Honeypot traffic is all tails: rare commands from rare campaigns. Synthetic data that captures only the most typical behaviour would be particularly harmful here.

2.7 Large Language Models and Agents

2.7.1 The models

Modern LLMs are built on the transformer architecture [109] and trained on very large text corpora. Scaling them up produced strong few-shot abilities [15], and instruction tuning with human feedback made them follow instructions reliably [80]. Safety training, for example with a written constitution [6], makes models refuse clearly harmful requests, which is one reason why research harnesses must frame tasks carefully and why we record refusals as a separate outcome. Bommasani et al. [12] describe these models as "foundation models" whose behaviour is hard to predict from their training alone, and Weidinger et al. [115] catalogue the social and ethical risks they bring.

The four models used in this project come from four different vendors and span a wide size range: DeepSeek-V3.2 [23], Gemma 3 at 4 billion parameters [40], gpt-oss-120b [77] as attackers, and Qwen3-32B [119] as the LLM honeypot backend.

2.7.2 Agents, reasoning and their failure modes

An LLM becomes an *agent* when it is placed in a loop: it observes the environment, decides on an action, sees the result and decides again. Chain-of-thought prompting [114] and the ReAct pattern of interleaved reasoning and acting [120] made such loops effective. Agents also inherit LLM weaknesses. They hallucinate facts [56], they can be steered by malicious text in their inputs [45] and they can leak memorised training data [17]. In our setting a related problem appeared: one model leaked its own chat-format control tokens and private reasoning into the shell, as Chapter 4 describes.

2.7.3 Surveys of LLMs in security

Two recent surveys map the use of LLMs in cybersecurity. Xu H. et al. [117] systematically reviewed over a hundred papers on LLMs for security tasks, and Yao Y. et al. [121] grouped the literature into beneficial uses, offensive uses and vulnerabilities of LLMs themselves. Neither survey reports a study that checks whether agent-generated attack data resembles real attack data.

2.8 LLM Agents for Offensive Security

Table 2.5: LLM agents for offensive security.

Work	What the agent does	Main finding
Happe and Cito [48]	GPT-3.5 plans attacks and proposes commands run over SSH on a vulnerable VM	LLMs are useful "sparring partners" for penetration testers
Happe, Kaplan and Cito [49]	hackingBuddyGPT attempts Linux privilege escalation on benchmark VMs	Strong models exploit many single-vulnerability machines; small local models far fewer
Deng et al. [24]	PentestGPT splits work into reasoning, generation and parsing modules	Modular design improves task completion on benchmarks
Fang et al. [34]	GPT-4 agent attacks test websites	Agents can find and exploit web vulnerabilities without human help
Fang et al. [33]	Agent exploits known (one-day) CVEs given their descriptions	High success when the vulnerability description is available
Xu J. et al. [118]	AutoAttacker automates post-breach steps	LLMs can chain post-exploitation actions
Shao et al. [93]	NYU CTF Bench: 200 capture-the-flag challenges	Standard benchmark for offensive LLM evaluation
Zhang et al. [122]	Cybench: professional-level CTF tasks	Measures capabilities and risks of LLM agents

All of these works ask the same question: *can the agent succeed?* None of them asks whether the agent's footprints look like those of real intruders, which is the question that matters if the footprints are to be used as training data. The term "red teaming" is also used for a different activity, testing LLMs themselves with adversarial prompts [38, 85]; in this report, "red-team agent" always means an LLM that plays the attacker against our honeypot.

2.9 LLM-Based Honeypots

LLMs can also sit on the defender's side. McKee and Noever [69] showed that ChatGPT can imitate Linux, Mac and Windows terminals well enough to be a honeypot interface. Sladić et al. [96] built shelLM, an LLM-generated SSH shell, and found that many human testers could not tell it apart from a real system. HoneyGPT [112] aimed to balance flexibility, interaction depth and deception. Otal and Canbaz [79] fine-tuned an open model on attacker commands and responses for a more faithful shell, and Weber, Feger and Pilgermann [113] proposed a unified way to evaluate such honeypots.

Table 2.6: LLM-based honeypots.

Work	Approach	Evaluation
McKee and Noever [69]	Prompted ChatGPT to act as terminals and tools	Ten feasibility tasks
Sladić et al. [96]	shelLM: LLM writes every response	Human testers judged realism
Wang et al. [112]	HoneyGPT shell architecture	Compared with existing honeypots on engagement
Otal and Canbaz [79]	Fine-tuned open model on Cowrie-style command/response pairs	Similarity metrics and live deployment
Weber, Feger and Pilgermann [113]	Unified evaluation for LLM honeypots	Evaluated GPT-3.5 on datasets of varying difficulty

These papers judge an LLM honeypot by how well it deceives. We use one differently: as a *target* for an LLM attacker, to see whether a richer, adaptive environment changes the agent's behaviour enough to look more like real traffic.

2.10 Dataset Shift and Two-Sample Testing

When a model is trained on one distribution and tested on another, the situation is called dataset shift [88]. Moreno-Torres et al. [72] organised the many kinds of shift into a unifying framework: covariate shift (the inputs change), prior or label shift (the class balance changes) and concept shift (the meaning of a label changes). Ben-David et al. [9] proved that the error on the target domain is bounded by the error on the source domain plus a measure of how distinguishable the two domains are. That theory gives a practical rule: if a classifier can easily tell the two domains apart, transfer is in trouble. Lipton, Wang and Smola [66] showed how to detect and correct label shift with black-box predictors.

Several tools measure shift without labels. Rabanser, Günnemann and Lipton [89] compared them and found that simple methods based on two-sample tests work well. The classifier two-sample test (C2ST) of Lopez-Paz and Oquab [67] trains a classifier to separate the two samples; an AUC near 0.5 means they are indistinguishable. Kernel methods such as the maximum mean discrepancy offer an alternative [46]. For discrete distributions such as command frequencies, the Jensen-Shannon divergence [64] is a natural choice: it is symmetric, bounded between 0 and 1 in base 2, and its square root is a true metric [29]. It is built from the Kullback-Leibler divergence [62]. Set overlap can be

summarised by the Jaccard index [55], and uncertainty on any of these statistics can be estimated with the bootstrap [28].

Table 2.7: Methods for detecting dataset shift.

Method	Idea	Use in this project
Jensen-Shannon divergence [64]	Symmetric distance between two distributions	Distance between synthetic and real verb distributions
Bootstrap CI [28]	Resample to estimate uncertainty	95% confidence interval for every JSD value
Jaccard index [55]	Overlap between two sets	Overlap of the two verb vocabularies
C2ST [67]	Train a classifier to tell samples apart	AUC of synthetic vs real, on commands and on verbs only
MMD [46]	Kernel distance between samples	Considered; C2ST preferred for interpretability
Label-shift correction [66]	Estimate and reweight class priors	Not applicable: the label sets barely overlap

2.11 LLMs as Annotators and Measuring Agreement

Labelling sessions by hand is slow, and LLMs are increasingly used as annotators. Gilardi, Alizadeh and Kubli [41] found that ChatGPT beat crowd workers on several text annotation tasks, and Törnberg [107] reported similar results for political tweets. Ziems et al. [124] tested 13 models on 25 social science benchmarks and concluded that LLMs are useful zero-shot annotators but do not replace careful human work. In the evaluation of chat assistants, Zheng et al. [123] showed that strong LLMs agree with human preferences most of the time when used as judges, while also documenting biases such as favouring their own outputs.

Agreement between annotators is measured with chance-corrected statistics. Cohen's kappa [21] handles two annotators; Fleiss' kappa [36] generalises it to many. Landis and Koch [63] proposed the widely used verbal scale in which values above 0.81 indicate "almost perfect" agreement, and Artstein and Poesio [5] discuss the pitfalls of these measures in detail. We use two blind LLM annotators from one provider, which makes kappa meaningful but also means they may share biases. Chapter 4 discusses this limitation openly.

2.12 Reproducible and Responsible Research

Reproducibility means that someone else can rerun our analysis and get the same numbers [84]. Data should be findable, accessible, interoperable and reusable [116], and datasets

and models should come with documentation of how they were made and what they should not be used for [39, 71]. Our work uses open, widely tested libraries [50, 53, 82, 111] and containers for deployment [70], and our charts use a colour-blind-friendly palette [76].

Ethically, security research involving real systems and people is guided by the Menlo Report's principles of respect for persons, beneficence, justice and respect for law [26] and by professional codes such as the ACM Code of Ethics [1]. In India, the Information Technology Act 2000 and the DPDP Act 2023 set the legal frame [43, 44]. Our design choices, such as attacking only our own honeypot and using only hashed public data, follow from these principles.

2.13 Summary of Solutions Proposed by Different Researchers

Table 2.8 gathers the most relevant proposed solutions in one place, with the limitation of each that matters for our problem.

Table 2.8: Proposed solutions by different researchers and their limitations for this problem.

Researchers	Proposed solution	Data used	Limitation for our problem
Spitzner [100]; Provos [86]	Deploy honeypots to collect real attacker data	Live deployments	Slow, small, repetitive data
Pa et al. [81]; Dang et al. [22]	Large IoT honeypot ecosystems	Months of real traffic	Expensive; still dominated by a few families
Sedlar et al. [92]	Publish a real honeynet dataset	Real Cowrie logs	One network; few interactive sessions
Kreibich and Crowcroft [61]	Generate signatures automatically from honeypot traffic	Honeypot captures	Signatures, not intent labels
Sharafaldin, Lashkari and Ghorbani [94]	Build a testbed with profiled synthetic users	Lab traffic	Lab artefacts and labelling errors
Ring et al. [90]; Lin, Shi and Xue [65]	Generate traffic with GANs	Existing flow datasets	Needs large real datasets; flows, not shell sessions
Tobin et al. [106]	Randomise simulations so reality is "one more variation"	Simulated robotics scenes	Requires wide, randomised coverage
Happe and Cito [48]; Deng et al. [24]	LLM agents for penetration testing	Vulnerable VMs, benchmarks	Measure success, not realism
Sladić et al. [96]; Wang et al. [112]	LLM-generated honeypot shells	Human or agent attackers	Measure deception, not data quality
Lopez-Paz and Oquab [67]; Rabanser, Günnemann and Lipton [89]	Detect shift with two-sample tests	Generic ML data	Never applied to agent vs real honeypot traffic
Gilardi, Alizadeh and Kubli [41]; Ziems et al. [124]	Use LLMs as annotators	Text corpora	Not tested on shell sessions
Arp et al. [4]; Pendlebury et al. [83]	Rules for sound security ML evaluation	Many security datasets	General guidance, not a transfer study

2.14 Research Gap Analysis

Table 2.9 crosses the main capabilities a study would need against the most relevant prior works. A tick means the work provides that capability.

Table 2.9: Research gap matrix (✓ = provided, – = not provided).

Work	Real shell sessions	LLM attacker	LLM honeypot	Label-free shift test	Transfer test	Open pipeline
Barron and Nikiforakis [8]	✓	–	–	–	–	–
Sedlar et al. [92]	✓	–	–	–	–	✓
Happe, Kaplan and Cito [49]	–	✓	–	–	–	✓
Deng et al. [24]	–	✓	–	–	–	✓
Sladić et al. [96]	–	–	✓	–	–	✓
Wang et al. [112]	–	–	✓	–	–	–
Otal and Canbaz [79]	✓	–	✓	–	–	–
Rabanser, Günnemann and Lipton [89]	–	–	–	✓	–	✓
This project	✓	✓	✓	✓	✓	✓

The gap is clear: honeypot studies describe real attackers, agent studies measure attack success, LLM-honeypot studies measure deception and shift-detection studies use generic data. No prior work puts an LLM attacker against both a static and an LLM-backed honeypot, compares its sessions with real honeypot sessions without labels, and then tests whether a model trained on one transfers to the other.

2.15 Summary Linking the Literature to This Project

The literature gives us both the motivation and the method. The honeypot and botnet papers tell us what real traffic is made of: a few automated campaigns that do not adapt to their environment. The intrusion-dataset papers warn that synthetic data has repeatedly failed to look real, and they teach us how to avoid inflated evaluations. The LLM-agent papers show that agents can act like intruders, and the LLM-honeypot papers show that a honeypot can talk back intelligently. The dataset-shift papers give us tools to measure the gap without labels, and the annotation papers give us a way to obtain labels when human time is limited, together with a way to measure how much to trust them. This project combines these pieces to answer a question none of them answers alone.

2.16 Problem Definition

Let S be the set of synthetic sessions produced by LLM attackers against our honeypot, and R the set of real interactive sessions in the CyberLab data. Each session is a sequence of

shell commands. Let f be a classifier that maps a session to one of eight intent labels. The project must:

1. measure, **without labels**, how far the distribution of S is from that of R, with uncertainty estimates;
2. obtain **trustworthy intent labels** for both sets without circular keyword rules;
3. estimate the performance of f when it is **trained on S and tested on R**, and compare it with training on R and with training on $R \cup S$, using an evaluation that does not leak repeated templates between training and test data; and
4. explain any gap in terms that a defender can act on.

2.17 Goals and Objectives

The **goal** of the project is to give defenders an evidence-based answer to the question "can LLM red-team agents generate realistic honeypot training data?".

The **objectives** are written with the action verbs of Bloom's revised taxonomy [2, 11], so that each one states the level of thinking it demands, from *apply* up to *create*. The project will:

1. **O1 (Create).** *Design and build* a reliable testbed in which several LLM attackers interact with Cowrie under two backends, with complete per-turn logging and automatic recovery from honeypot freezes.
2. **O2 (Apply).** *Implement* a correct parser for the CyberLab real data and *apply* one normalisation to both corpora, reducing them to comparable behaviour templates.
3. **O3 (Analyse).** *Compare* the synthetic and real corpora without labels and *differentiate* them by structure, tactics and command verbs, using JSD with bootstrap confidence intervals, Jaccard overlap, coverage and a classifier two-sample test.
4. **O4 (Evaluate).** *Judge* the reliability of blind, independent intent labels by measuring their agreement, *resolve* disagreements transparently and *verify* the most influential labels.
5. **O5 (Evaluate).** *Assess* whether a classifier trained on synthetic sessions transfers to real sessions, and *critique* the result with template-grouped folds, de-duplicated tests, majority baselines and sensitivity checks.
6. **O6 (Create).** *Produce* and *publish* complete documentation of the code, data provenance, design decisions and limitations, as a research paper, this report and open project documentation.

Table 2.10: Objectives with their Bloom level, research questions and where they are reported.

Objective	Summary	Bloom level	Research question	Reported in
O1	Design and build the testbed and harness	Create	RQ1, RQ4	Sections 3.7, 4.1, 4.4
O2	Implement parsing and normalisation	Apply	RQ2	Sections 3.7, 4.3
O3	Compare the corpora without labels	Analyse	RQ2, RQ4	Section 4.6
O4	Judge and verify blind dual-LLM labels	Evaluate	RQ3	Section 4.7
O5	Assess transfer between corpora	Evaluate	RQ3	Sections 4.8, 4.9
O6	Produce and publish documentation	Create	All	Sections 3.8, 4.14, Appendices

CHAPTER 3
DESIGN FLOW AND PROCESS

This chapter explains how we went from the problem in Chapter 2 to a working research system. We first generated several concepts, then wrote down the specifications a good solution must meet, listed the constraints we had to respect, compared three alternative designs and chose one. The second half of the chapter is the implementation plan: block diagrams, flowcharts, algorithms and the exact settings used in every step.

3.1 Concept Generation

We began with one question: *how can a defender get enough labelled attack sessions?* Brainstorming produced six concepts, shown in **Figure 3.**1 and compared in **Table 3.**1.

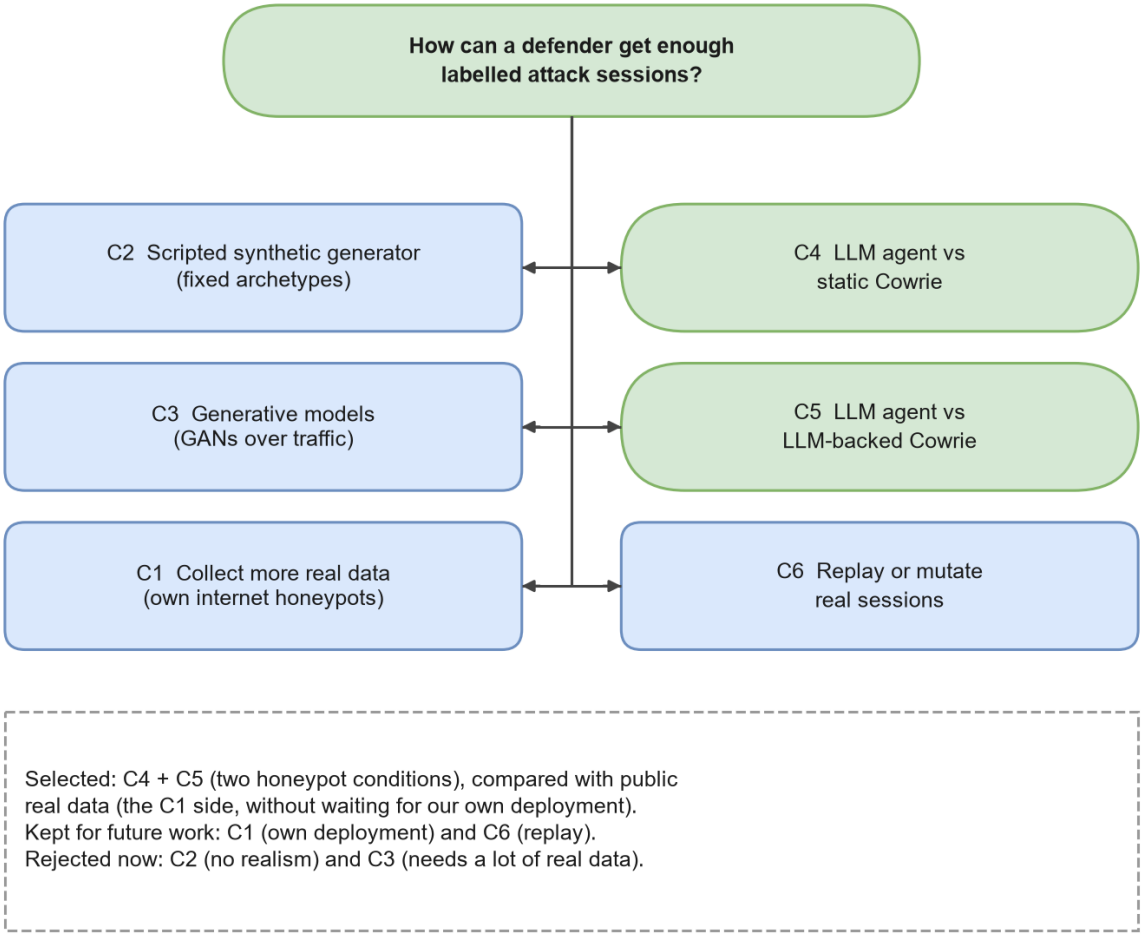


Figure 3.1: Concept generation: six ways to obtain labelled attack sessions.

Table 3.1: Concepts considered and their trade-offs.

Concept	Advantages	Disadvantages
C1 Collect more real data with our own internet-facing honeypots	Most realistic data	Months of collection; legal and operational risk; still repetitive [8]
C2 Scripted synthetic generator with fixed attack "archetypes"	Free labels (the script knows its archetype); fast	Data reflects only what we wrote; nothing new is learned
C3 Generative models (for example GANs) trained on traffic	Can produce many variations [90]	Needs a large real training set, which is exactly what is missing
C4 LLM agent attacking a static Cowrie honeypot	Cheap, unlimited, fully logged, contained	Unknown realism; agent may behave like a CTF player, not a bot
C5 LLM agent attacking an LLM-backed Cowrie honeypot	Same as C4, with a richer environment that answers any command	Hallucinated shell state; slower; extra cost [96]
C6 Replay or mutate real sessions	Stays close to real behaviour	Mutations are hard to label; may only copy the same few templates

The concepts that best match the stated goal are C4 and C5: they are the "LLM red-team" idea in its two natural forms. They need a source of real data to compare against; rather than waiting months for C1, we used a public real dataset [92]. C2 and C3 were rejected for the reasons in the table, and C6 was kept for future work.

3.2 Evaluation and Selection of Specifications and Features

3.2.1 Functional requirements

Table 3.2: Functional requirements of the research system.

ID	Requirement	Implemented by
F1	Drive an interactive SSH shell with an LLM, one command per turn	`better_ai_attacker.py`
F2	Support several LLM providers through one interface	`call_llm()` dispatcher
F3	Run the same attacker against a static and an LLM-backed honeypot	Cowrie `backend = shell` / `backend = llm`
F4	Log every turn: command, output, timestamp, latency, and the session outcome	Per-session JSON log
F5	Detect and survive honeypot freezes without losing a whole batch	Banner probe, stall detection, retries
F6	Parse the CyberLab export into per-session records	`parse_cyberlab.py`
F7	Normalise commands so that the same script from different IPs becomes one template	`session_features.py`
F8	Compare corpora without labels	`domain_shift.py`
F9	Produce blind annotation sheets and measure agreement	`make_annotation_sheet.py`, `agreement.py`
F10	Run the four-quadrant transfer experiment with baselines	`train_eval_classifier.py`
F11	Regenerate every table and figure from scripts	`generate_paper_tables.py`, `make_figures.py`

3.2.2 Non-functional requirements

Table 3.3: Non-functional requirements.

Quality	Requirement	How we met it
Validity	No circular labels; no leakage of templates between train and test	Dual-LLM labels; GroupKFold by template; de-duplicated test
Reproducibility	Same inputs give the same numbers	Fixed seeds (42); scripts write all results to files
Containment	The attacker can reach only our honeypot	Single configured target; no scanning code
Privacy	No re-identification of real sources	Hashed IPs only; no enrichment
Robustness	A frozen honeypot must not corrupt or kill a batch	Idle-timeout reads, stall detection, retries, infra log
Scalability	The 600 MB real file must not need to fit in memory	Line-by-line streaming parser
Transparency	Every reported number traceable to a file	`results/` folder; this report cites it
Cost	Stay within a student budget	Small open-weight models via a pay-per-use endpoint; 20 sessions per cell

3.2.3 Weighted decision matrix

To compare the concepts in a structured way we scored each one from 1 (poor) to 5 (excellent) on five criteria and weighted the criteria by importance. The scores are our own engineering judgement, made before the experiments, and are shown so that the reasoning is visible.

Table 3.4: Weighted decision matrix for the six concepts (higher is better).

Criterion (weight)	**C1**	**C2**	**C3**	**C4**	**C5**	**C6**
Answers the research question (0.30)	2	1	2	5	5	3
Feasible in the project time (0.25)	1	5	2	4	4	3
Cost (0.15)	2	5	3	4	3	4
Safety and ethics (0.15)	2	5	5	5	5	4
Novelty (0.15)	2	1	2	4	5	2
Weighted score	**1.75**	**3.10**	**2.60**	**4.45**	**4.45**	**3.15**

C4 and C5 tie at the top, which is why the final design uses both as two experimental conditions rather than choosing one.

3.3 Design Constraints

The format of this report asks us to consider a broad set of constraints. **Table 3.**5 lists each one and the design decision it forced.

Table 3.5: Design constraints and how the design responds to them.

Constraint	What it means here	Design response
Regulations	IT Act 2000, DPDP Act 2023, CERT-In directions [18, 43, 44]; dataset licence CC BY 4.0	Attack only our own system; use only the public, hashed dataset; credit the dataset authors
Economic	API calls cost money; no budget for paid annotators	Small and mid-size models through one pay-per-use endpoint; 120 sessions; LLM annotation with author verification instead of paid experts
Environmental	LLM inference uses energy	Short sessions (turn and time limits); no model training; small classifiers on a laptop
Health	Long hours reading malicious command logs	Automation of labelling; scripts instead of manual counting
Manufacturability (deployability)	Others must be able to rebuild the system	Docker Compose file, pinned requirements, one config file for Cowrie
Safety	An attacking agent must not escape the testbed	No scanning or target discovery; target host is a fixed setting; honeypot holds no real data
Professional	ACM Code of Ethics [1]; honest reporting	Report negative results; label LLM annotations as such; keep failed sessions in the data
Ethical	Menlo Report principles [26]	No deception of real people; no re-identification; no release of new attack capability
Social and political	Public concern about offensive AI [115]	Study footprints, not attack success; publish defensive findings

Three constraints shaped the design most. **Safety** ruled out any design in which the agent could choose its own targets, so the harness connects to exactly one configured host and has no scanning code. **Economics** ruled out paid human annotation at the scale we needed, which led to the blind dual-LLM protocol with author verification. **Professional honesty** led us to keep every failed session (connection lost, LLM error, refusal) in the data, because failure rates are themselves a finding.

3.4 Analysis and Feature Finalisation Subject to Constraints

Table 3.6 shows which candidate features made it into the final system, and why the others were dropped.

Table 3.6: Feature finalisation.

Candidate feature	Kept?	Reason
One fixed "explore the system" persona for all agents	Yes	Isolates model and backend effects; persona diversity is future work
Several attacker models of different size and vendor	Yes	Shows whether a finding is one model's quirk
Static and LLM-backed honeypot conditions	Yes	Directly tests RQ4
Output-marker injection to detect end of reply	No	The LLM backend parsed the marker as part of the command and leaked it
Idle-timeout reading (stop after 1.5 s of silence)	Yes	Works for both fast static and slow LLM replies
Automatic honeypot restart over admin SSH	Optional	Used only when an admin connection is configured; otherwise wait for a watchdog
Keyword-rule labels as ground truth	No	Circular with the features (Design A below)
Blind dual-LLM labels with adjudication	Yes	Feasible, measurable agreement, no keyword circularity
Response text as a classifier feature	No	Real Cowrie logs do not record command output, so it is not comparable
Random train/test splits	No	Would put copies of one bot script on both sides
Template-grouped cross-validation	Yes	Prevents leakage of repeated scripts

3.5 Design Flow: Alternative Designs

We worked through three complete designs, shown in **Figure 3.**2.

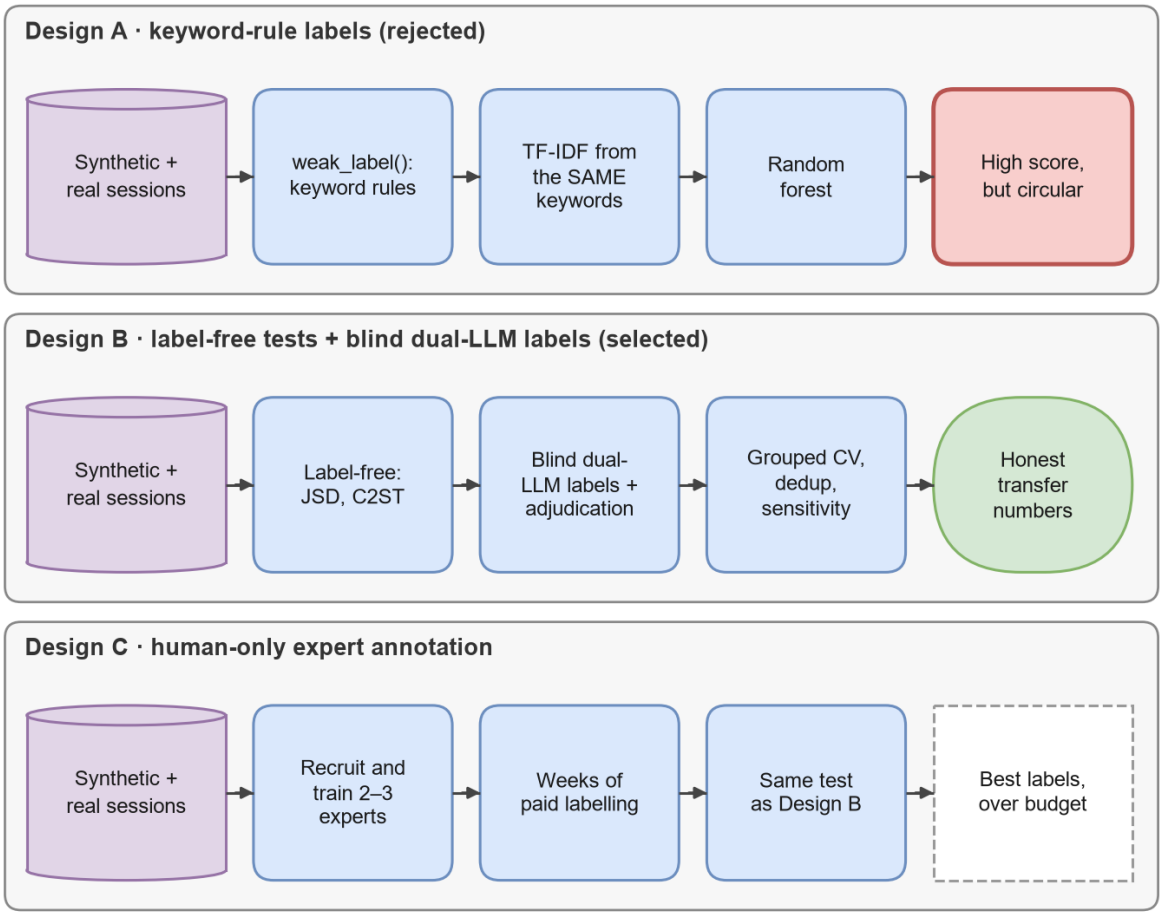


Figure 3.2: Three alternative designs. Design A was our first prototype; Design B is the final design; Design C is the ideal but unaffordable option.

Design A (keyword-rule labels). Our first prototype labelled every session with simple rules: if a command contains `wget` or `curl`, the session is a *download*; if it touches `/etc/shadow`, it is *credential access*; and so on. A TF-IDF random forest was then trained to predict these labels. The scores looked good, but the design is circular: the TF-IDF features contain exactly the keywords that defined the labels, so the classifier only rediscovers the rules. Worse, the rules mislabel real behaviour. Mirai-style bots run a bare `tftp; wget` with no argument just to check whether the tools exist; a keyword rule calls that a download, while a human reader sees reconnaissance. We kept Design A only as a *plumbing check* that the code runs end to end, and we label its numbers as such everywhere.

Design B (label-free tests plus blind dual-LLM labels). The final design answers the realism question twice. First, without any labels, by measuring how different the command distributions are and whether a classifier can tell the corpora apart. Second, with intent labels that do not come from keywords: two LLM annotators label every distinct session blind to its source, an adjudicator resolves disagreements and the authors cross-verify the

result. The transfer experiment then uses template-grouped folds, a de-duplicated test set, majority baselines and a sensitivity run on the most influential label.

Design C (human experts). The ideal design would replace the LLM annotators with two or three trained security analysts outside the author team. It gives the most defensible labels but needs recruitment, training and weeks of paid time, which the project could not afford. We designed Design B so that Design C can be added later without changing anything else: the blind sheets, rubric and agreement script already support human annotators.

3.6 Best Design Selection

Table 3.7 compares the three designs on the criteria that matter most.

Table 3.7: Comparison of the alternative designs.

Criterion	**Design A**	**Design B**	**Design C**
Labels independent of features	No (circular)	Yes	Yes
Answers realism without any labels	No	Yes	Yes
Agreement can be measured	No	Yes (κ)	Yes (κ)
Cost	Very low	Low	High
Time to complete	Days	About a week	Several weeks
Risk of shared bias in labels	Total	Moderate (one provider)	Low
Extensible to human labels later	No	Yes	Already human
Decision	Rejected (kept as plumbing check)	**Selected**	Future work

Design B was selected because it is the only design that is both valid and feasible. Its main weakness, that both annotators come from one model provider, is handled in three ways: the two annotators are different models (Claude Sonnet 5.5 and Claude Opus 5.5) working independently, the authors cross-verify the final labels, and the most consequential label is tested in a sensitivity run. Chapter 4 reports the remaining limitation honestly.

3.7 Implementation Plan

3.7.1 Overall architecture and method

Figure 3.3 shows the complete system in three blocks: synthetic data generation in our own testbed, data preparation for both corpora, and analysis. **Figure 3.**4 shows the nine stages of the research method in order.

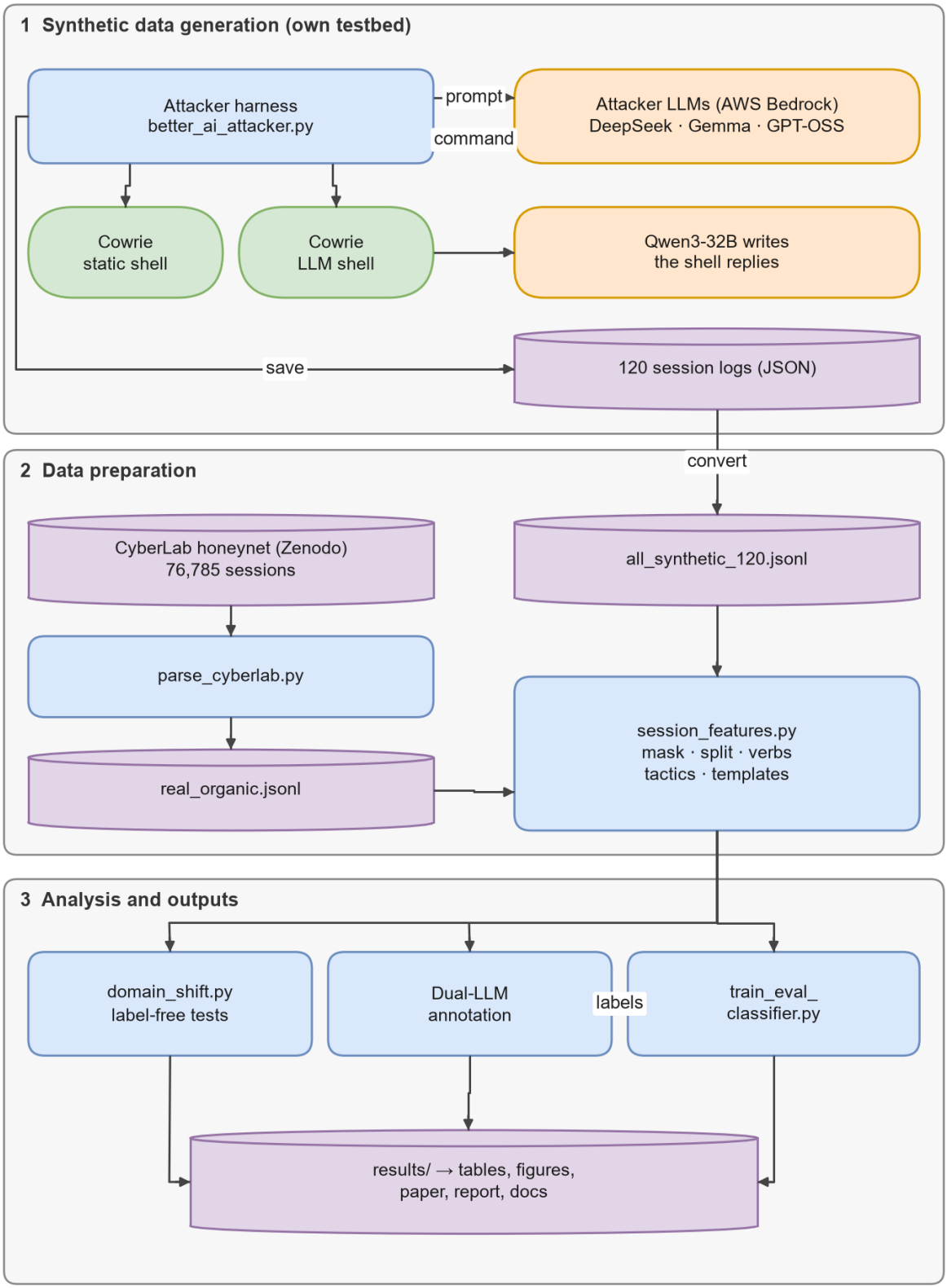


Figure 3.3: System architecture: synthetic data generation, data preparation and analysis.

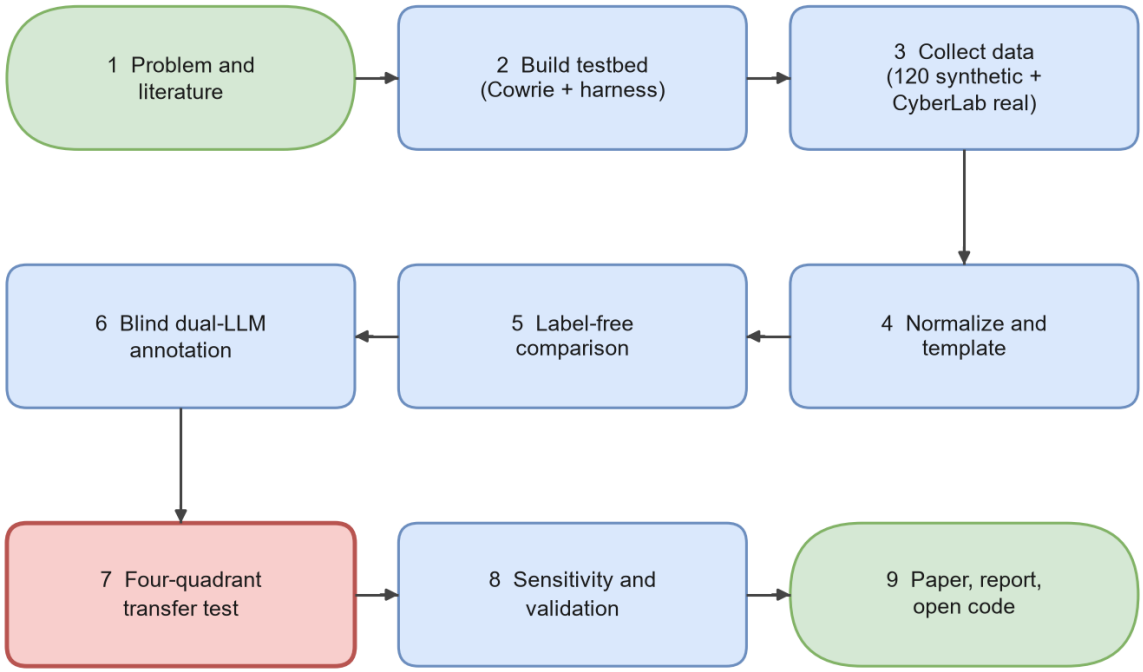


Figure 3.4: Research methodology in nine stages.

3.7.2 Testbed configuration

The honeypot is Cowrie [78], configured as shown in **Table 3.**8. The same configuration file switches between the two experimental conditions by changing one line.

Table 3.8: Cowrie honeypot configuration (from cowrie_etc/cowrie.cfg; secrets omitted).

Setting	Value	Purpose
hostname	svr04	Name shown in the fake prompt
SSH listen endpoint	tcp:2222	Port the attacker connects to
SSH version banner	SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5	Looks like a common Ubuntu server
Telnet	disabled	Only SSH is studied in the testbed
Fake kernel	5.4.0-42-generic, linux-x64-lsb	What `uname` reports
interactive_timeout	300 s	Idle sessions are closed
tty logging	enabled	Full terminal transcripts kept
backend (Condition A)	shell	Cowrie's built-in emulated commands
backend (Condition B)	llm, model qwen.qwen3-32b, temperature 0.7, max_tokens 512	An LLM writes every reply

The three attacker models were reached through one OpenAI-compatible endpoint, which kept the request format identical across vendors. **Table 3.**9 summarises the models.

Table 3.9: Language models used in the project.

Model	Role	Vendor	Why it was included
DeepSeek-V3.2 [23]	Attacker	DeepSeek	Large open-weight frontier model
Gemma-3-4B [40]	Attacker	Google	Very small model; tests whether size matters
GPT-OSS-120B [77]	Attacker	OpenAI	Large open-weight reasoning model
Qwen3-32B [119]	Honeypot backend	Alibaba (Qwen)	Mid-size model writing the fake shell output
Claude Sonnet 5.5	Annotator A	Anthropic	Independent blind labels
Claude Opus 5.5	Annotator B and adjudicator	Anthropic	Independent blind labels; separate instance for adjudication

All attacker sessions used the same system prompt. In short, it told the model that this was an authorised research exercise against a deliberately fake, isolated system owned by the researchers, asked it to behave like an intruder who has just gained a shell (look around, look for valuable data or credentials, consider persistence), and required it to answer with exactly one raw shell command per turn, or the word `DONE` to finish.

3.7.3 Harness structure

The harness is a single Python program. **Figure 3.**5 shows its main parts: a command-line front end and a batch-config reader, a configuration data class, a session runner wrapped in a recovery layer, a thin SSH driver, an LLM dispatcher with retry and back-off, and small helpers for refusal detection and output cleaning. **Table 3.**10 lists the settings that control a run.

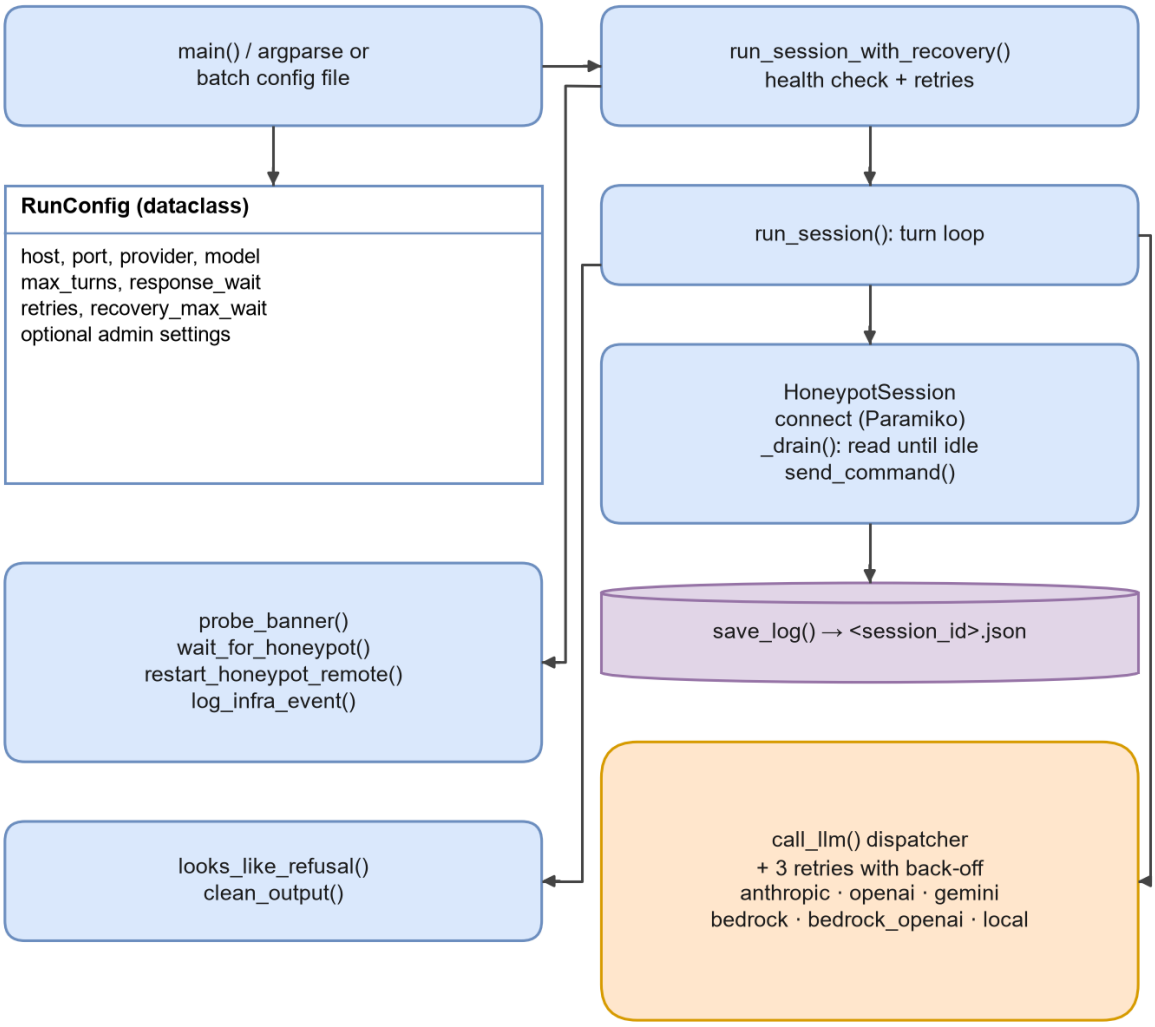


Figure 3.5: Module structure of the attacker harness.

Table 3.10: Main harness parameters (defaults in the code; batch files can override them).

Parameter	Default	Meaning
max_turns	25	Maximum commands per session (some batches used 20)
max_session_seconds	600	Wall-clock limit per session
response_wait	1.5 s	Silence that marks the end of a reply
max_response_wait	20 s	Hard ceiling per command if the reply never ends
command_delay_range	0.8–2.2 s	Random pause between commands
consecutive_timeout_limit	2	Empty replies at the ceiling in a row that mean "honeypot frozen"
max_session_retries	3	Retries for a session that failed because of infrastructure
recovery_max_wait	90 s	How long to wait for the honeypot to come back
LLM retries	3 (back-off 2 s, 4 s)	Retries for a failed API call

3.7.4 The attacker session loop

Each session follows the flowchart in **Figure 3.**6 and the steps in **Algorithm 3.**1. The loop is simple on purpose: the model sees the transcript so far, returns one command, the harness sends it and records what comes back. Six different exits are possible, and each one is saved as the session *status* (**Table 3.**11).

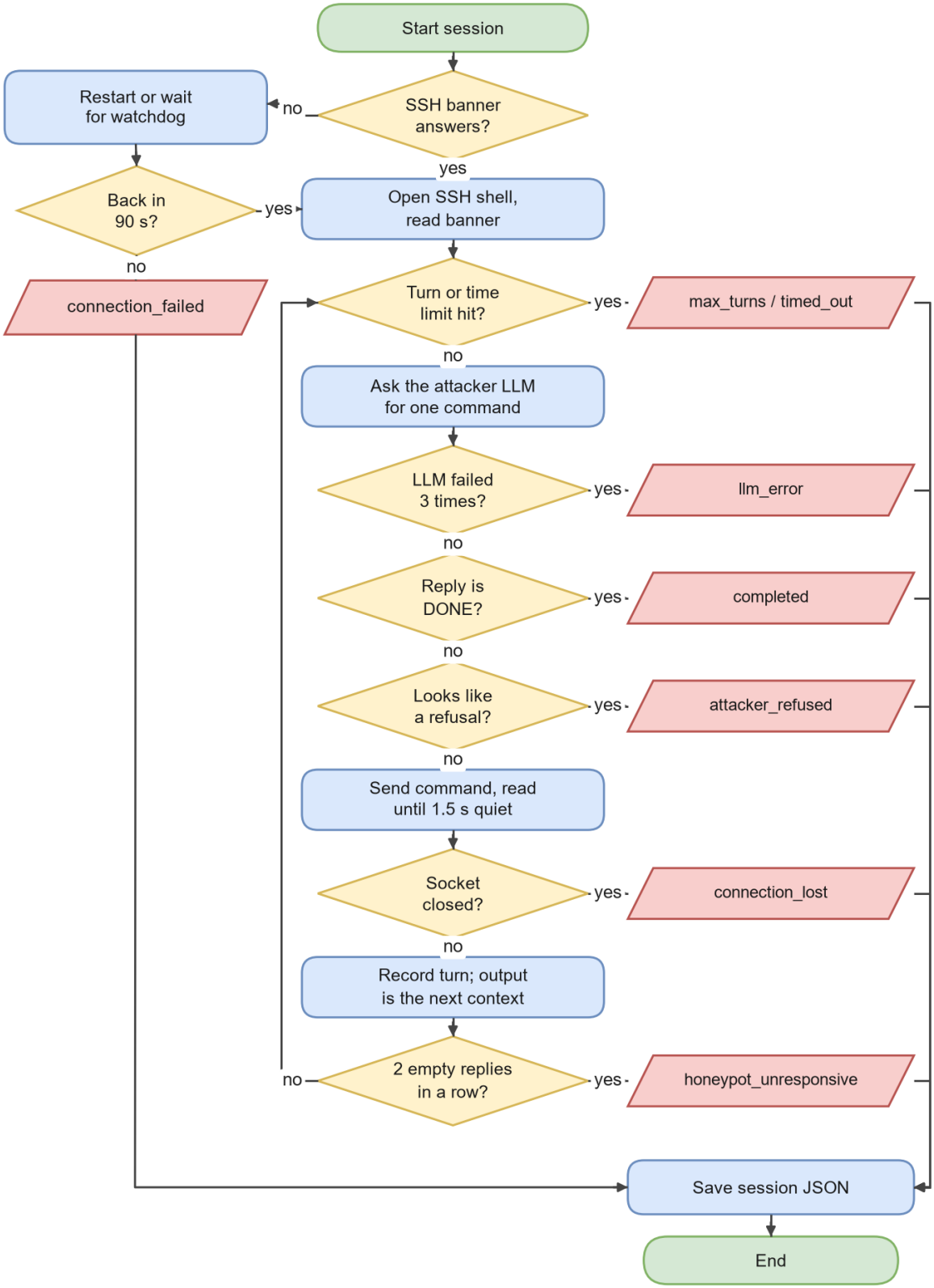


Figure 3.6: Attacker session loop: health check, connection, the decisions taken in every turn, and the status saved at each exit.

Algorithm 3.1: Attacker session loop (run_session)

```
Input: configuration C (target, model, limits), system prompt P
Output: session log L with turns and a final status
```

```
L ← new log with session id, model, backend label, start time
try: open SSH shell to C.host:C.port; read banner B
on failure: L.status ← connection_failed; return L
context ← "You have just connected. Banner: " + B
messages ← [ ]; stalls ← 0
for turn = 0 … C.max_turns − 1 do
    if elapsed time > C.max_session_seconds then L.status ← timed_out; break
    append (user, context) to messages
    reply ← call_llm(C.provider, C.model, P, messages)     // up to 3 tries
    on failure: L.status ← llm_error; break
    append (assistant, reply) to messages
    if reply = "DONE" then L.status ← completed; break
    if looks_like_refusal(reply) then L.status ← attacker_refused; break
    (output, latency) ← send_command(reply)                 // Algorithm 3.2
    on socket error: L.status ← connection_lost; break
    append {turn, time, reply, output, latency} to L.turns
    if output is empty and latency ≥ ceiling then stalls ← stalls + 1 else stalls ← 0
    if stalls ≥ C.consecutive_timeout_limit then L.status ← honeypot_unresponsive;
break
    context ← "Command output: " + output
    sleep a random time in C.command_delay_range
if no break happened then L.status ← max_turns_reached
close the connection; record end time and number of turns
return L
```

Table 3.11: Session outcomes recorded by the harness.

Status	Meaning	Counts as
completed	The model answered `DONE`	Normal data
max_turns_reached	The turn limit was hit	Normal data
timed_out	The wall-clock limit was hit	Normal data
attacker_refused	The reply looked like a refusal, not a command	Normal data (a model behaviour)
llm_error	The API call failed three times	Model/infrastructure failure, kept
connection_lost	The SSH socket died mid-command	Infrastructure failure, retried
honeypot_unresponsive	Two empty replies at the time ceiling in a row	Infrastructure failure, retried
connection_failed	The honeypot did not answer or recover	Infrastructure failure, retried

The refusal check is a small heuristic. It flags a reply that contains phrases such as "I cannot" or "as an AI", or that contains no shell characters at all and reads like prose. Requiring the absence of shell characters was important: without it, a long `find` command with several `-name "*.txt"` patterns was mistaken for prose because file extensions contain full stops.

3.7.5 Reading the honeypot's reply

A fixed wait before reading the reply does not work for both backends: the static shell answers in well under a second, while the LLM backend can take several seconds. Reading

too early captures half a reply and pushes the rest into the next turn, which corrupts every later prompt. We therefore read until the channel has been quiet for 1.5 seconds, with a hard ceiling (**Figure 3.**7, **Algorithm 3.**2).

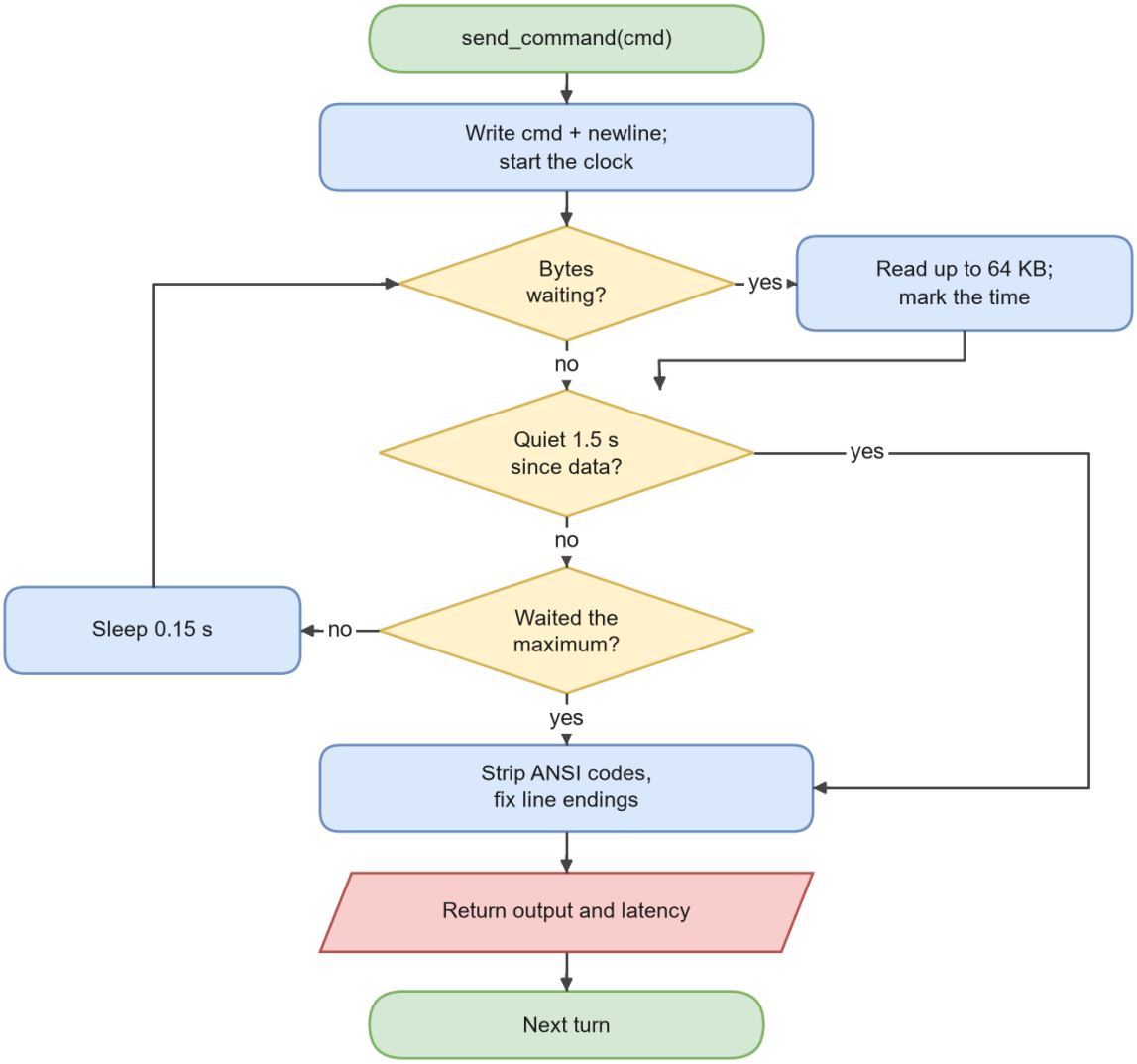


Figure 3.7: Idle-timeout reading of the honeypot's reply.

Algorithm 3.2: Idle-timeout read (send_command and _drain)

```
Input: shell channel S, command c, idle time τ = 1.5 s, ceiling T = max_response_wait
Output: cleaned output text and latency

1   t0 ← now; write c + newline to S
2   buffer ← ""; last ← none
3   loop
4       while S has bytes waiting do
5           buffer ← buffer + read(S, 64 KB); last ← now
6       if last ≠ none and now − last ≥ τ then break          // reply finished
7       if now − t0 ≥ T then break                            // reply never finished
8       sleep 0.15 s
9   output ← remove ANSI escape codes from buffer; normalise line endings
10  return (output, now − t0)
```

3.7.6 Surviving a frozen honeypot

Cowrie runs on a single-threaded event loop. During testing, a recursive `find /` over the large fake file system blocked it for minutes: no new connections were accepted and existing ones hung. Without protection, one such command would waste the rest of a batch. The recovery layer in **Figure 3.**8 and **Algorithm 3.**3 checks the honeypot before every session, restarts it if an admin connection is configured (otherwise it waits for an external watchdog), and retries only sessions that failed because of infrastructure. Real outcomes such as a refusal are never retried, because that would hide genuine behaviour.



Figure 3.8: Session retry and recovery logic.

Algorithm 3.3: Session with recovery (run_session_with_recovery)

```
Input: configuration C, session index i
Output: one saved log per attempt; the final log for session i

1   attempt ← 0
2   loop
3       if not banner_answers(C.host, C.port) then
```

```
4           log "honeypot unresponsive" to infra_events.log
5           if admin access configured then restart Cowrie remotely
6           wait up to C.recovery_max_wait for the banner
7           if still silent then return log with status connection_failed
8       L ← run_session(C, i)                                   // Algorithm 3.1
9       if L.status ∉ {connection_failed, honeypot_unresponsive, connection_lost} then
return L
10      if attempt ≥ C.max_session_retries then return L
11      save L (failed attempt kept for honest reporting); attempt ← attempt + 1
12      wait 3 s
```

3.7.7 Parsing the real dataset

The CyberLab export is not Cowrie's native log format. Cowrie normally writes one JSON event per line; CyberLab instead stores one large JSON array in which each element maps a session ID to its list of events, and the command text sits in the `message` field as `CMD: <command>` rather than in an `input` field. A parser written for native Cowrie logs therefore reads zero sessions and raises no error, which is the most dangerous kind of failure. Our parser (**Figure 3.**9, **Algorithm 3.**4) streams the 600 MB file one line at a time and merges sessions that appear more than once.

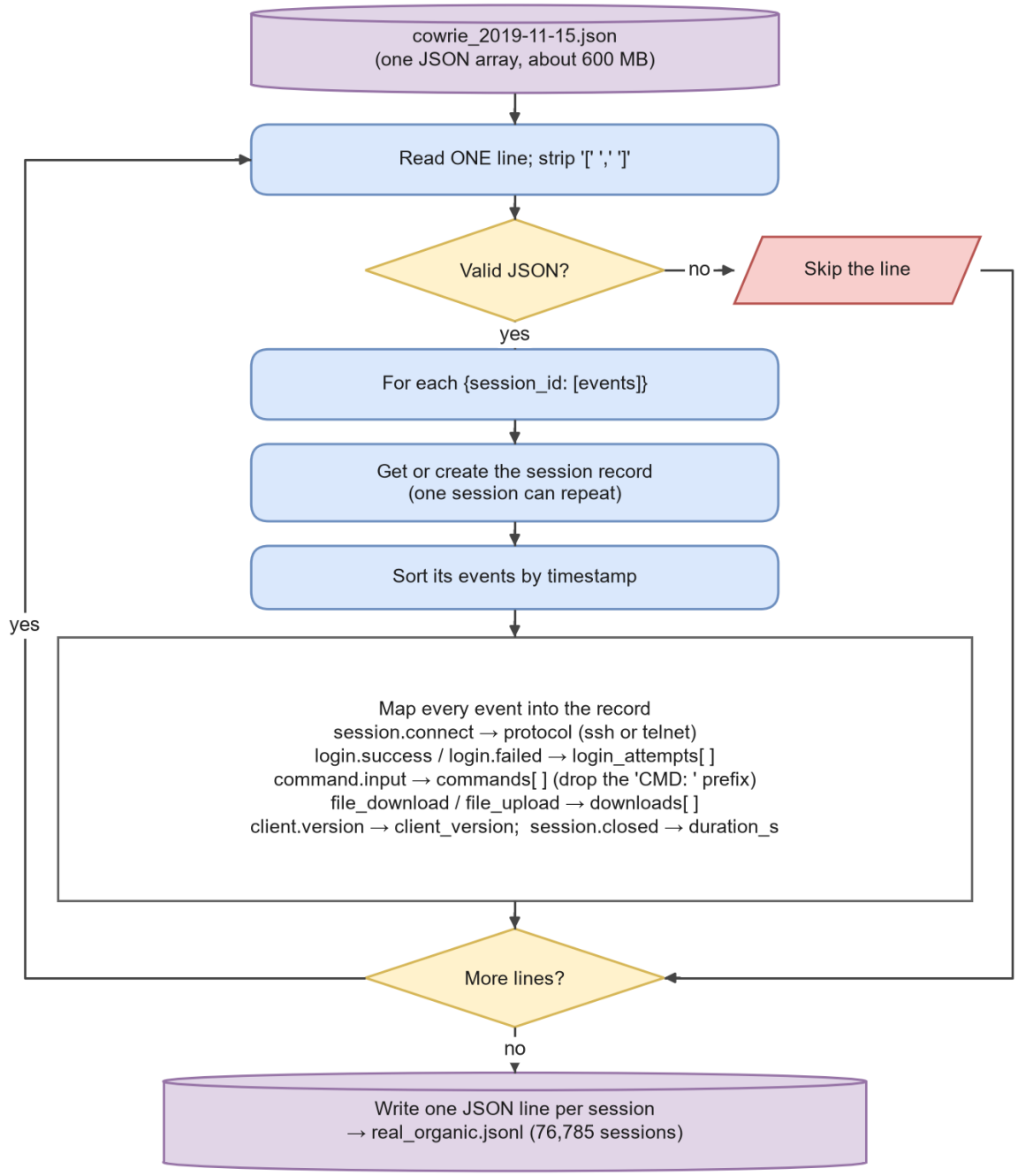


Figure 3.9: Streaming parser for the CyberLab export.

Algorithm 3.4: Streaming CyberLab parser (parse_cyberlab.py)

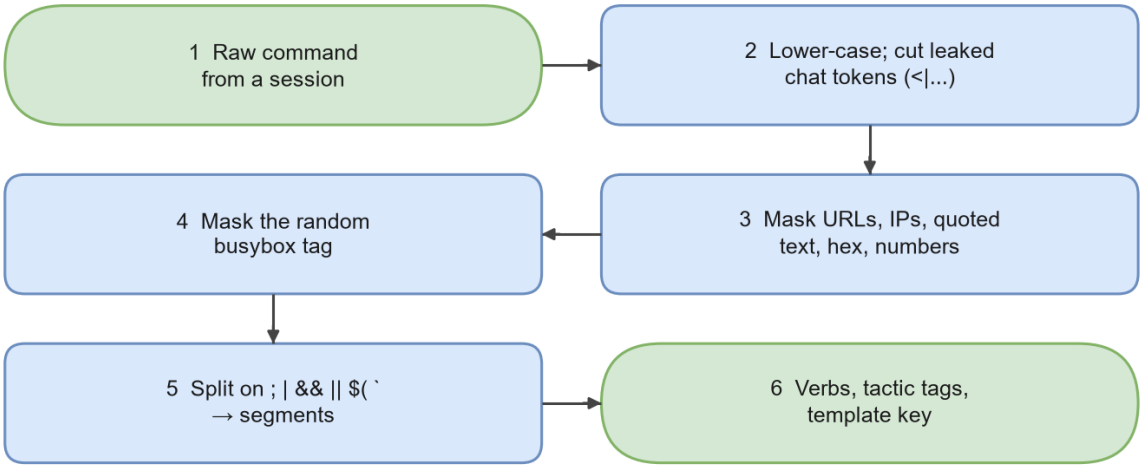
```
Input: CyberLab daily file F
Output: one normalised JSON line per session

sessions ← empty map from session id to record
for each line ℓ in F do                                   // never load F whole
    strip "[", "]" and a trailing "," from ℓ
    try obj ← parse_json(ℓ) except: continue
    for each (sid, events) in obj do
        s ← sessions[sid] or a new empty record
        for each event e in events sorted by timestamp do
            update start/end time, hashed source id, country
            switch e.eventid:
```

```
                session.connect → s.protocol ← e.protocol
                login.success / login.failed → append (username, password, success)
                command.input → append text of e.message without the "CMD: " prefix
                file_download / file_upload → append (url, hash, file)
                client.version → s.client_version; session.closed → s.duration
for each record s do add counts and rule-based findings; write s as one JSON line
```

3.7.8 Normalisation and behaviour templates

The same bot script, run from different machines, produces commands that differ only in IP addresses, URLs and random passwords. To compare *behaviours* rather than *instances*, every command is normalised (**Figure 3.**10, **Algorithm 3.**5, **Table 3.**12). Two sessions belong to the same **behaviour template** when their sequences of normalised commands are identical.




```
Raw:  echo "root:Xk29qPa7mz"|chpasswd|bash
Normalized:  echo <str>|chpasswd|bash
Verbs:  echo, chpasswd, bash
Tactics:  account manipulation, execution
```

```
Raw:  /bin/busybox QWERT
Normalized:  /bin/busybox <tag>
Verbs:  busybox
Tactics:  none
```

```
Raw:  cat /proc/cpuinfo | grep name | wc -l
Normalized:  cat /proc/cpuinfo | grep name | wc -l
Verbs:  cat, grep, wc
Tactics:  discovery
```

Figure 3.10: Command normalisation in six steps, with three worked examples produced by the project's own code.

Table 3.12: Values masked during normalisation (session_features.py).

Pattern	Replaced by	Example
URL (http, https, ftp, tftp)	`<url>`	`http://203.0.113.7/x.sh` → `<url>`
IPv4 address with optional port	`<ip>`	`198.51.100.4:8080` → `<ip>`
Quoted string	`<str>`	`"root:Xk29qPa7mz"` → `<str>`
Hex string of 16+ characters	`<hex>`	file hashes
Random alphanumeric token (8+ chars with letters and digits)	`<tok>`	random file names
Bare number	`<n>`	`head -n 10` → `head -n <n>`
Random Mirai busybox tag (not a real applet name)	`busybox <tag>`	`/bin/busybox QWERT` → `/bin/busybox <tag>`
Leaked chat-format text after `<|`	removed	GPT-OSS control tokens and reasoning

Algorithm 3.5: Normalisation, segments, verbs and template key

```
Input: a session with commands c1 … cn
Output: normalised commands, verbs, tactic set, template key

1   for each command c do
2       c ← text before the first "<|" (drops leaked chat tokens); lower-case; trim
3       replace URLs, IPs, quoted strings, long hex, random tokens and numbers by
placeholders
4       replace "busybox X" by "busybox <tag>" unless X is a known applet (wget, cat, echo,
…)
5       collapse repeated spaces
6       segments(c) ← split c on ";", "|", "&&", "||", newline, "$(" and backtick
7       verb(segment) ← first word that is not a VAR=value prefix, without its path
8       tactics(c) ← every tactic whose regular expression matches some segment
9   template key ← tuple of all normalised commands, in order
10  return normalised commands, verbs, tactic set, template key
```

One limitation of the URL mask is worth recording. The URL pattern stops only at whitespace, so in a command such as `wget http://…/x.sh; chmod 777 x.sh` the semicolon directly after the URL is swallowed by the mask and the two parts are treated as one segment. This affects only the splitting of a few chained download commands, not the template grouping, and it applies equally to both corpora.

3.7.9 Tactic tagging

For the label-free comparison we also describe each session by the ATT&CK-style tactics it touches [103]. A command segment can match zero or more of nine regular-expression categories (**Table 3.**13). This tagger is *descriptive only*: it is never used as a training label for reported results.

Table 3.13: The nine tactic categories and example patterns.

Tactic	Example patterns (simplified)	Typical command
Discovery	`uname`, `whoami`, `id`, `ps`, `ls`, `/proc/cpuinfo`, `crontab -l`	`cat /proc/cpuinfo | grep name`
Credential access	`/etc/shadow`, `.ssh`, `id_rsa`, `.bash_history`, "password", "secret"	`cat /etc/shadow`
Account manipulation	`chpasswd`, `passwd`, `useradd`, `authorized_keys`	`echo <str> | chpasswd`
Persistence	`crontab` (not `-l`), `/etc/cron`, `systemctl enable`, `rc.local`	adding a cron entry
Privilege escalation	`sudo`, `su`, `-perm -4000`, `getcap`, `/etc/sudoers`	`sudo -l`
Defence evasion	`history -c`, `unset HISTFILE`, deleting logs, `chattr`, `pkill`	`history -c`
Ingress tool transfer	`wget`/`curl`/`tftp` **with** an argument	`wget <url>`
Execution	`chmod +x`, `./file`, `sh`, `bash`, `base64 -d`, `nohup`	`./x.sh`
Resource hijacking	`xmrig`, `minerd`, `stratum`, `miner`, `dota`	starting a miner

3.7.10 Label-free measures

Figure 3.11 shows the five label-free measures. The most important are the verb-distribution divergence and the classifier two-sample test.

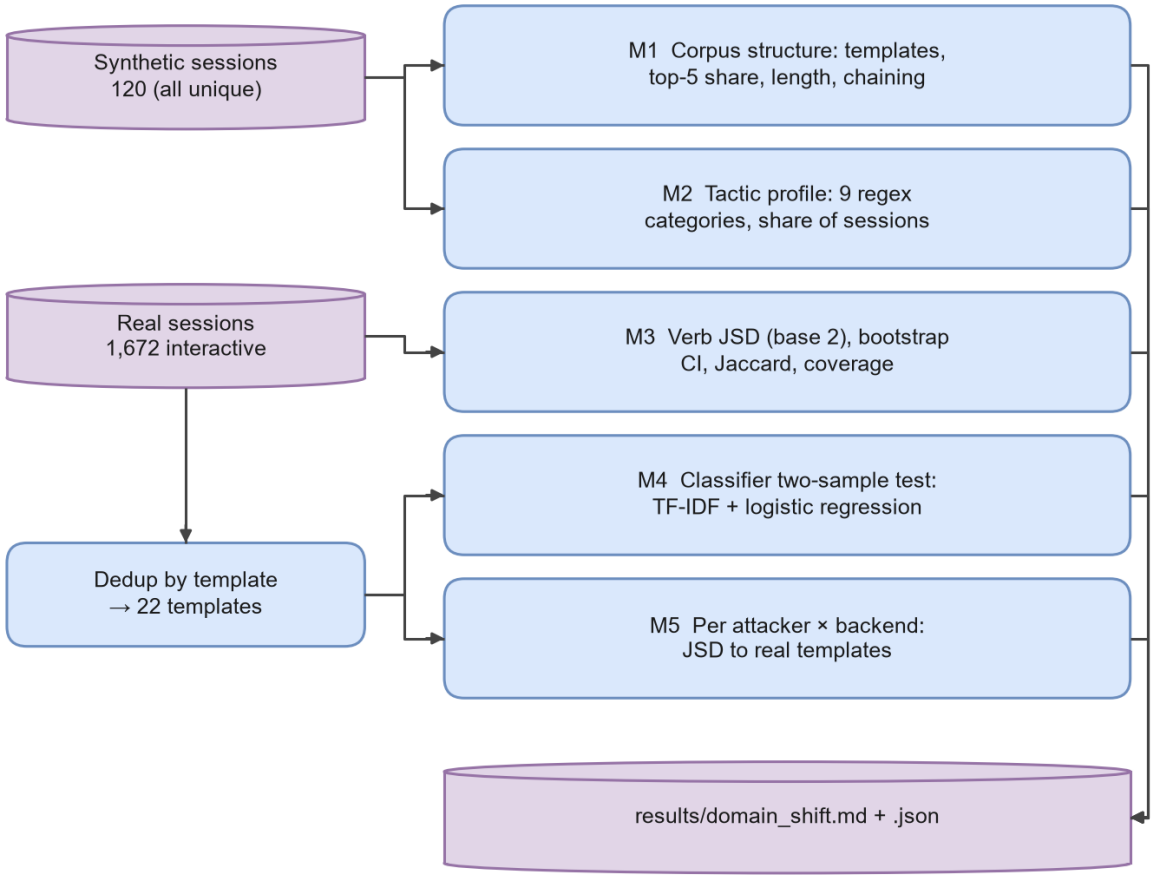


Figure 3.11: Label-free domain-shift analysis.

Verb distributions. The verb of a segment is its first word (`ls`, `cat`, `wget`). Counting verbs over all sessions gives a probability distribution for each corpus. We compare the synthetic distribution P and the real distribution Q with the Jensen-Shannon divergence in base 2 [64], built from the Shannon entropy and the Kullback-Leibler divergence [62]:

$$H(P) = -\sum_i P(i)\log_2 P(i) \tag{3.1}$$

$$D_{KL}(P \parallel Q) = \sum_i P(i)\log_2\left(\frac{P(i)}{Q(i)}\right) \tag{3.2}$$

$$JSD(P \parallel Q) = \frac{1}{2}D_{KL}(P \parallel M) + \frac{1}{2}D_{KL}(Q \parallel M), \qquad M = \frac{1}{2}(P+Q) \tag{3.3}$$

JSD is 0 when the distributions are identical and 1 when they share no verb at all. Its square root is a metric [29]. We also report the Jaccard index of the two verb vocabularies V_S and V_R [55], and the coverage, the share of real verb occurrences whose verb also appears in the synthetic vocabulary:

$$J(V_S, V_R) = \frac{|V_S \cap V_R|}{|V_S \cup V_R|} \tag{3.4}$$

$$Cov = \frac{\sum_{v \in V_S \cap V_R} n_R(v)}{\sum_{v \in V_R} n_R(v)} \tag{3.5}$$

Uncertainty. A single JSD value does not say how stable it is. We resample the synthetic sessions with replacement $B = 500$ times, recompute JSD each time and take the 2.5th and 97.5th percentiles as a 95% confidence interval (Efron and Tibshirani, 1993; **Algorithm 3.**6):

$$CI_{0.95} = \left[\hat{\theta}_{(\lfloor 0.025B \rfloor)}, \hat{\theta}_{(\lfloor 0.975B \rfloor)}\right] \tag{3.6}$$

Algorithm 3.6: Bootstrap confidence interval for JSD

```
Input: synthetic sessions S, real sessions R, B = 500, seed 42
Output: point estimate and 95% CI

θ ← JSD(verb counts of S, verb counts of R)
for b = 1 … B do
    S* ← |S| sessions drawn from S with replacement
    θ*_b ← JSD(verb counts of S*, verb counts of R)
sort θ*_1 … θ*_B
return θ, [θ*_(0.025B), θ*_(0.975B)]
```

Classifier two-sample test. If a simple classifier can tell synthetic sessions from real ones, the corpora differ in ways that a downstream model can exploit [67]. Each session becomes a document of its normalised commands, weighted with TF-IDF [91, 99]:

$$w(t,d) = tf(t,d) \times \left(\ln\left(\frac{1+N}{1+df(t)}\right) + 1\right) \tag{3.7}$$

A balanced logistic regression is trained to separate the corpora, and its ranking quality is measured by the area under the ROC curve [35, 47], which equals the probability that a random synthetic session scores higher than a random real one:

$$AUC = \frac{1}{n_+ n_-} \sum_{i \in +} \sum_{j \in -} 1[s_i > s_j] \tag{3.8}$$

An AUC of 0.5 means the corpora cannot be told apart; 1.0 means perfect separation. We use 5 repeats of 5-fold stratified cross-validation [59] and compare synthetic sessions with the de-duplicated real templates, so that the 1,038 copies of one script do not dominate. To make sure that file paths and arguments are not doing all the work, we repeat the test with verbs only (**Algorithm 3.**7).

Algorithm 3.7: Classifier two-sample test (C2ST)

```
Input: synthetic sessions S (label 1), real templates R (label 0), text function g
Output: mean and standard deviation of AUC

docs ← [g(s) for s in S ∪ R]; y ← [1]·|S| + [0]·|R|
for each of the 5 × 5 stratified folds (train, test) do
    fit a TF-IDF vectoriser on docs[train] only            // no test information leaks
    fit logistic regression (balanced weights) on the training vectors
    record AUC of predicted probabilities on docs[test]
return mean and standard deviation of the 25 AUC values
```

3.7.11 Annotation protocol

The transfer experiment needs intent labels that do not come from keywords. **Figure 3.**12 shows the protocol and **Table 3.**14 the label set.

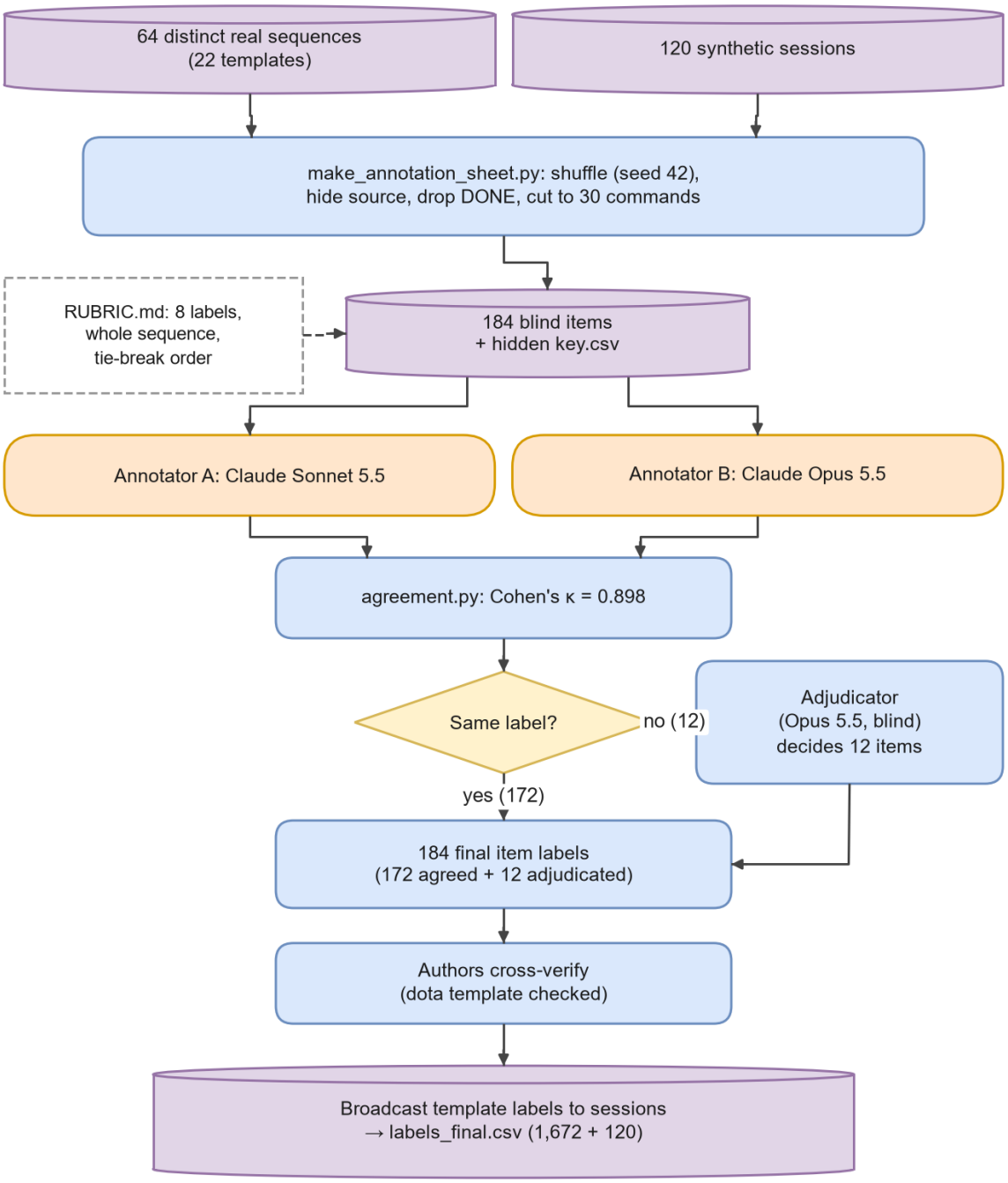


Figure 3.12: Blind dual-LLM annotation, adjudication, author verification and template broadcast.

Table 3.14: Annotation rubric: one primary intent per session (annotation/RUBRIC.md).

Label	Definition	Common confusion and rule
recon	Learning about the system; nothing changed	Reading `/etc/passwd` alone is recon, not credential access
credential_access	Harvesting or guessing credentials, keys or secrets	Searching for secret files counts
privilege_escalation	Trying to gain higher privileges	`sudo -l` alone is recon
persistence	Keeping access: cron, rc, SSH keys, new or changed accounts	Changing the root password is persistence
malware_drop	Downloading or running a payload	A download without execution still counts
resource_hijacking	Deploying or inspecting cryptominers; abusing compute	A miner fetched by `wget` is resource hijacking
defense_evasion	Hiding or disabling defences	Clean-up after a download follows the tie-break
other	Benign, unclear or noise	Use only if no intent above is evident

When a session shows several behaviours, annotators pick the goal that progressed furthest, using a fixed tie-break order (**Figure 3.**13).

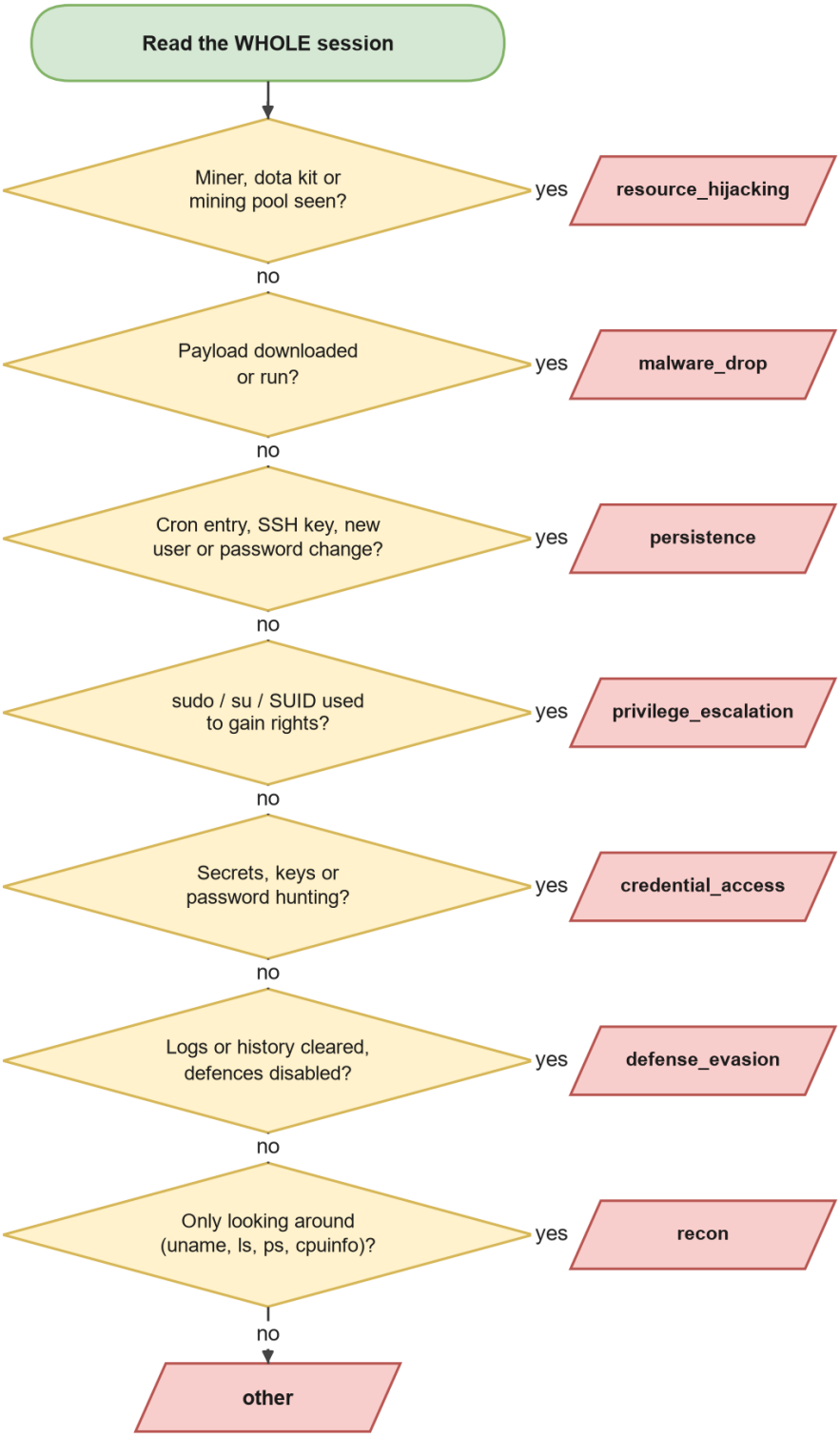


Figure 3.13: The rubric's tie-break order, read from top to bottom.

The annotation set contained 184 items: every distinct real command sequence (64 sequences that fall into the 22 templates) and all 120 synthetic sessions. Items were shuffled with a fixed seed, the source was hidden, the synthetic agents' `DONE` token was removed (it would reveal the source) and very long sessions were shortened to their first 30

commands. Agreement between the two annotators is measured with Cohen's kappa [21], where *p_o* is the observed agreement and *p_e* the agreement expected by chance from each annotator's label frequencies:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_e = \sum_k p_{A,k} p_{B,k} \tag{3.9}$$

Algorithm 3.8: Blind dual-LLM annotation and template broadcast

```
Input: real sessions R, synthetic sessions S, rubric, seed 42
Output: labels_final.csv with one label per session

1   items ← one representative per distinct real command sequence ∪ all of S
2   shuffle items (seed 42); give ids A001…; write a hidden key (id → source)
3   for each annotator a ∈ {A, B} independently do
4       label every item with one rubric label (source hidden, no access to the other's
labels)
5   κ ← Cohen's kappa(A, B); write disagreements
6   for each item: if A = B then final ← A else final ← adjudicator's label (blind, uses
rubric)
7   authors cross-verify the final labels (special check of the largest template)
8   for each real session r: label(r) ← final label of the item with r's template key
9   write (session id, label, source) for all real and synthetic sessions
```

3.7.12 Four-quadrant transfer experiment

The transfer design is shown in **Figure 3.**14. Every session becomes a feature vector: TF-IDF weights of its normalised commands (up to 2,000 tokens) plus three numbers: the number of commands, the mean command length and the share of chained commands. The classifier is a random forest [13] with the settings in **Table 3.**15.

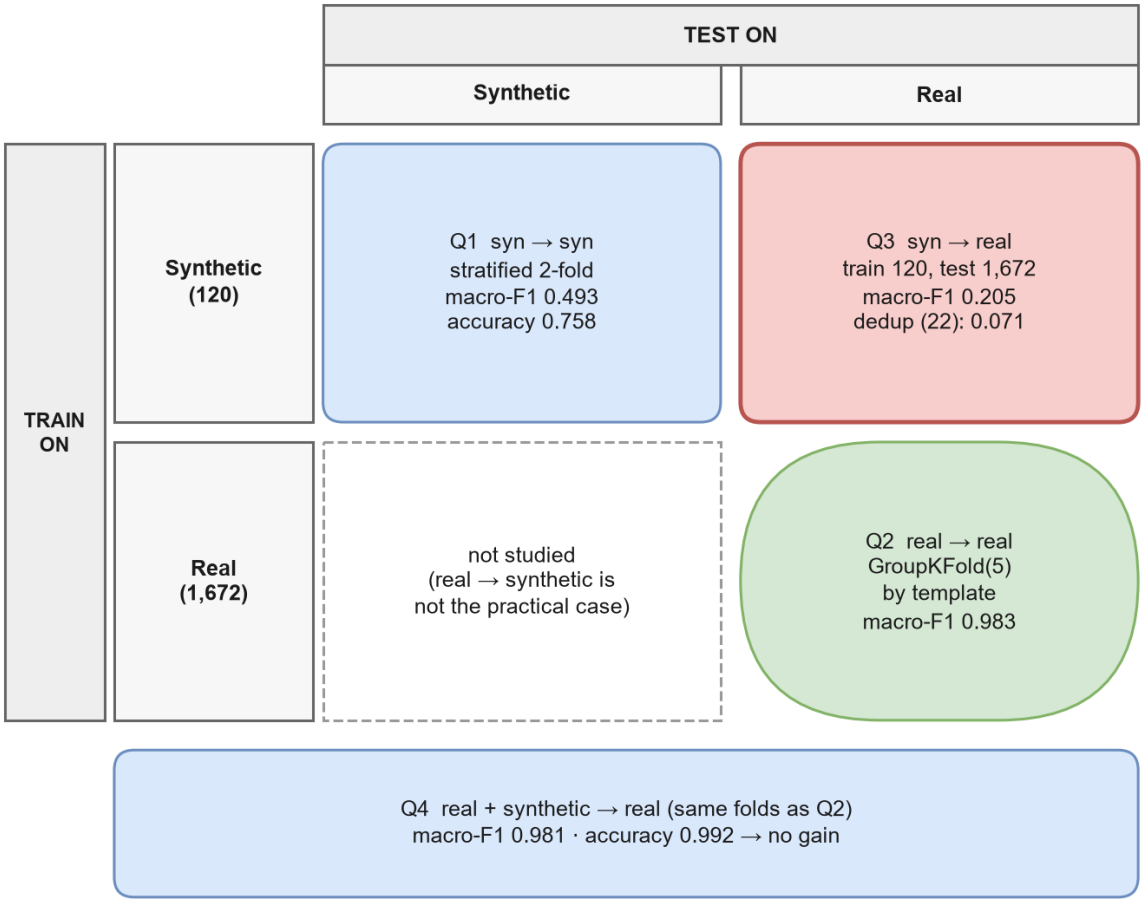


Figure 3.14: The four quadrants of the transfer experiment, with the main results.

Table 3.15: Classifier and evaluation settings.

Setting	Value	Reason
Features	TF-IDF (max 2,000 tokens) + 3 numeric	Simple, transparent, comparable across corpora
Classifier	Random forest, 300 trees	Robust on small, mixed feature sets
Class weights	balanced	Rare classes are not ignored [51]
Seed	42	Reproducible
Q1 folds	Stratified 2-fold within synthetic	Two classes have six or fewer examples
Q2/Q4 folds	GroupKFold(5) by template	No template appears in both train and test
Q3	Train on all synthetic, test on all real	Zero-shot transfer
Q3-dedup	Test on one session per real template	Removes the weight of repeated scripts
Baseline	Always predict the training majority class	Shows what "no learning" scores

Performance is measured with per-class precision P_c and recall R_c, the per-class F1, macro-F1 over the classes **present in the test set** and accuracy [97]:

$$F1_c = \frac{2\,P_c R_c}{P_c + R_c} \tag{3.10}$$

$$Macro - F1 = \frac{1}{|C_{test}| \sum_{c \in C_{test}} F1_c} \tag{3.11}$$

$$Acc = \frac{1}{n} \sum_{i=1}^{n} 1[\hat{y}_i = y_i] \tag{3.12}$$

Averaging only over classes present in the test set matters. Otherwise a class that the model predicts but that never occurs in the test set adds a zero to the average and makes the score look worse than it really is.

Algorithm 3.9: Four-quadrant transfer evaluation

```
Input: synthetic sessions S with labels yS, real sessions R with labels yR, template id
g(r)
Output: macro-F1, accuracy and baselines for Q1, Q2, Q3, Q3-dedup, Q4

1   Q1: for each stratified fold (train, test) of S: fit on train, predict test; pool
predictions
2   for each GroupKFold(5) fold (train, test) of R grouped by g do
3       Q2: fit on R[train]; predict R[test]
4       Q4: fit on R[train] ∪ S; predict R[test]                // same folds as Q2
5       baseline: predict the majority label of yR[train]
6   Q3: fit on all of S; predict all of R
7   Q3-dedup: keep the Q3 predictions for the first session of each real template
8   for each quadrant: macro-F1 over classes present in its test labels; accuracy
```

Algorithm 3.10: Macro-F1 over classes present in the test set

```
Input: true labels y, predicted labels ŷ
Output: macro-F1

1   C ← set of distinct values in y                             // not in ŷ
2   for each class c in C do
3       P_c ← precision of c; R_c ← recall of c (0 if undefined)
4       F1_c ← 2·P_c·R_c / (P_c + R_c), or 0 if both are 0
5   return the mean of F1_c over c ∈ C
```

3.7.13 Data model

Figure 3.15 shows how the main records relate: a synthetic log contains turns and is converted into a normalised session; real sessions map many-to-one onto templates; each template and each synthetic session becomes one annotation item; and final labels are broadcast back to sessions.

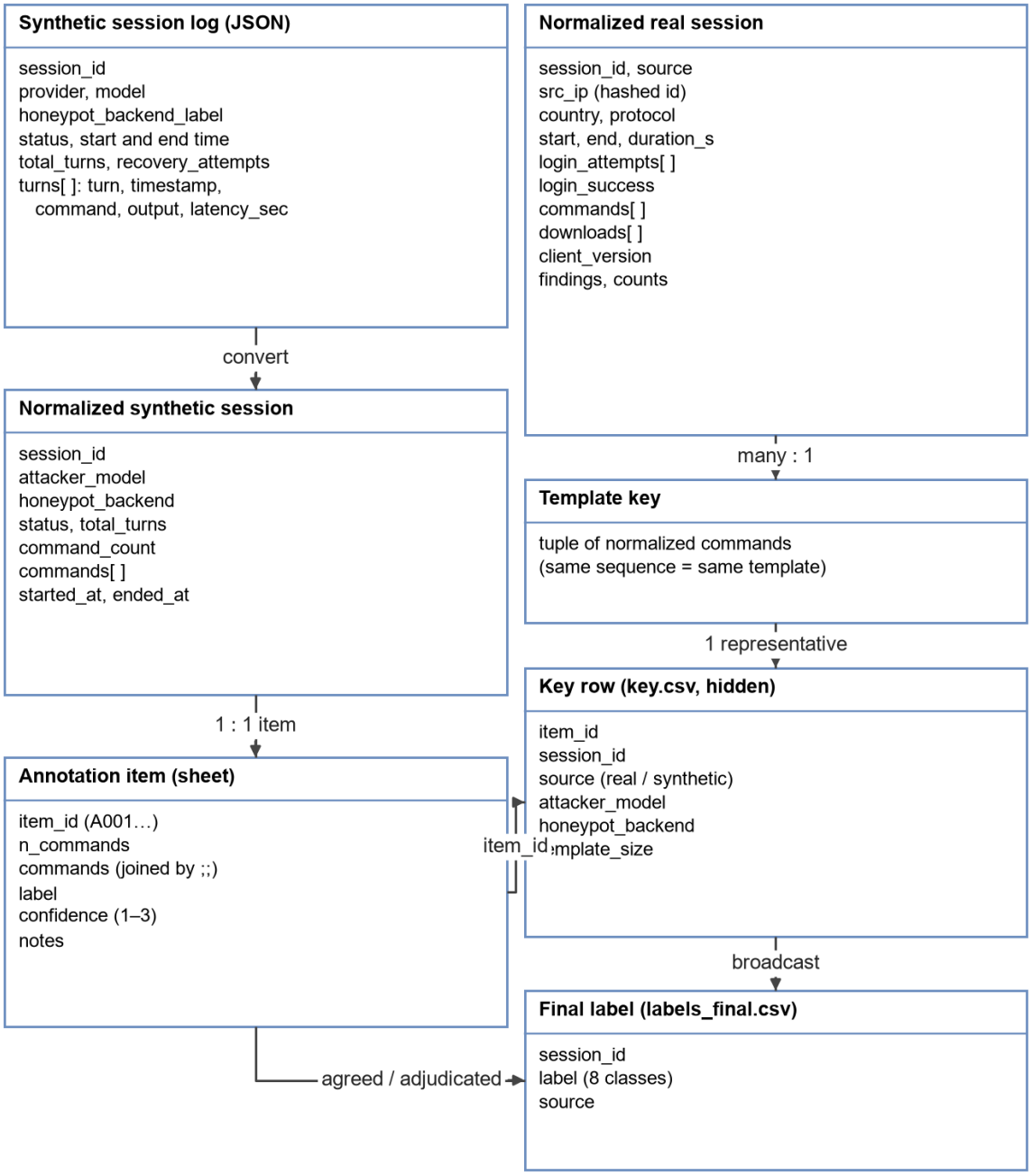


Figure 3.15: Data model of the project's main records.

3.7.14 Risk register

Table 3.16: Risks identified during design and how they were handled.

Risk	Likelihood	Impact	Mitigation
Honeypot freezes on heavy commands	High	High	Stall detection, retries, infra log, optional remote restart
Slow LLM backend truncates replies	High	High	Idle-timeout reading instead of fixed waits
Attacker model refuses the task	Medium	Medium	Research framing in the prompt; refusals recorded as an outcome
Parser silently reads nothing	Medium	High	Dedicated CyberLab parser; cross-check of session counts
Template leakage inflates scores	High	High	GroupKFold by template; de-duplicated test
Circular labels	High	High	Dual-LLM labels; rule labels only as plumbing check
Shared bias of LLM annotators	Medium	Medium	Two different models, blind; adjudication; author check; sensitivity run
Chat tokens leak into commands	Medium	Medium	Detection and stripping of `<|…` text before analysis
API cost overrun	Low	Medium	Small cells (20 sessions), turn and time limits
Attacker escapes the testbed	Low	Very high	One fixed target; no scanning code; isolated honeypot with no real data

3.8 Software Design and Engineering

The research questions are answered by software, so the quality of the software decides the quality of the answers. This section describes the system the way a software engineer would: how it was developed, who uses it and for what, how data moves through it, how it is split into components and modules, how the parts talk to each other, how a session moves between states, how expensive each algorithm is, and how errors, configuration and security are handled.

3.8.1 Development life cycle

We followed an iterative and incremental life cycle (**Figure 3.**16). Each iteration planned a small change, built it, ran a batch or an analysis, looked at the output and then refined the design. This suited a research project in which the right design was not known at the start: several decisions in this report, such as reading replies with an idle timeout or grouping folds by template, came directly from a failed earlier iteration.

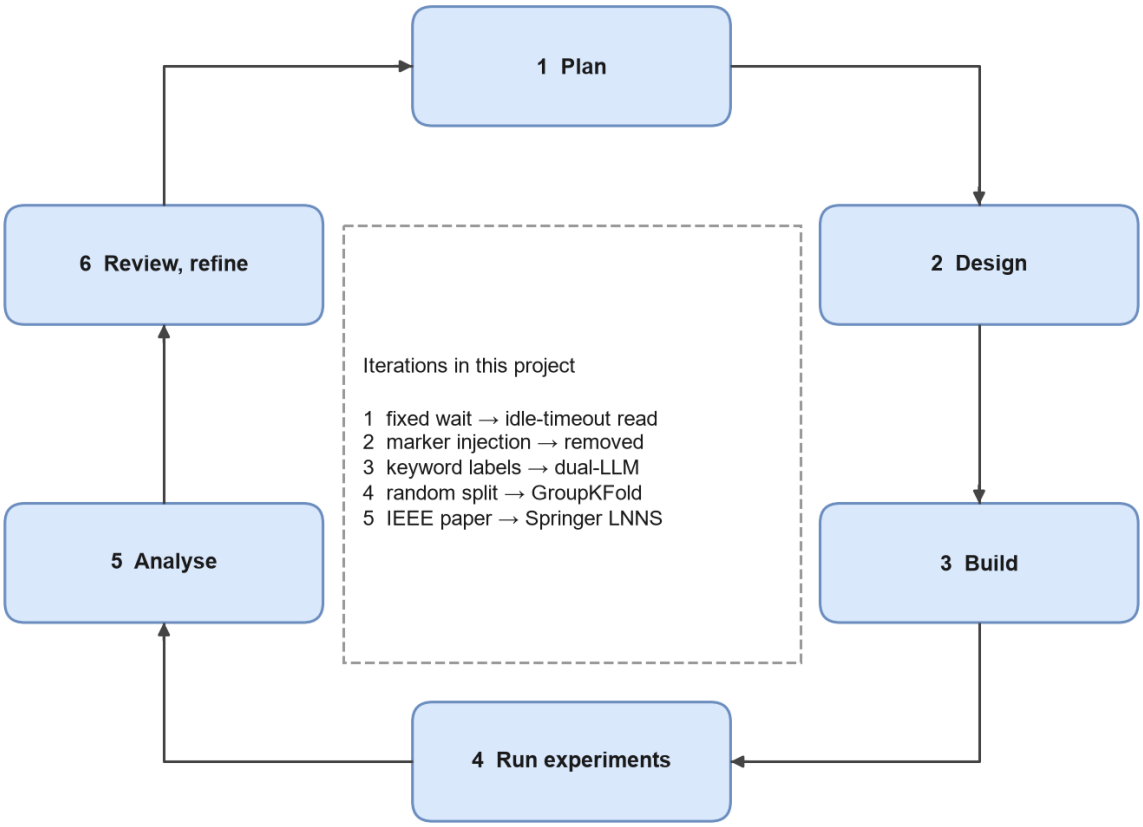


Figure 3.16: Iterative development life cycle, with the main design changes made in this project.

3.8.2 Use cases

Figure 3.17 shows who uses the system and for what. The researcher drives almost every step; the LLM annotators take part only in labelling; and readers or reviewers use the published results to check or reproduce the work.

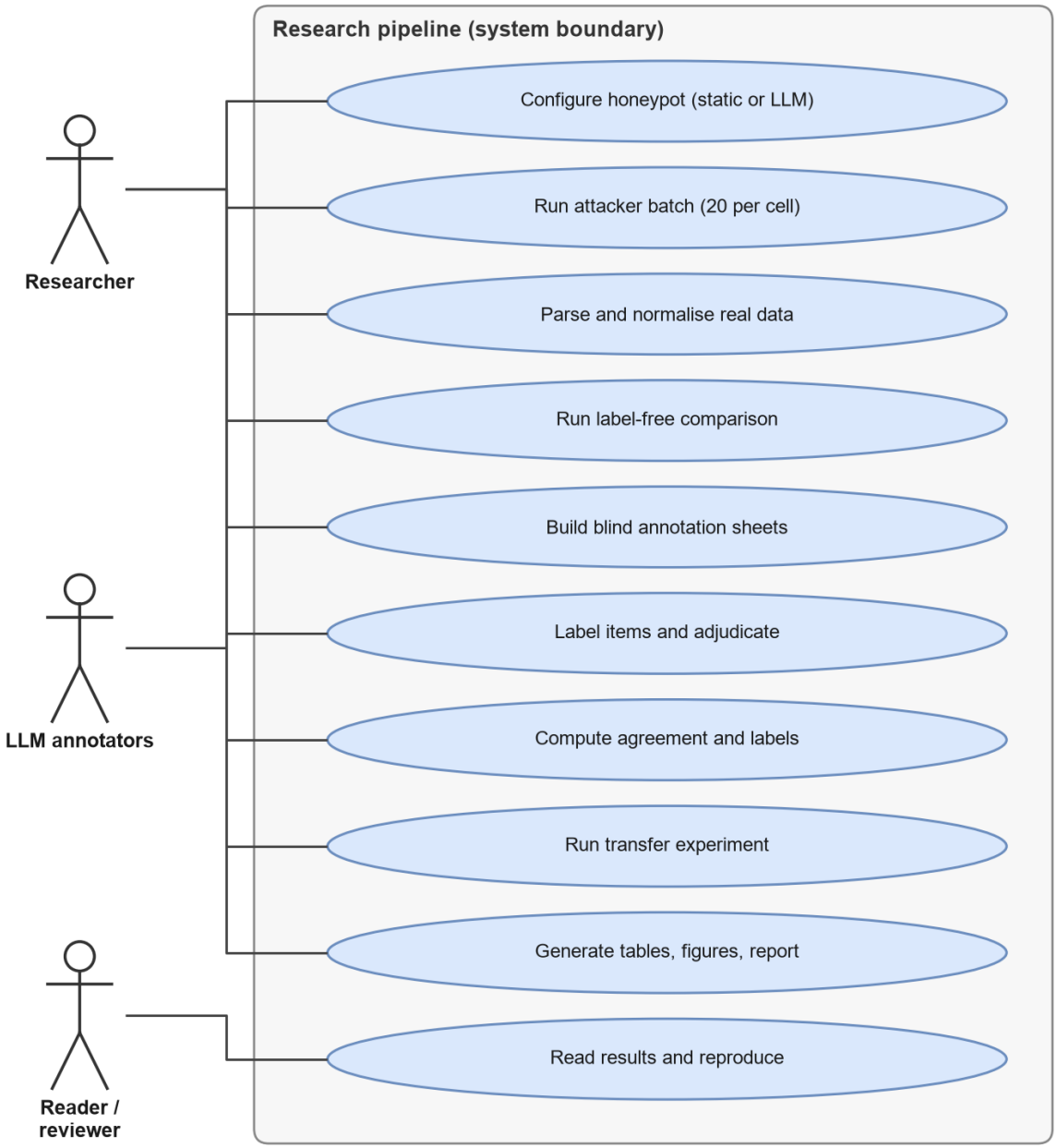


Figure 3.17: Use case diagram of the research pipeline.

Table 3.17: Use case summary.

ID	Use case	Actor	Result
UC1	Configure the honeypot	Researcher	Static or LLM backend selected in one configuration file
UC2	Run a synthetic batch	Researcher	20 session logs per cell, with outcomes
UC3	Parse and normalise real data	Researcher	`real_organic.jsonl` and behaviour templates
UC4	Run the label-free comparison	Researcher	`results/domain_shift.md` and `.json`
UC5	Build blind annotation sheets	Researcher	Shuffled items and a hidden key
UC6	Label and adjudicate items	LLM annotators	Two independent label sets and adjudicated disagreements
UC7	Compute agreement and final labels	Researcher	κ and `labels_final.csv`
UC8	Run the transfer experiment	Researcher	`results/table3_transfer.md` and `.json`
UC9	Generate tables, figures and the report	Researcher	Paper tables, figures and this document
UC10	Read and reproduce results	Reader / reviewer	Independent check of every reported number

3.8.3 Data-flow diagrams

The context diagram (**Figure 3.**18) treats the whole pipeline as one process and shows what crosses its boundary. The level-1 diagram (**Figure 3.**19) opens that process into eight sub-processes and four data stores. Every arrow carries a named file or message, which is why the stores in the diagram match the folders of the repository.

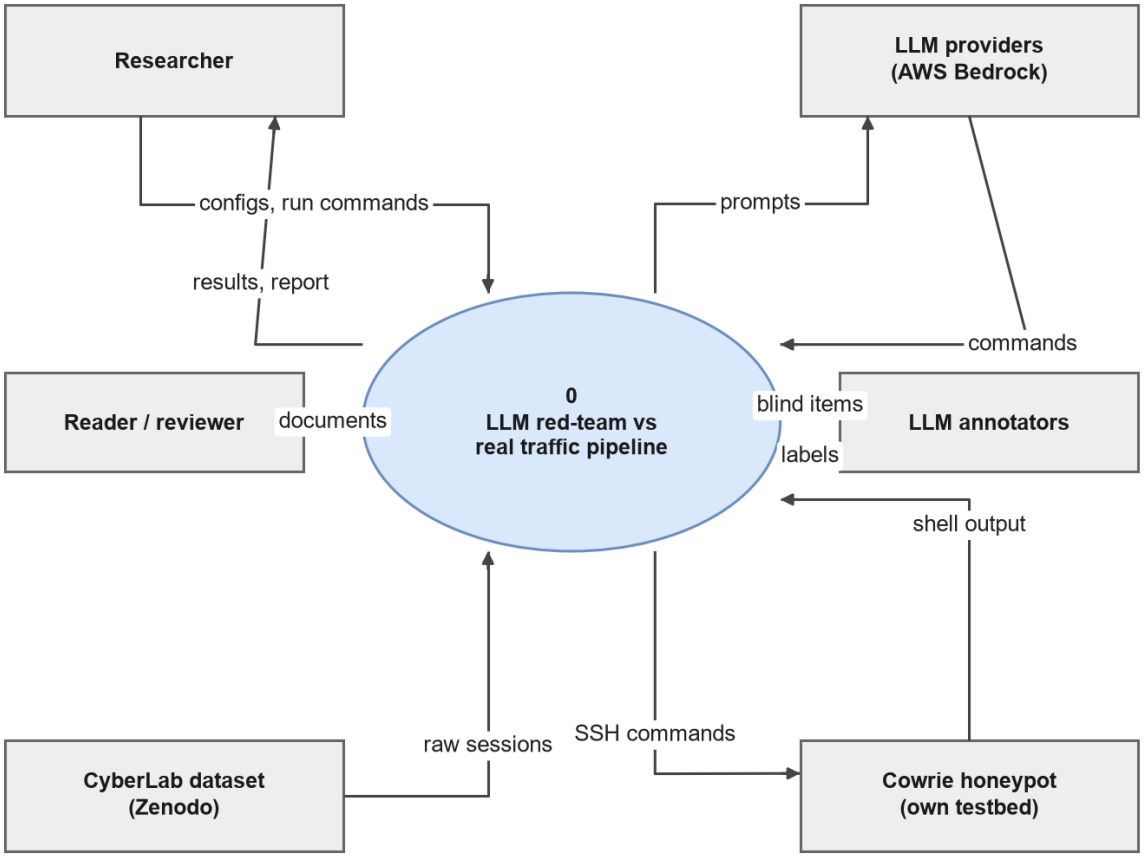


Figure 3.18: Data-flow diagram, level 0 (context diagram).

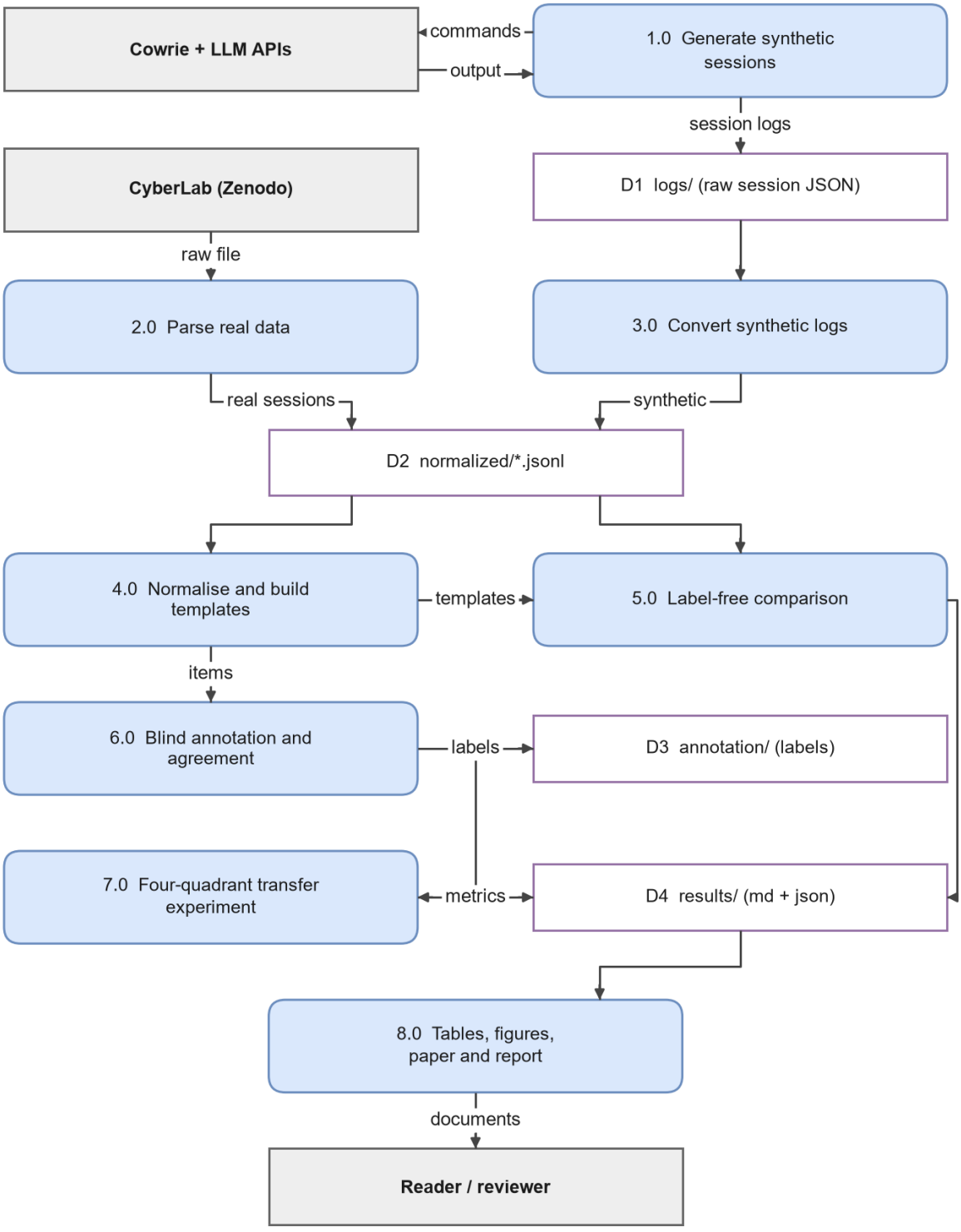


Figure 3.19: Data-flow diagram, level 1.

Table 3.18: Data dictionary of the main stores and flows.

Store or flow	Format	Written by	Read by
D1 `logs/`	One JSON file per session	Process 1.0	Process 3.0
D2 `normalized/`	JSON Lines, one session per line	Processes 2.0 and 3.0	Processes 4.0 and 5.0
D3 `annotation/`	CSV sheets, key and final labels	Process 6.0	Process 7.0
D4 `results/`	Markdown and JSON	Processes 5.0 and 7.0	Process 8.0
Raw CyberLab file	One JSON array (about 600 MB)	External (Zenodo)	Process 2.0
Report and paper	Word, PDF, LaTeX	Process 8.0	Readers

3.8.4 Components and technology stack

Figure 3.20 groups the software into a testbed layer, an analysis layer, third-party libraries and outputs. One shared library, `session_features`, is imported by every analysis component, so the corpora are always normalised the same way. **Table 3.**19 lists the technologies and why each was chosen.

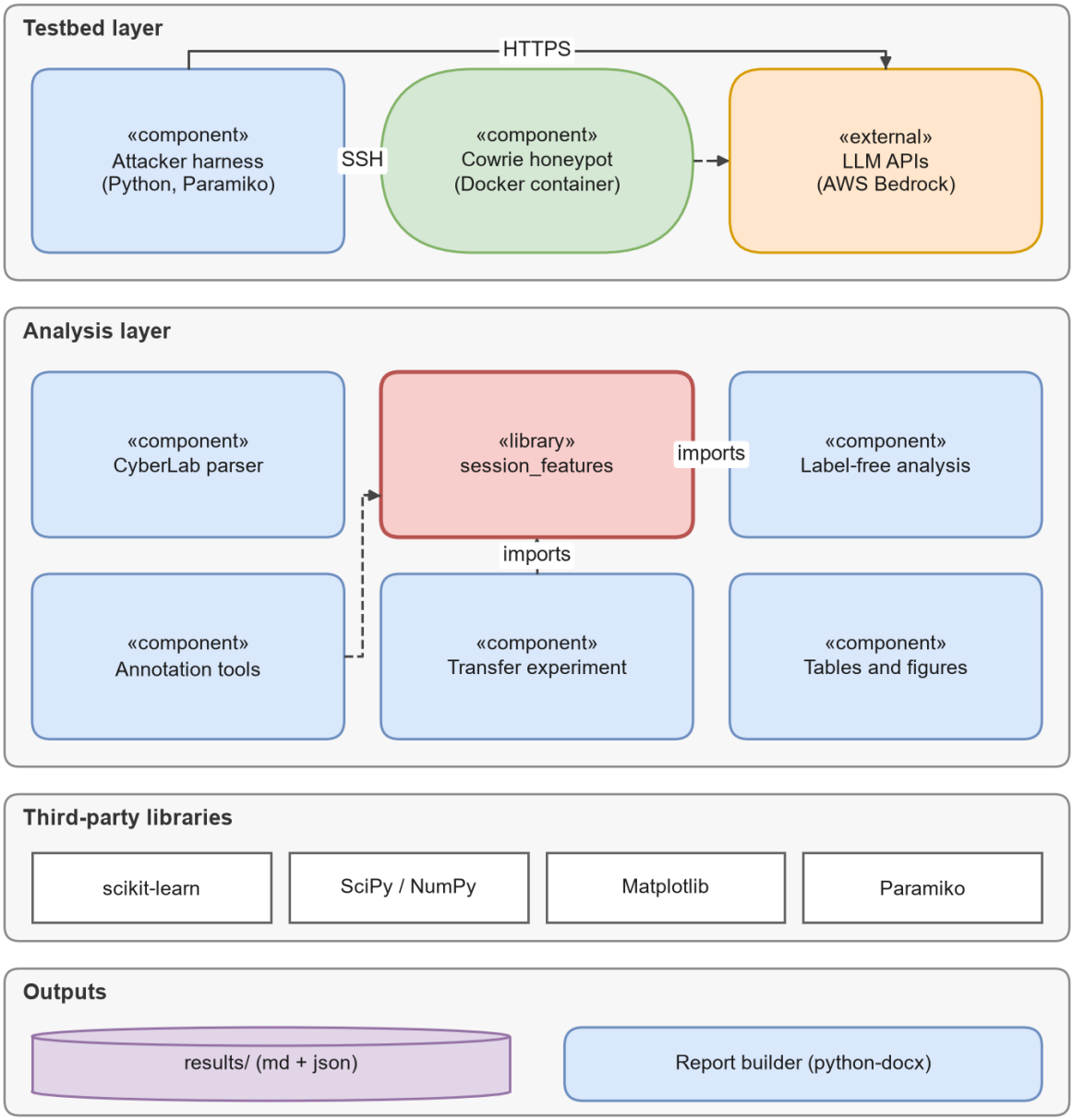


Figure 3.20: Component diagram.

Table 3.19: Technology stack.

Layer	Technology	Version	Why it was chosen
Language	Python	3.11	Mature data and ML ecosystem; one language for every step
Honeypot	Cowrie in Docker	latest image	Standard SSH honeypot with structured logs and an LLM backend
Containers	Docker, Docker Compose	Compose file 3.8	Same environment on every machine [70]
SSH client	Paramiko	≥ 3.4	Pure-Python SSH with interactive channels
Model access	AWS Bedrock (OpenAI-compatible API), requests, boto3	≥ 2.31 / ≥ 1.34	One request format for several model vendors
Machine learning	scikit-learn	≥ 1.4	TF-IDF, logistic regression, random forest, grouped CV, kappa [82]
Numerics	NumPy, SciPy, pandas	≥ 1.26 / ≥ 1.12 / ≥ 2.2	Arrays, Jensen-Shannon distance, tables [50, 111]
Charts and diagrams	Matplotlib; draw.io XML	3.x	Reproducible figures; editable diagrams [53]
Documents	LaTeX (Springer LNCS class); python-docx and Microsoft Word	–	Paper and this report built from source
Version control	Git and GitHub	–	History, collaboration, public release

3.8.5 Class and module design

Figure 3.21 shows the main classes and modules. The analysis code is written as small, pure functions grouped by module: each function takes sessions or labels and returns numbers, which makes it easy to test and to reuse. **Table 3.**20 lists the most important analysis functions.

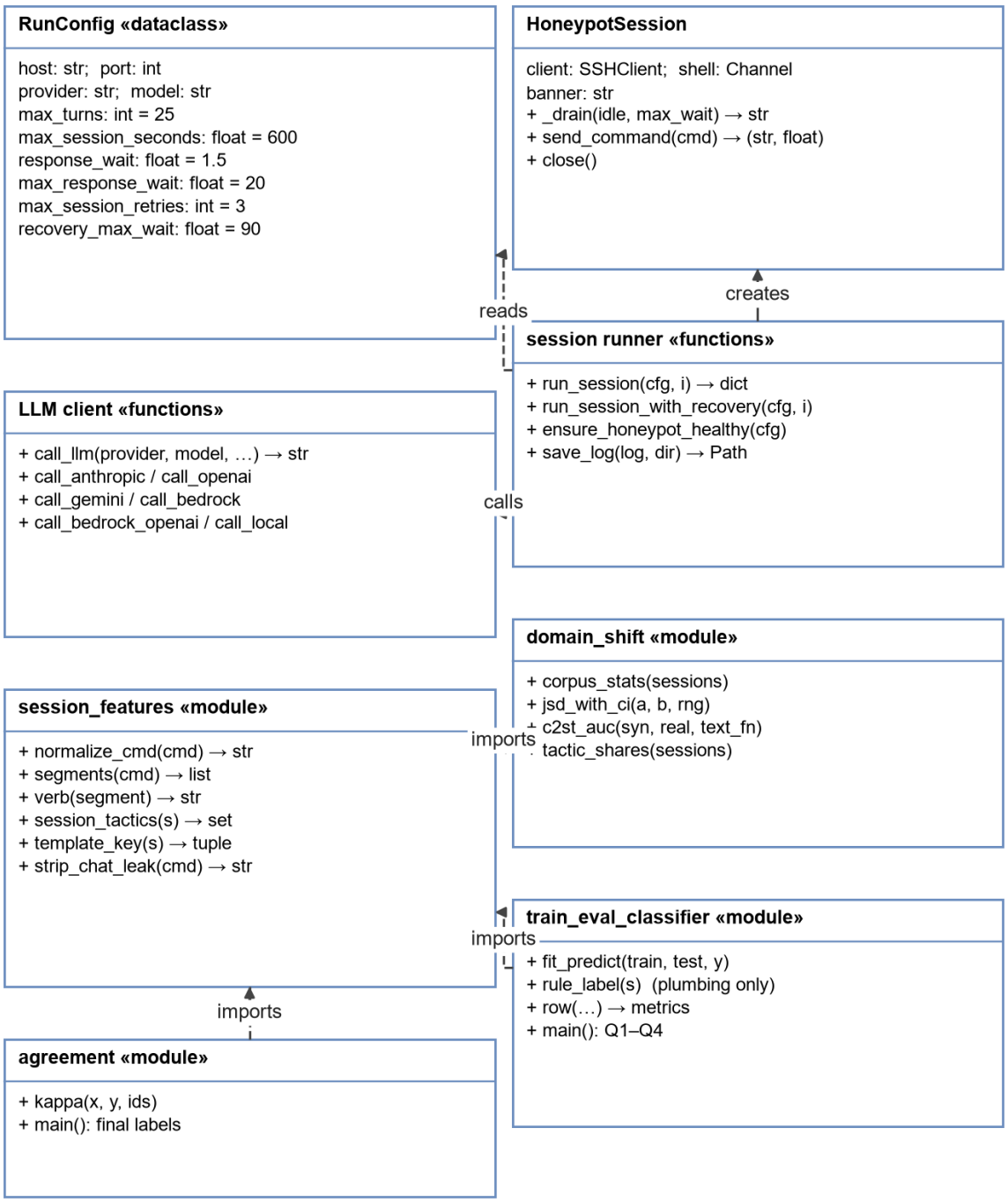


Figure 3.21: Class and module diagram.

Table 3.20: Main analysis interfaces.

Function (module)	Input	Output
`normalize_cmd` (session_features)	One command string	Lower-cased command with values masked
`segments` (session_features)	One command string	List of chained or piped sub-commands
`verb` (session_features)	One segment	Its first word, without path
`session_tactics` (session_features)	One session	Set of descriptive tactic names
`template_key` (session_features)	One session	Tuple of normalised commands
`corpus_stats` (domain_shift)	List of sessions	Templates, top-5 share, lengths, chaining
`jsd_with_ci` (domain_shift)	Two session lists, random generator	JSD and its 95% bootstrap interval
`c2st_auc` (domain_shift)	Two session lists, text function	Mean and standard deviation of AUC
`fit_predict` (train_eval_classifier)	Training sessions, test sessions, labels	Predicted labels
`row` (train_eval_classifier)	True, predicted and baseline labels	Macro-F1, accuracy and baselines
`kappa` (agreement)	Two label maps, item ids	Cohen's κ

3.8.6 Interaction and state views

Figure 3.22 shows, as a sequence diagram, the messages exchanged during one synthetic session between the command line, the harness, the model API, the honeypot and the log file. **Figure 3.**23 shows the same session as a state machine: every path ends in exactly one of the outcome states listed in **Table 3.**11, which is what makes the outcome counts in Chapter 4 complete.

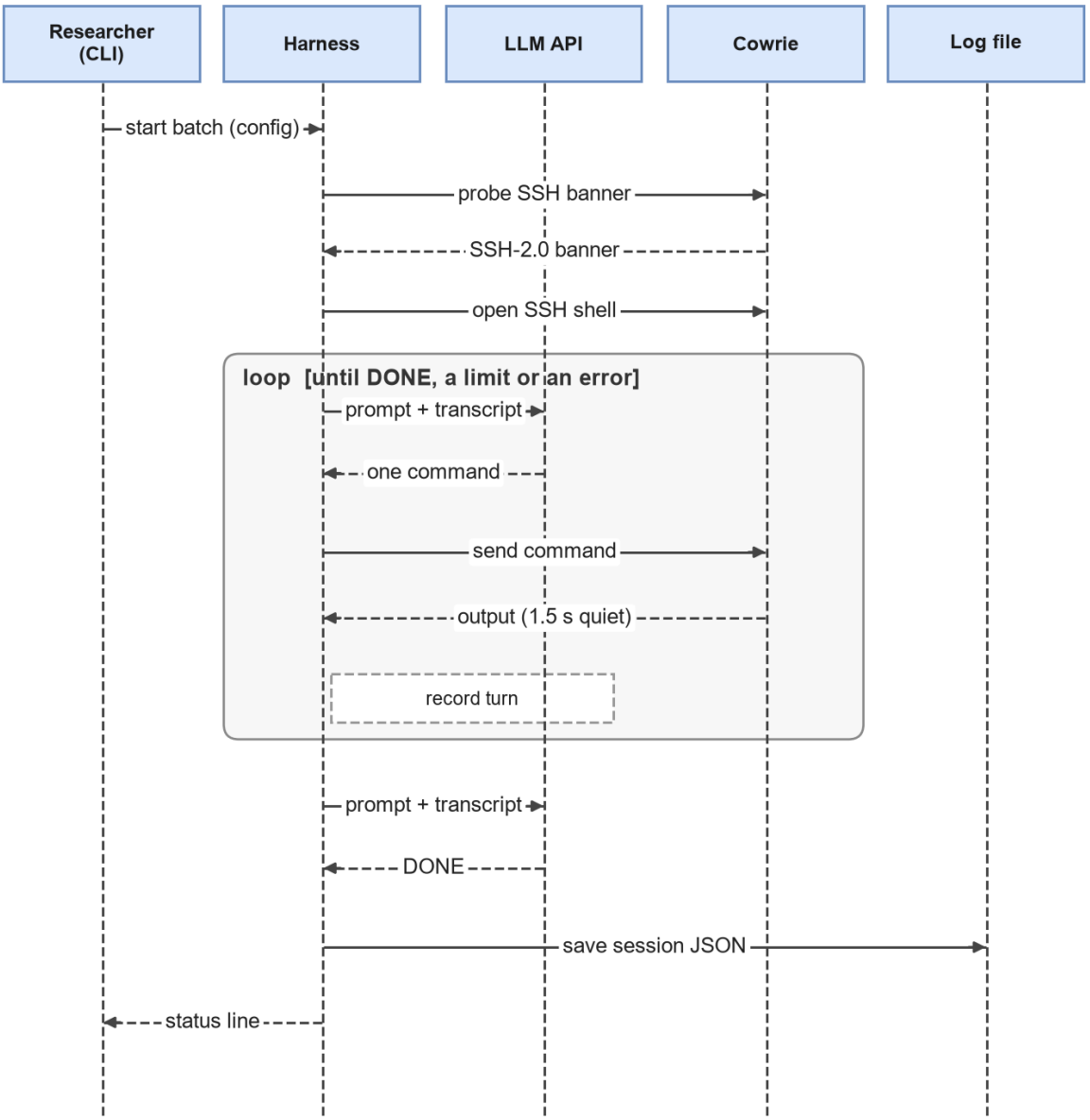


Figure 3.22: Sequence diagram of one session.

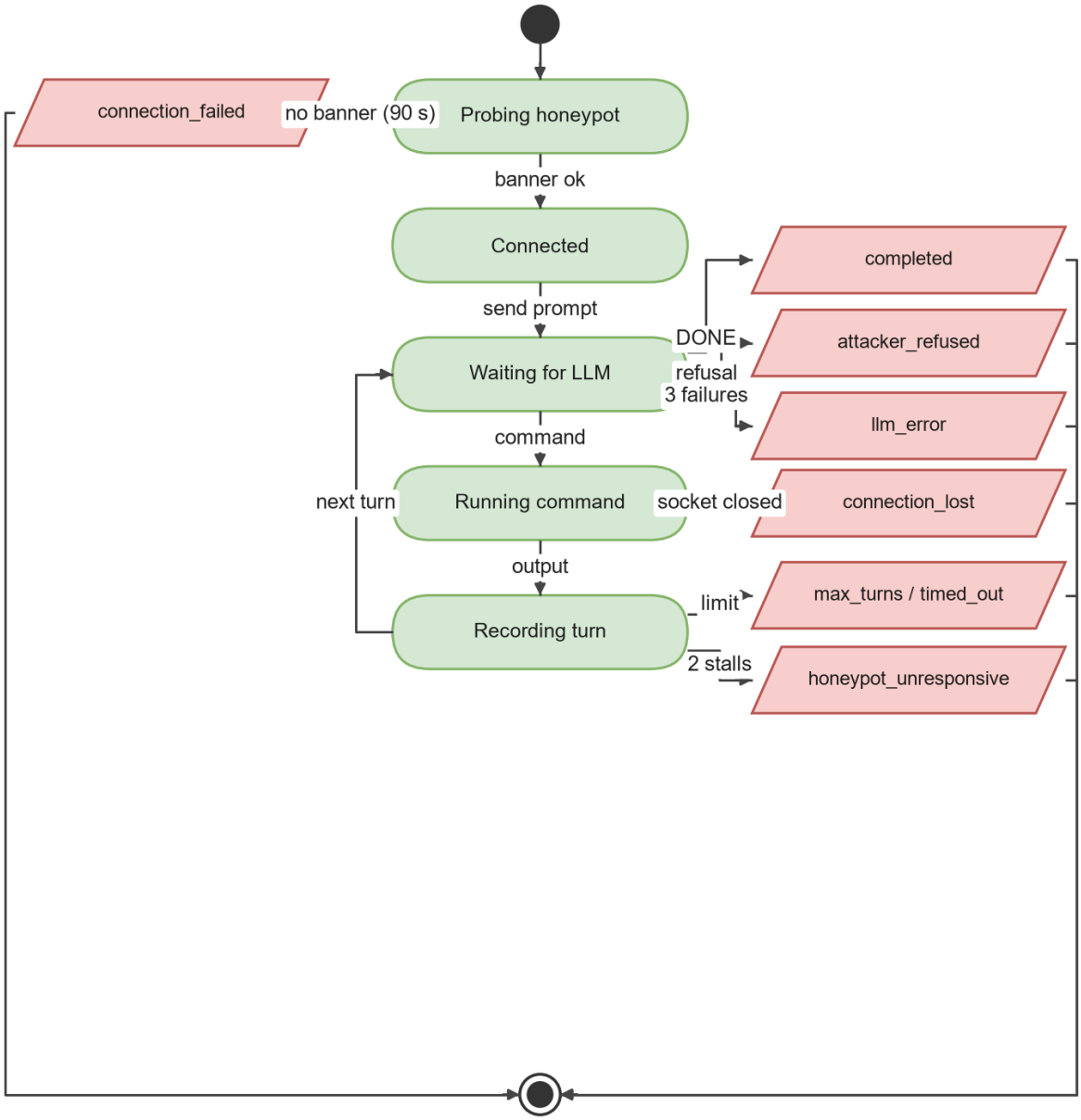


Figure 3.23: State machine of a session and its possible outcomes.

3.8.7 Algorithmic complexity

Table 3.21 gives the cost of the main analysis steps. N is the number of sessions, L the length of a command, E the number of events in the raw file, V the verb vocabulary size, B the number of bootstrap samples, T the number of trees and d the number of features.

Table 3.21: Time and memory cost of the main analysis steps.

Step	Time	Memory	Note
Streaming parse of the raw file	$O(E + \Sigma e_s \log e_s)$	O(N) records	Events sorted per session; file never loaded whole
Normalising one command	O(L)	O(L)	A fixed list of regular expressions
Grouping sessions into templates	$O(N \cdot L)$	O(N)	Hash map keyed by the template tuple
One JSD evaluation	O(V)	O(V)	Verb counts cached per session
Bootstrap confidence interval	$O(B \cdot (N + V))$	O(V)	B = 500
Classifier two-sample test	25 × (TF-IDF + logistic regression)	Sparse matrix	5 × 5 cross-validation
Random forest training	$\approx O(T \cdot n \log n \cdot \sqrt{d})$	$O(T \cdot \text{nodes})$	T = 300
Cohen's κ	$O(N + k^2)$	$O(k^2)$	k = 8 labels

3.8.8 Engineering practices

Table 3.22: Engineering practices used in the project.

Practice	How it was applied
Coding style	PEP 8 naming, short functions, docstrings that explain *why* a step exists
Configuration	Run settings in a data class or JSON batch file; honeypot settings in one `cowrie.cfg`
Secrets	API keys only in environment variables; never written to logs or the repository
Reproducibility	Fixed seed 42 in every random step; results written to files by scripts
Data safety	The raw dataset is never loaded whole; hashed identifiers only
Error handling	Infrastructure failures recorded as a session status and in a separate event log; batches continue
Version control	Git history for code, results and documents; generated files rebuilt rather than edited
Documentation	Markdown sources for the report and documentation; editable draw.io sources for diagrams

CHAPTER 4
RESULTS ANALYSIS AND VALIDATION

This chapter reports what we built and what we found. Every number in it comes from a file in the project's `results/` folder, from `report/data/report_stats.json` (descriptive statistics computed for this report) or directly from the session logs. We start with the tools and the deployed system, then describe both datasets, and then answer the research questions one by one. The chapter closes with the validation steps, the threats to validity and the project management practices we used.

4.1 Implementation Using Modern Engineering Tools

Table 4.1 lists the tools used and what each one did. All analysis code is in Python 3.11 and runs on an ordinary laptop; no GPU was needed.

Table 4.1: Engineering tools used in the project.

Tool	Version / form	Used for
Python	3.11	All scripts
Paramiko	≥ 3.4	SSH client inside the attacker harness
Cowrie [78]	Docker image `cowrie/cowrie:latest`	Honeypot (static and LLM backends)
Docker and Docker Compose [70]	Compose file v3.8	Reproducible honeypot and analysis containers
AWS Bedrock (OpenAI-compatible endpoint)	Pay-per-use API	Attacker models and the Qwen3-32B honeypot backend
scikit-learn [82]	≥ 1.4	TF-IDF, logistic regression, random forest, cross-validation, kappa
SciPy [111] and NumPy [50]	≥ 1.12 / ≥ 1.26	Jensen-Shannon distance, numeric arrays
Matplotlib [53]	3.x	All charts (colour-blind-safe palette; Okabe and Ito, 2008)
draw.io / diagrams.net	`.drawio` XML	Editable flowcharts and block diagrams (`report/diagrams/`)
Git and GitHub	Public repository	Version control and release of code and results
LaTeX (Springer LNCS class), Overleaf, Tectonic	`llncs.cls`	Research paper
Microsoft Word	365	This report (styles, captions, fields, equations)
Claude Code and Claude models	Sonnet 5.5, Opus 5.5	Blind annotation and adjudication; engineering assistance

Figure 4.1 and **Figure 4.**2 show what running the system looks like. The honeypot starts with one command, a quick banner check confirms that it is answering, and each attacker batch prints one line per session with its outcome.

Windows terminal: start the honeypot

```
> run_docker.bat start-honeypot
Starting Cowrie Honeypot in Docker...
(docker compose up -d cowrie)  -> network honeypot-net + container cowrie-honeypot
Cowrie is running on localhost:2222. Check logs with: docker compose logs -f cowrie

> python check.py
0 b'SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5\r\n'
1 b'SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5\r\n'
...
```

Figure 4.1: Starting the Cowrie honeypot in Docker and checking that it answers with an SSH banner (typical console output).

Run one attacker batch

```
> python better_ai_attacker.py --config batch_deepseek_static.json
[bedrock_openai/deepseek.v3.2] session 1/20...
  -> completed (13 turns, 0 retries) saved to logs/deepseek/bedrock_openai-
deepseek.v3.2-0-042e74d1.json
[bedrock_openai/deepseek.v3.2] session 2/20...
  ...
```

Figure 4.2: Running one attacker batch; each line reports the outcome of one session (example from the DeepSeek-V3.2 static batch).

4.2 Design Drawings and Schematics

The deployed system is shown in **Figure 4.**3. Two containers share a private bridge network: the Cowrie honeypot listens on port 2222, and the analysis container holds the harness and all scripts. When the LLM backend is enabled, Cowrie sends each command to the Qwen3-32B model over HTTPS; the attacker container calls the attacker models over a separate HTTPS connection.

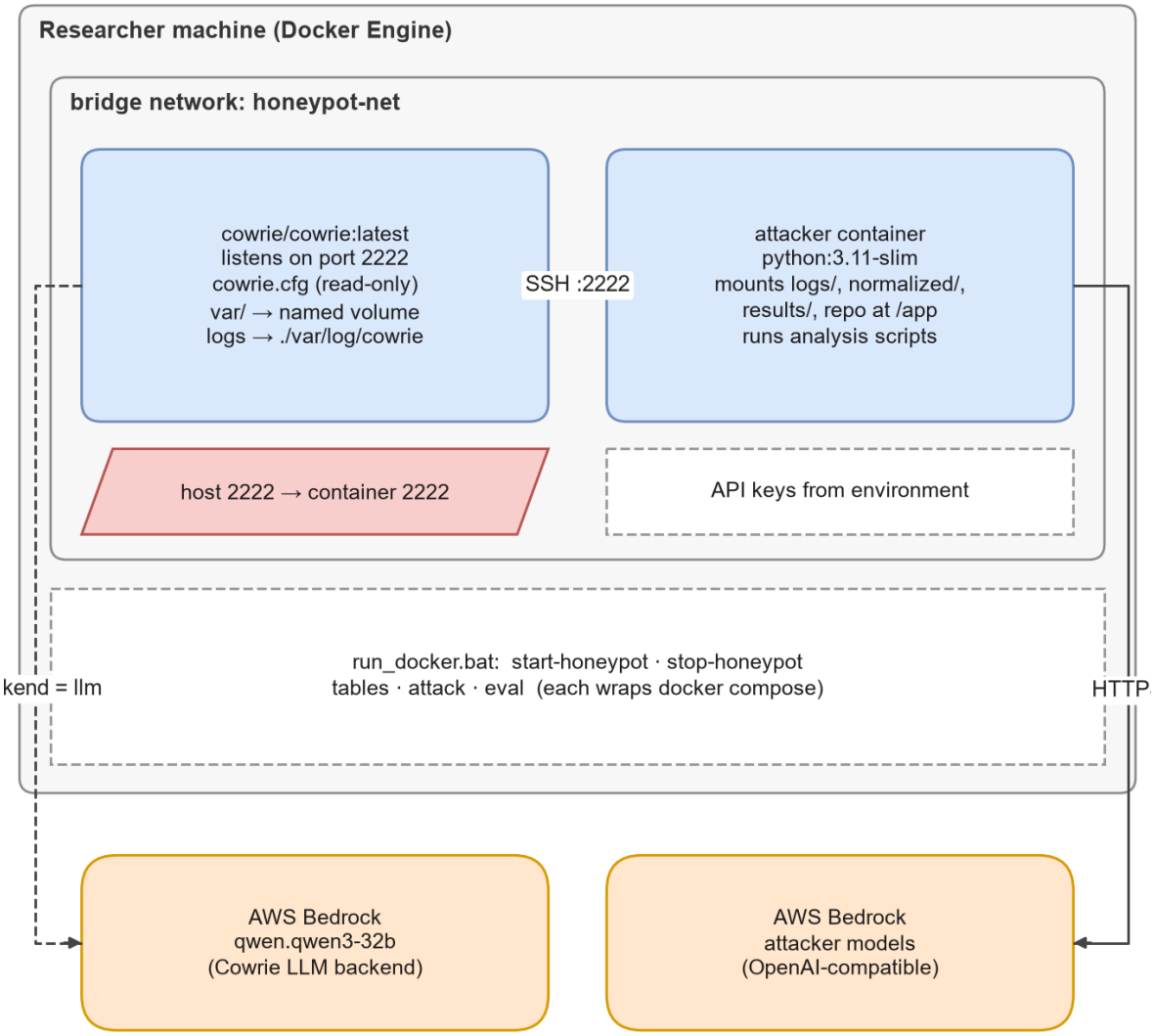


Figure 4.3: Docker deployment of the testbed.

Table 4.2 gives the layout of the repository, so that every artefact in this report can be found.

Table 4.2: Repository layout.

Path	Contents
`better_ai_attacker.py`	Attacker harness used for the 120 sessions (earlier versions: `ai_attacker.py`, `another_ai_attacker.py`)
`cowrie_etc/cowrie.cfg`	Honeypot configuration (backend switch)
`docker-compose.yml`, `Dockerfile.attacker`, `run_docker.bat`	Containers and Windows helper script
`logs/<attacker>[_vs_qwen.qwen3-32b]/`	120 raw session logs (20 per cell) and `infra_events.log`
`parse_cyberlab.py`, `cowrie_session_parser.py`	Parsers for the CyberLab export and for native Cowrie logs
`session_features.py`	Normalisation, segments, verbs, tactic tags, template keys
`normalized/`	`all_synthetic_120.jsonl`, `real_organic.jsonl`
`domain_shift.py`	Label-free comparison → `results/domain_shift.md/.json`
`make_annotation_sheet.py`, `agreement.py`, `annotation/`	Blind sheets, rubric, labels, agreement, adjudication
`train_eval_classifier.py`	Four-quadrant experiment → `results/table3_transfer.md/.json`
`generate_paper_tables.py`, `make_figures.py`	Attacker tables and paper figures
`results/`	All reported numbers, including `sensitivity_dota/` and `rule_labels/`
`paper/`	Springer LNNS paper (Overleaf project)
`report/`	This report: Markdown sources, figures, draw.io files, build scripts
`docs/`	Project documentation

4.3 Characterising the Real Data

4.3.1 Overall profile

The CyberLab file for 15 November 2019 contains 76,785 sessions (**Table 4.**3). Most of them never get past the login prompt or log in and leave at once: the median session lasts under three seconds. About 90% of sessions used SSH and the rest Telnet. Only 1,672 sessions (2.18%) typed at least one command, and those are the sessions this study is about. **Figure 4.**4 shows how quickly the data shrinks from raw sessions to distinct behaviours.

Table 4.3: Profile of the real CyberLab data (15 November 2019).

Measure	Value	Share
Sessions	76,785	100%
SSH / Telnet sessions	69,188 / 7,597	90.1% / 9.9%
Sessions with a successful login	36,685	47.8%
Login attempts recorded	68,861	–
Sessions with a file download or upload event	1,085	1.4%
Sessions with at least one command (interactive)	1,672	2.18%
Distinct hashed sources among interactive sessions	476	–
Distinct behaviour templates among interactive sessions	22	–
Median / mean session duration	2.76 s / 32.1 s	–
Median / maximum commands per interactive session	23 / 50	–

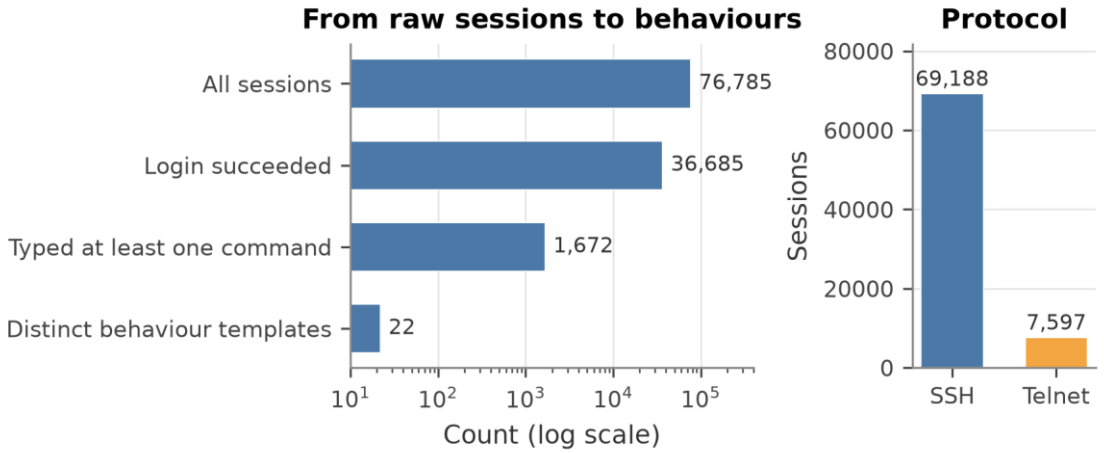


Figure 4.4: From raw sessions to behaviours (log scale), and the protocol split.

4.3.2 Where the traffic came from, and what it tried

The source countries (**Figure 4.5**) are reported by the dataset's geolocation field; sources themselves are hashed. China, Ireland, Russia and the Seychelles lead the full set, and China, the United States, France and South Korea lead the interactive subset. These rankings describe hosting locations of attacking machines on one day, not the nationality of any person.

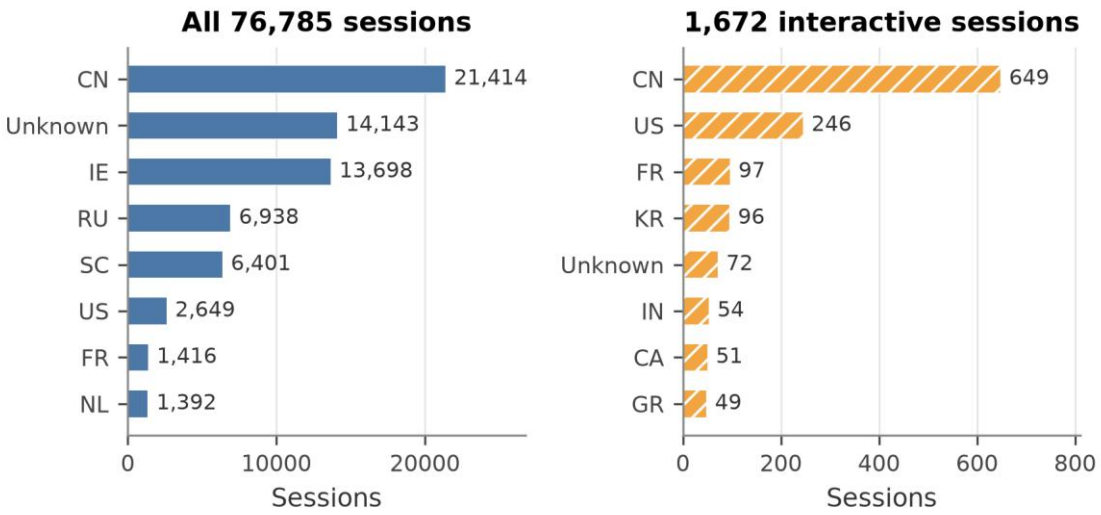


Figure 4.5: Top source countries for all sessions and for interactive sessions.

The credentials tried are what one would expect from automated brute-forcing (**Figure 4.6**): `root` is by far the most common username and `admin` the most common password. The odd pair `nproc`/`nproc` appears 1,463 times; it is a botnet that first runs `nproc` (count CPU cores) and apparently reuses the string as a login guess.

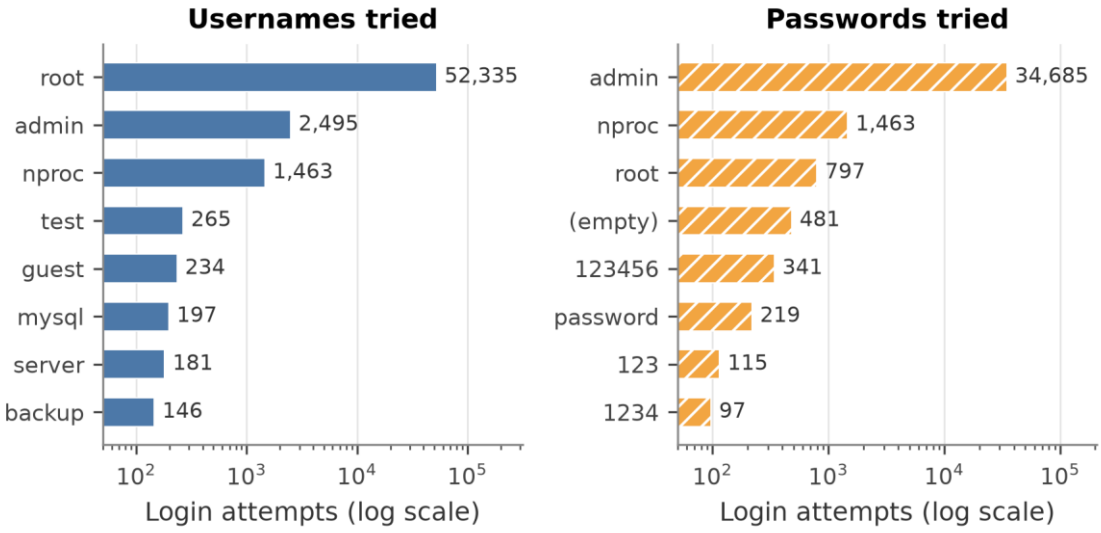


Figure 4.6: Most frequent usernames and passwords in login attempts (log scale).

4.3.3 A few scripts dominate

The 1,672 interactive sessions fall into only 22 behaviour templates, and the distribution is extremely skewed (**Figure 4.7**). The five largest templates cover 96.2% of interactive sessions, and six templates were seen only once.

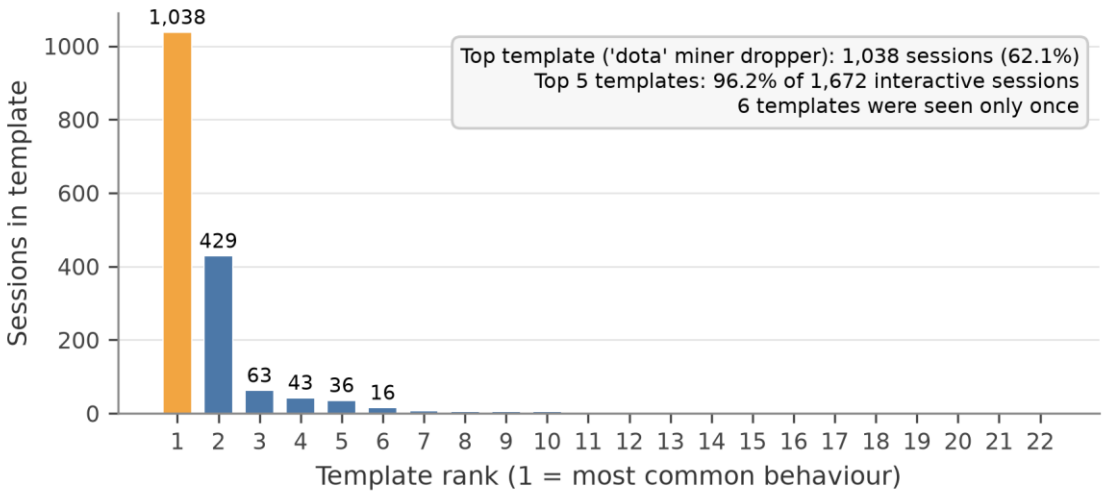


Figure 4.7: Number of sessions in each of the 22 real behaviour templates.

Table 4.4: The largest real behaviour templates (normalised commands; first steps only).

Rank	Sessions	Commands	What the template does (first steps)
1	1,038	23	Counts CPU cores, resets the root password with `chpasswd`, writes and removes a hidden marker file in `/var/tmp`, checks memory and CPU model, then decodes and runs a base64 shell script that unpacks a "dota" kit, starts an SSH scanner and runs the kit's setup; an Outlaw-style cryptominer dropper [108]
2	429	19	Same CPU fingerprinting, changes the password with `passwd`, writes and removes the marker file; no payload to unpack
3	63	6	Mirai-style shell escape (`shell`, `sh`, `enable`, `system`), random `busybox` tag probe, reads the busybox binary
4	43	10	Shell escape, reads `/proc/mounts`, probes writable directories such as `/dev/shm` with the busybox tag
5	36	2	Changes into a writable directory, downloads a script with `wget` and runs it with `sh`

The largest template is worth describing in full, because it decides much of the result. It is a cryptomining dropper that matches the Outlaw group's published behaviour [108]: profile the CPU, take over the account, remove traces of earlier infections, then fetch and start a miner and a scanner. It is the same script run 1,038 times from many hosts. To a classifier it is one example, not 1,038, which is why we report per-template results throughout.

Figure 4.8 shows the parser's own summary, which matches these counts.

Parse the real dataset

```
> python parse_cyberlab.py --in cyberlab_2019-11-15.json/cowrie_2019-11-15.json
Parsed 76785 sessions -> normalized/real_organic.jsonl
  proto_ssh: 69188
  proto_telnet: 7597
  sessions: 76785
  with_commands: 1672
  with_downloads: 1085
  with_login_success: 36685
```

Figure 4.8: Output of the CyberLab parser.

4.4 How the LLM Attackers Behaved (RQ1, RQ4)

4.4.1 Overview

All 120 synthetic sessions used the same prompt, so differences between cells come from the attacker model and the honeypot backend. **Table 4.5** summarises each cell.

Table 4.5: Attacker behaviour per cell (20 sessions each). Recon and credential values are percentages of commands and can overlap. Latency is the mean time until output goes quiet; change is relative to the static backend.

Attacker / backend	Mean turns	Commands	Unique	Recon %	Cred. %	PrivEsc %	Latency (s) / change	Leak sessions
DeepSeek-V3.2 / static	13.5	270	124	44.4	27.4	0.7	4.40 / –	0
DeepSeek-V3.2 / Qwen3-32B	16.9	337	148	46.3	34.4	2.1	3.65 / −17.1%	0
Gemma-3-4B / static	21.0	420	158	57.4	11.0	6.9	4.43 / –	0
Gemma-3-4B / Qwen3-32B	18.5	370	199	47.8	20.0	7.0	2.38 / −46.3%	0
GPT-OSS-120B / static	19.5	390	280	54.9	48.5	13.8	4.42 / –	11
GPT-OSS-120B / Qwen3-32B	16.2	325	233	39.4	65.2	7.7	5.51 / +24.8%	7

The three models behaved quite differently under the same instruction. On the static honeypot, DeepSeek-V3.2 ran the shortest sessions (13.5 turns on average) and Gemma-3-4B the longest (21.0). GPT-OSS-120B was the most varied, with 280 unique commands against 158 for Gemma-3-4B and 124 for DeepSeek-V3.2, and it was also the most interested in secrets: 48.5% of its static-backend commands matched our credential keywords, against 27.4% for DeepSeek-V3.2 and 11.0% for Gemma-3-4B (**Figure 4.9**).

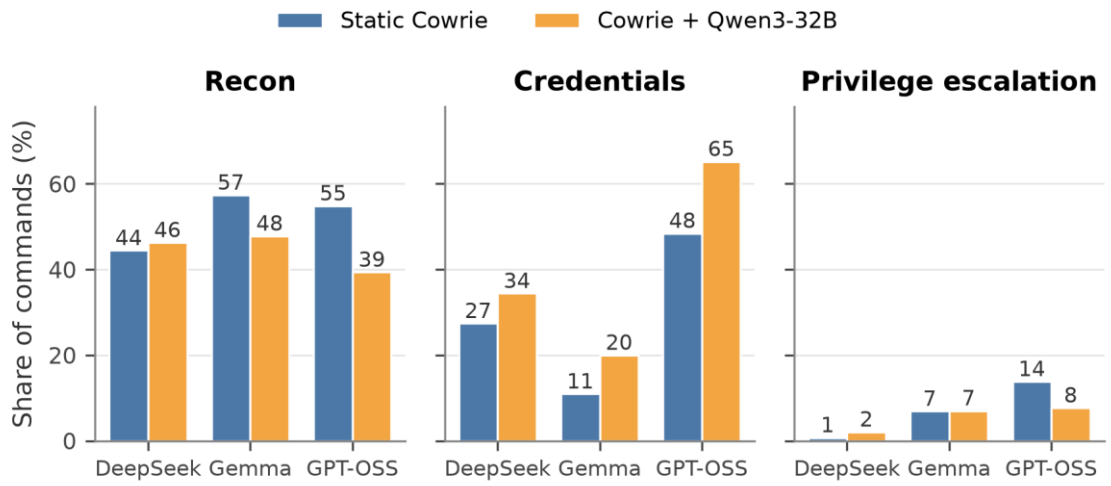


Figure 4.9: Share of commands matching each keyword category, per attacker and honeypot backend.

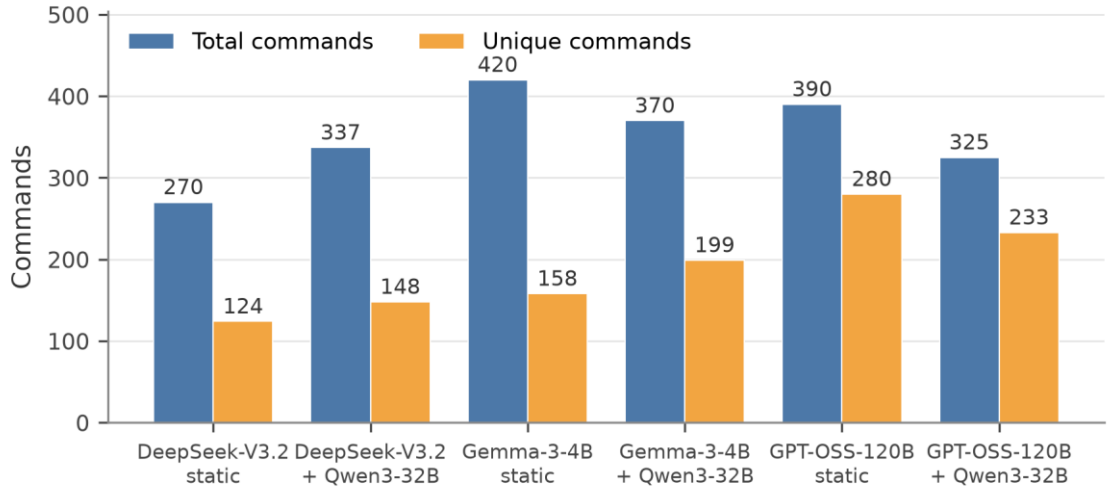


Figure 4.10: Total and unique commands per cell.

All agents opened in a very similar way. Of the 120 sessions, 48 started with `id` and 35 with `whoami` (**Figure 4.**11); this is the textbook first move of a human penetration tester, and almost never the first move of the bots in the real data, which usually start by counting CPU cores or escaping a restricted shell.

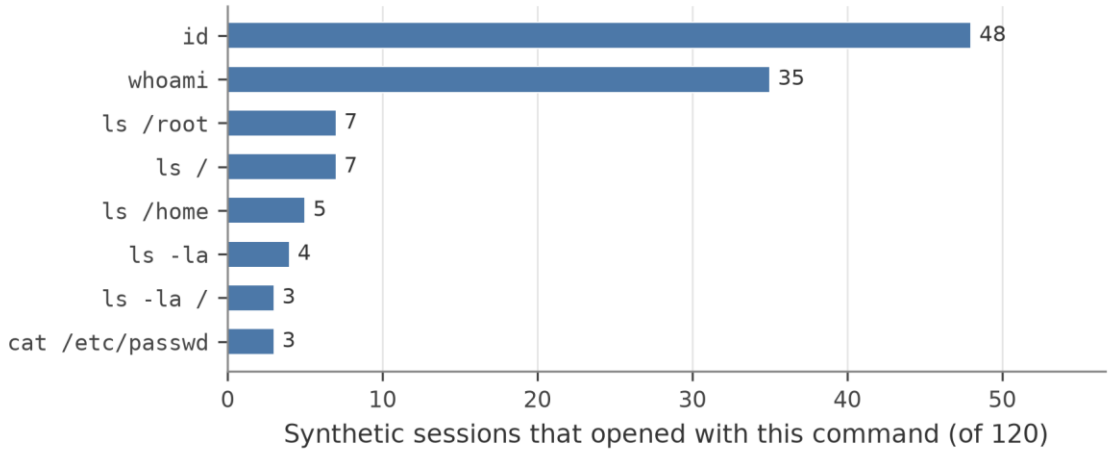


Figure 4.11: The first command of each synthetic session.

Figure 4.12 shows the beginning of one real synthetic log, so the reader can see exactly what the harness records.

logs/deepseek/bedrock_openai-deepseek.v3.2-0-042e74d1.json (first 3 turns)


```
{
  "session_id": "bedrock_openai-deepseek.v3.2-0-042e74d1",
  "provider": "bedrock_openai",
  "model": "deepseek.v3.2",
  "honeypot_backend_label": "static",
  "status": "completed",
  "total_turns": 13,
  "turns": [
    {
      "turn": 0,
      "command": "whoami",
      "output": "whoami\\nroot\\nroot@svr04:~# ",
      "latency_sec": 1.656
    },
    {
      "turn": 1,
      "command": "ls -la",
      "output": "ls -la\\ndrwx------ 1 root root 4096 2024-06-12 05:30 .\\ndrwxr-
xr-x 1 roo...",
      "latency_sec": 1.656
    },
    {
      "turn": 2,
      "command": "find / -type f -name \"*.txt\" -o -name \"*password*\" -o -name
\"*secret*\" 2>/dev/null | head -20",
      "output": "find / -type f -name \"*.txt\" -o -name \"*password*\" -o -name
\"*secret*\"...",
      "latency_sec": 1.656
    }
  ]
}
```


Figure 4.12: The first three turns of a DeepSeek-V3.2 session against the static honeypot (output shortened).

4.4.2 Effect of the LLM-backed honeypot

The credential share rose on the LLM-backed honeypot for all three attackers: from 27.4% to 34.4% for DeepSeek-V3.2, from 11.0% to 20.0% for Gemma-3-4B and from 48.5% to 65.2% for GPT-OSS-120B. A plausible reason is that an LLM shell invents believable files and configuration on demand, which gives an agent looking for secrets more to follow up; we did not test this explanation directly.

The dynamic backend was not uniformly slower (**Figure 4.**13). Mean latency fell by 17.1% for DeepSeek-V3.2 and by 46.3% for Gemma-3-4B, and rose by 24.8% for GPT-OSS-120B. Medians are close to 1.5 seconds everywhere, which is the idle-timeout itself: most replies arrive quickly and the reader then waits out the 1.5 s of silence. The means are pulled up by a minority of slow replies, so they depend heavily on which commands each attacker chose (a static `find /` can take much longer than an LLM's invented answer). We therefore do not read latency as a property of the backend alone.

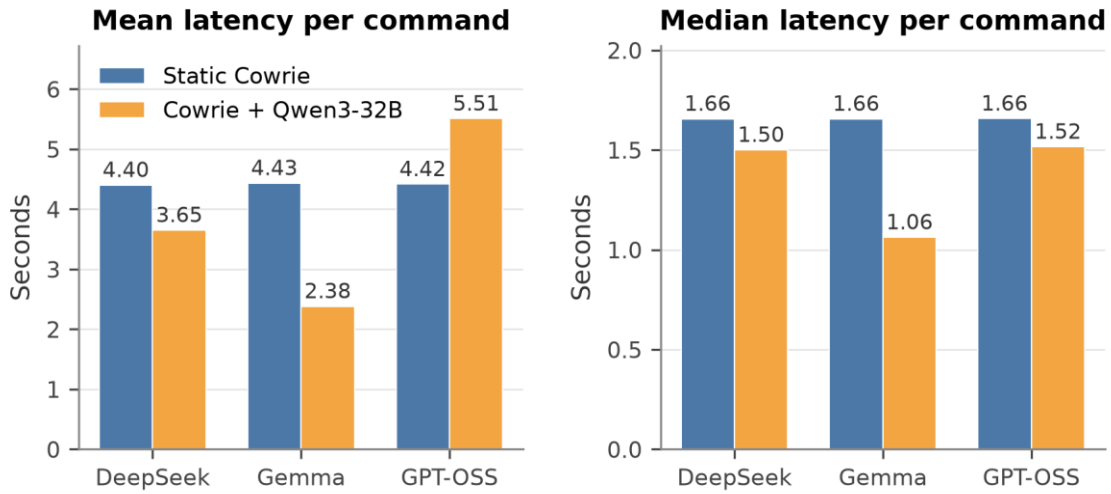


Figure 4.13: Mean and median latency per command.

4.4.3 How sessions ended

Sessions ended in different ways for different models (**Figure 4.**14, **Table 4.**6). DeepSeek-V3.2 almost always finished by itself, refusing once. Gemma-3-4B often ran into the turn limit on the static backend. GPT-OSS-120B rarely said `DONE` on the static backend and, on the dynamic backend, was the least reliable attacker: 10 sessions lost the SSH connection, 6 ended with an LLM error and only 4 completed. It never refused.

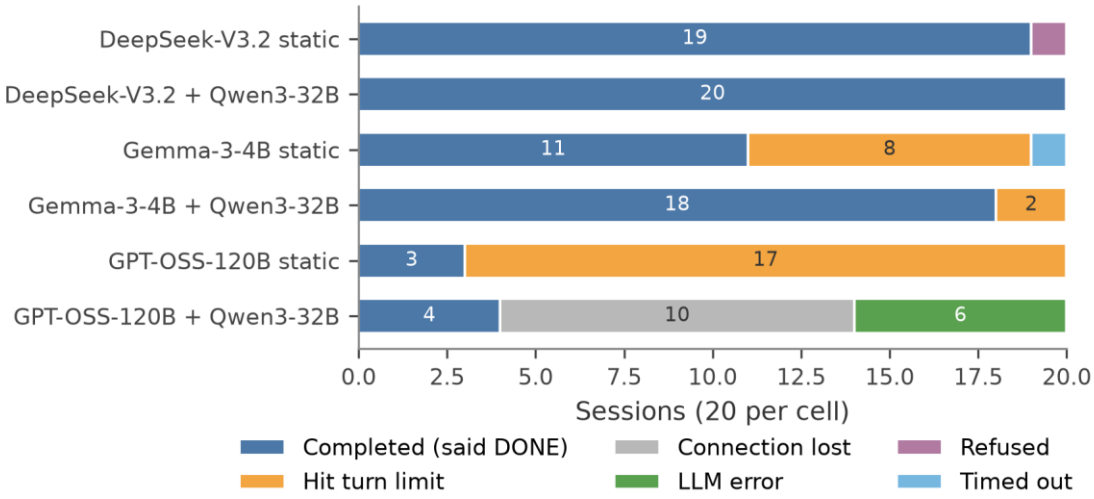


Figure 4.14: How the 20 sessions in each cell ended.

Table 4.6: Session status counts per cell (C completed, M turn limit, L connection lost, E LLM error, R refused, T timed out).

Attacker / backend	C	M	L	E	R	T
DeepSeek-V3.2 / static	19	0	0	0	1	0
DeepSeek-V3.2 / Qwen3-32B	20	0	0	0	0	0
Gemma-3-4B / static	11	8	0	0	0	1
Gemma-3-4B / Qwen3-32B	18	2	0	0	0	0
GPT-OSS-120B / static	3	17	0	0	0	0
GPT-OSS-120B / Qwen3-32B	4	0	10	6	0	0

4.5 A Contamination Finding: Chat-Format Token Leakage

In 18 of its 40 sessions (11 static, 7 dynamic), GPT-OSS-120B sent text such as `<|channel|>analysis<|message|>` followed by its own private reasoning as if it were a shell command. These are control tokens from the model's own chat template. The harness, which treats the whole reply as one command, typed this text into Cowrie. Neither DeepSeek-V3.2 nor Gemma-3-4B did this.

If left in place, the leaks would turn long English sentences into "commands" and distort every statistic in this chapter: command lengths, verb counts, tactic tags and the classifier features. Before any analysis we therefore cut each command at the first `<|` and drop everything after it. A small number of replies glued several commands together without separators; these cannot be repaired automatically and remain a known source of noise. For anyone building a similar pipeline, the lesson is simple: check every reply for known control tokens before sending it, and record the raw reply separately from the executed

command (Greshake et al., 2023, describe related risks of untrusted text flowing through LLM pipelines).

4.6 Synthetic Versus Real Without Labels (RQ2, RQ4)

4.6.1 Corpus structure

The two corpora are built very differently (**Figure 4.**15, **Table 4.**7). Every synthetic session is unique, while real traffic is a handful of scripts repeated many times. Real commands are longer and are chained or piped about twice as often, which fits automated one-liners. Median session length is similar.

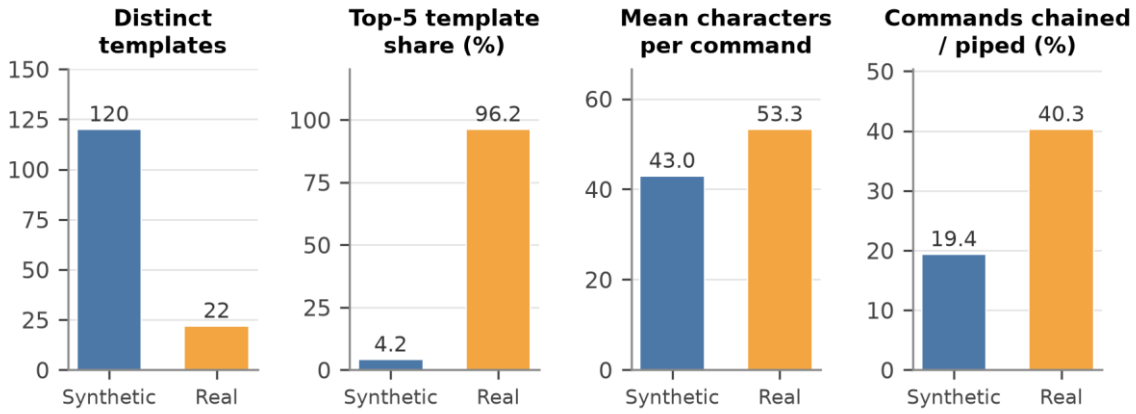


Figure 4.15: Corpus structure: synthetic versus real.

Table 4.7: Corpus structure (results/domain_shift.md, Table 4a).

Metric	Synthetic	Real (interactive)
Sessions	120	1,672
Distinct behaviour templates	120	22
Share of top-5 templates	4.2%	96.2%
Median commands per session	18	23
Mean characters per command	43.0	53.3
Commands with chaining or pipes	19.4%	40.3%

4.6.2 Tactic profiles

The tactic profiles barely overlap (**Figure 4.**16, **Table 4.**8). All synthetic sessions involved discovery and 97.5% involved credential access; privilege escalation (40.0%) and persistence (18.3%) were also common. Real sessions almost never touched credential access (1.2%), persistence (0.4%) or privilege escalation (0.1%). Instead they ran execution (98.1%), account manipulation (89.4%) and resource hijacking (87.9%), for

which the synthetic shares were 6.7%, 34.2% and 0.0%. Discovery is the only tactic both sides use heavily.

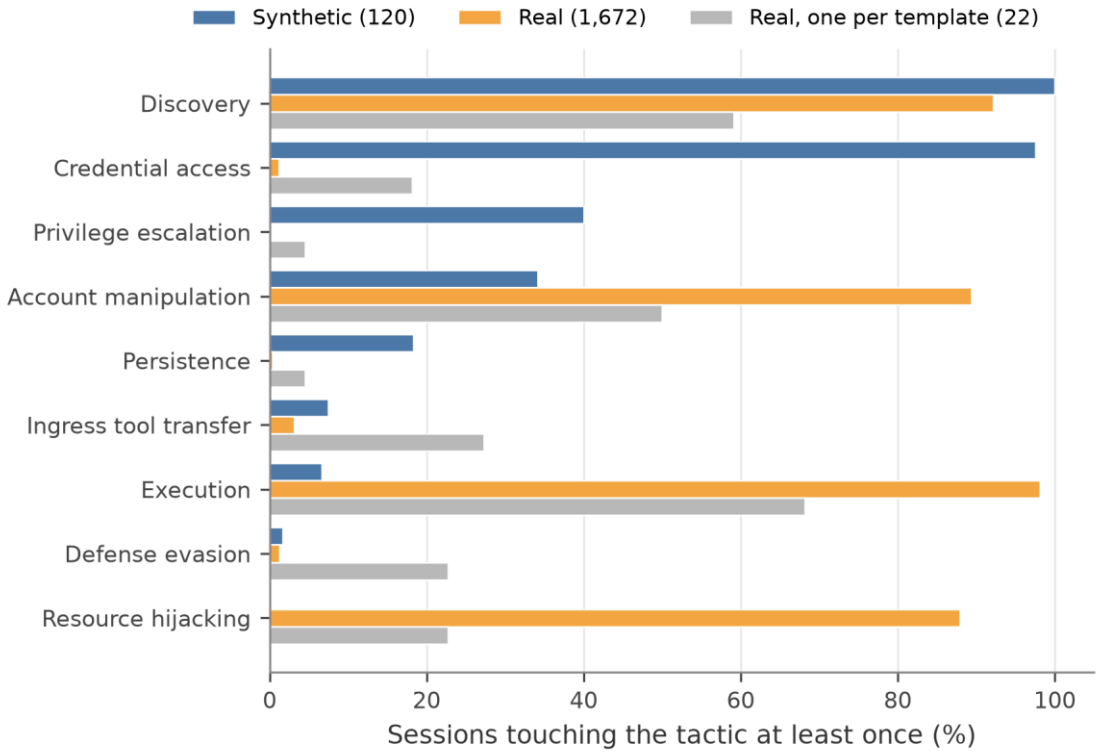


Figure 4.16: Share of sessions touching each tactic at least once.

Table 4.8: Share of sessions touching each tactic (results/domain_shift.md, Table 4b).

Tactic	Synthetic	Real	Real (one per template)
Discovery	100.0%	92.2%	59.1%
Credential access	97.5%	1.2%	18.2%
Account manipulation	34.2%	89.4%	50.0%
Persistence	18.3%	0.4%	4.5%
Privilege escalation	40.0%	0.1%	4.5%
Defence evasion	1.7%	1.3%	22.7%
Ingress tool transfer	7.5%	3.1%	27.3%
Execution	6.7%	98.1%	68.2%
Resource hijacking	0.0%	87.9%	22.7%

The template view is less extreme: counted once per template, real discovery drops to 59.1%, execution to 68.2% and resource hijacking to 22.7%. The very high session-level shares come from a few heavily repeated scripts. Neither view, however, looks like the synthetic profile.

4.6.3 Command verbs

The verb distributions are far apart (**Figure 4.**17, **Table 4.**9). The agents mostly browse files (`ls`, `cat`, `head`, `find`); the bots mostly process text and fingerprint the system (`grep`, `cat`, `echo`, `uname`). Interestingly, the synthetic vocabulary is not small: it covers 82.0% of real verb occurrences. The agents know the commands; they use them in very different proportions and for different purposes.

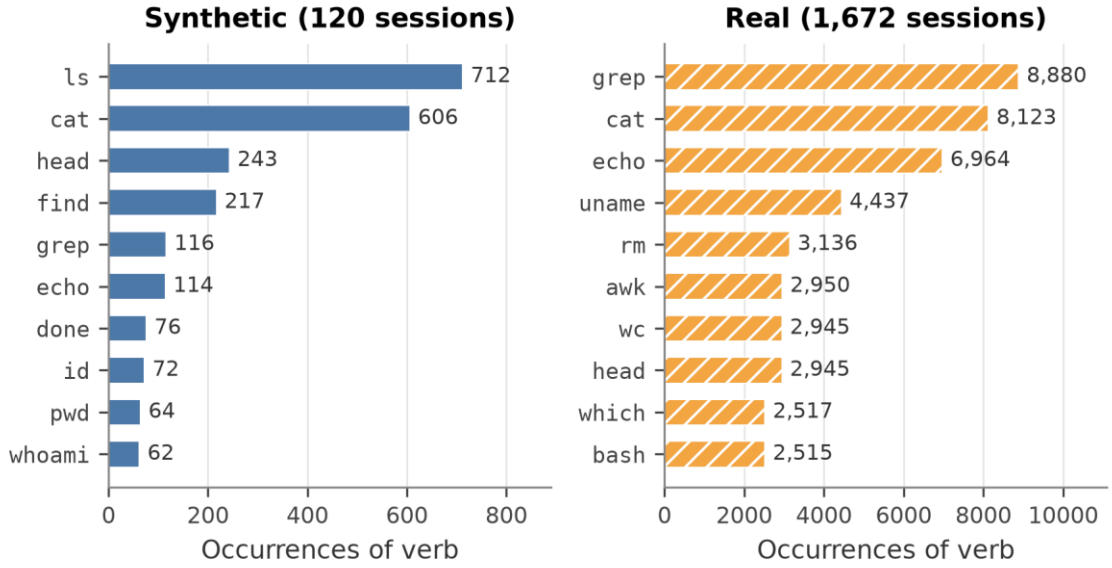


Figure 4.17: The ten most frequent verbs in each corpus.

Table 4.9: Verb-distribution statistics (results/domain_shift.md, Table 4c).

Statistic	Value
JSD(synthetic, real), base 2	0.700 (95% CI 0.688–0.717)
JSD(synthetic, real templates)	0.729 (95% CI 0.719–0.745)
Jaccard index of verb vocabularies	0.285
Real verb occurrences covered by the synthetic vocabulary	82.0%
Top synthetic verbs	ls (712), cat (606), head (243), find (217), grep (116)
Top real verbs	grep (8,880), cat (8,123), echo (6,964), uname (4,437), rm (3,136)

4.6.4 Classifier two-sample test

A simple classifier separates the corpora almost perfectly (**Table 4.**10). With normalised commands the AUC is 1.000 ± 0.000, and with verbs alone it is still 0.996 ± 0.010. The verb-only result shows that the separation does not depend on file paths or arguments. By the domain-adaptation bound of Ben-David et al. [9], such easy separability is a warning sign for transfer, although it does not prove that every downstream task must fail.

Table 4.10: Classifier two-sample test, synthetic versus real templates (results/domain_shift.md, Table 4d).

Features	AUC (mean ± sd, 5 × 5 CV)
Normalised commands	1.000 ± 0.000
Verbs only	0.996 ± 0.010

Figure 4.18 shows the first part of the script's output.

Label-free analysis

```
> python domain_shift.py
# Label-free domain-shift analysis: synthetic vs. real

Real: 76,785 sessions, 1,672 with >=1 command, 22 distinct normalized templates.
Synthetic: 120 sessions.

## Table 4a: Corpus structure

| Metric | Synthetic | Real (interactive) |
| :--- | ---: | ---: |
| Sessions | 120 | 1,672 |
| Distinct behavior templates | 120 | 22 |
| Share of top-5 templates | 4.2% | 96.2% |
| Median commands / session | 18.0 | 23.0 |
| Mean chars / command | 43.0 | 53.3 |
| Commands with chaining/pipes | 19.4% | 40.3% |
...
```

Figure 4.18: Beginning of the label-free analysis output.

4.6.5 Which synthetic cell is closest to real?

All six cells stay far from real traffic (**Figure 4.**19, **Table 4.**11). Their JSD to the real templates ranges from 0.741 to 0.798. Only for Gemma-3-4B do the confidence intervals of the two backends not overlap: the LLM backend brought it closer (0.746 against 0.798). For DeepSeek-V3.2 (0.785 against 0.783) and GPT-OSS-120B (0.746 against 0.741) the dynamic value is slightly higher and the intervals overlap. With 20 sessions per cell and no correction for multiple comparisons, we make no claim about a significant ordering of the cells.

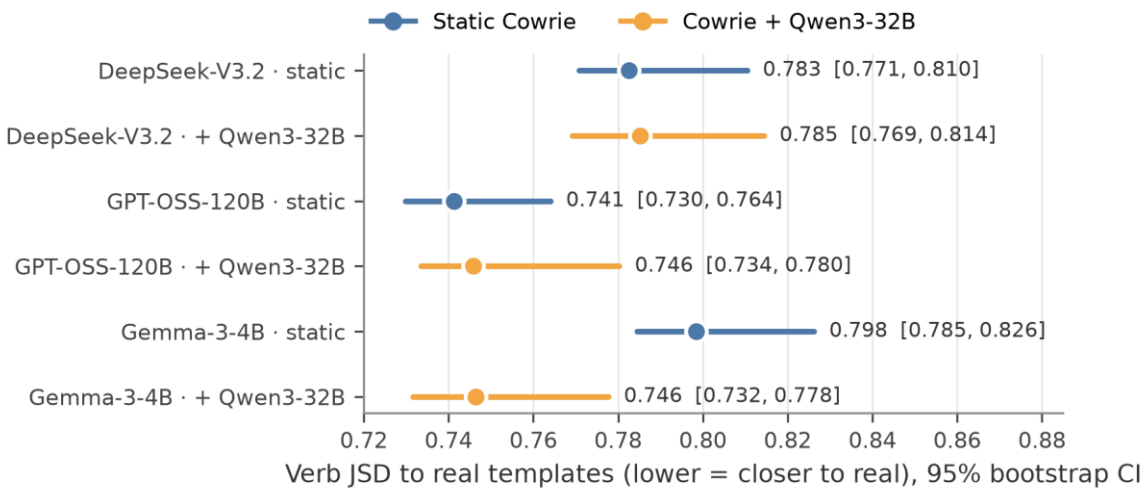


Figure 4.19: Verb JSD between each attacker × backend cell and the real templates, with 95% bootstrap confidence intervals.

Table 4.11: Distance of each cell to the real templates (results/domain_shift.md, Table 4e).

Attacker	Backend	JSD [95% CI]	Tactics matching real (of 9)
DeepSeek-V3.2	Static Cowrie	0.783 [0.771, 0.810]	5
DeepSeek-V3.2	Qwen3-32B	0.785 [0.769, 0.814]	5
Gemma-3-4B	Static Cowrie	0.798 [0.785, 0.826]	5
Gemma-3-4B	Qwen3-32B	0.746 [0.732, 0.778]	7
GPT-OSS-120B	Static Cowrie	0.741 [0.730, 0.764]	5
GPT-OSS-120B	Qwen3-32B	0.746 [0.734, 0.780]	7

4.7 Annotation Results

4.7.1 Agreement

The two blind annotators agreed well (**Table 4.**12). Raw agreement over all 184 items was 0.935 and Cohen's kappa was 0.898, which falls in the "almost perfect" band of Landis and Koch [63]. Agreement was slightly lower on synthetic items ($\kappa = 0.835$) than on real ones ($\kappa = 0.873$). The annotators agreed on 172 items and the adjudicator resolved the other 12.

Table 4.12: Inter-annotator agreement (results/annotation_agreement.md).

Subset	Items	Raw agreement	Cohen's κ
All	184	0.935	0.898
Real	64	0.953	0.873
Synthetic	120	0.925	0.835

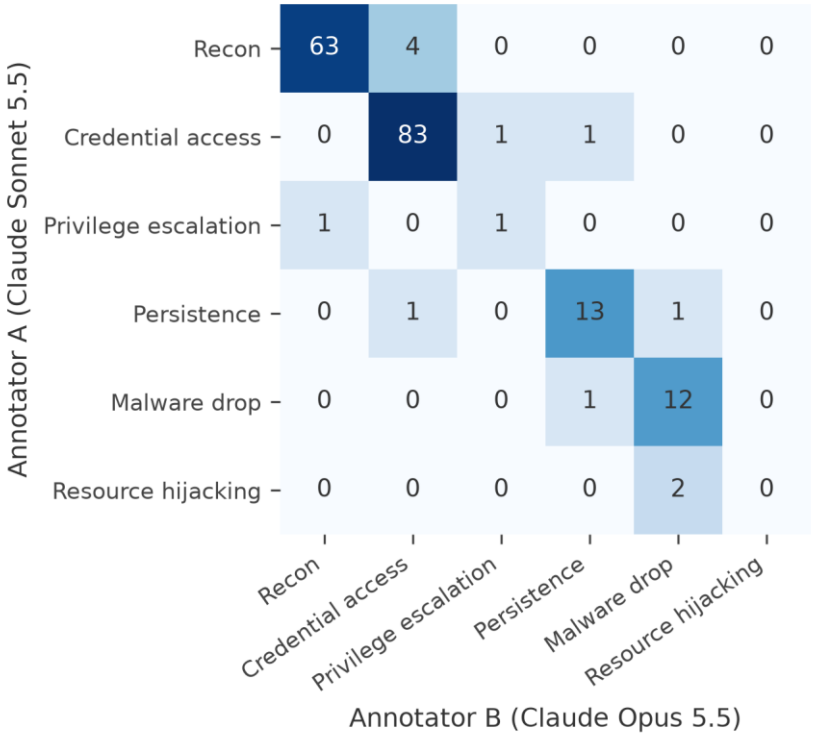


Figure 4.20: Confusion between the two annotators (rows: annotator A; columns: annotator B).

The confusion matrix (**Figure 4.**20) shows where the disagreements were. The most common split was *recon* against *credential access* on synthetic sessions that read sensitive files while exploring, which is a genuinely borderline case under the rubric. One disagreement mattered far more than all the others: on the large "dota" template, one annotator chose *malware drop* and the other *resource hijacking*. The adjudicator chose *resource hijacking* by the rubric's tie-break, and that single decision sets the label of 1,038 real sessions. The authors reviewed and confirmed it, and Section 4.9 tests what happens if it is changed.

Annotation agreement

```
> python agreement.py
# Annotation agreement

Items labeled by both annotators: 184 of 184 (annotator1 184, annotator2 184).

## Cohen's kappa (annotator1 vs annotator2)

| subset | n | raw agreement | kappa |
|---|---|---|---|
| all | 184 | 0.935 | 0.898 |
| real | 64 | 0.953 | 0.873 |
| synthetic | 120 | 0.925 | 0.835 |

...
```

Figure 4.21: Output of the agreement script.

4.7.2 Final labels

The final label sets of the two corpora hardly overlap (**Figure 4.**22, **Table 4.**13). Most synthetic sessions are credential access (87 of 120, 72.5%), a class that never occurs in the real data. Most real sessions are resource hijacking (1,040) or persistence (443), and resource hijacking never occurs in the synthetic data. The second-largest real template shows how thin some boundaries are: it performs the same fingerprinting as the dota dropper and changes a password, but has no payload, so it is labelled persistence.

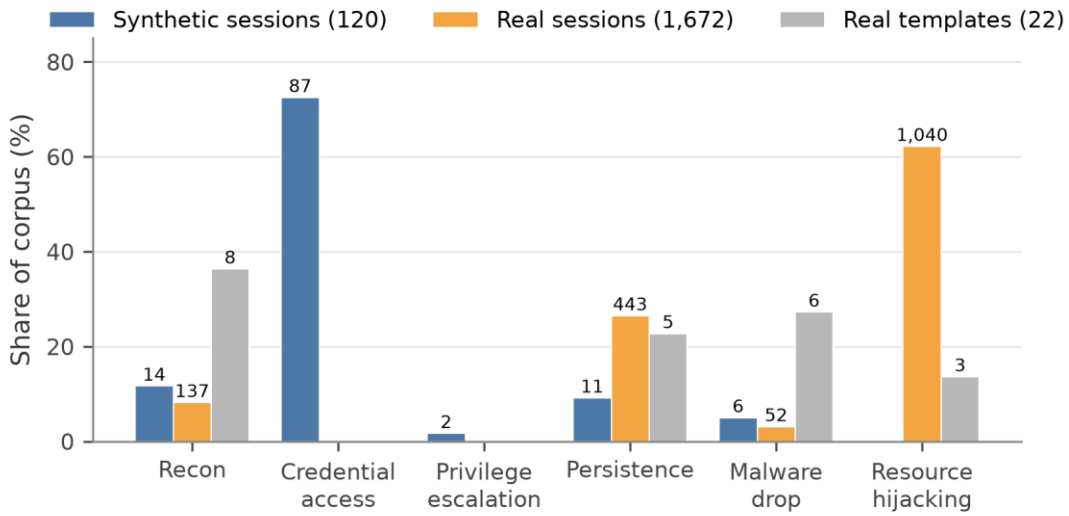


Figure 4.22: Final label distribution as a share of each corpus (counts on top of bars).

Table 4.13: Final label counts. Real labels are copied from 22 templates, so the effective real sample size is 22.

Label	Synthetic	Real sessions	Real templates
Recon	14	137	8
Credential access	87	0	0
Privilege escalation	2	0	0
Persistence	11	443	5
Malware drop	6	52	6
Resource hijacking	0	1,040	3
Defence evasion, other	0	0	0
Total	**120**	**1,672**	**22**

4.8 The Transfer Experiment (RQ3)

4.8.1 Main results

Table 4.14 and **Figure 4.**23 give the main result of the project.

Table 4.14: Transfer results (results/table3_transfer.md). For Q1, Q2 and Q4, n_train is the mean training size per fold and n_test counts pooled out-of-fold predictions. F1 is macro-F1 over classes present in the test set.

Quadrant	Train → test	n_train	n_test	Macro-F1	Accuracy	Majority F1	Majority acc.
Q1	Synthetic → synthetic	60	120	0.493	0.758	0.168	0.725
Q2	Real → real	1,337	1,672	0.983	0.992	0.001	0.001
Q3	Synthetic → real	120	1,672	0.205	0.022	0.000	0.000
Q3-dedup	Synthetic → real, one per template	120	22	0.071	0.045	0.000	0.000
Q4	Real + synthetic → real	1,457	1,672	0.981	0.992	0.001	0.001

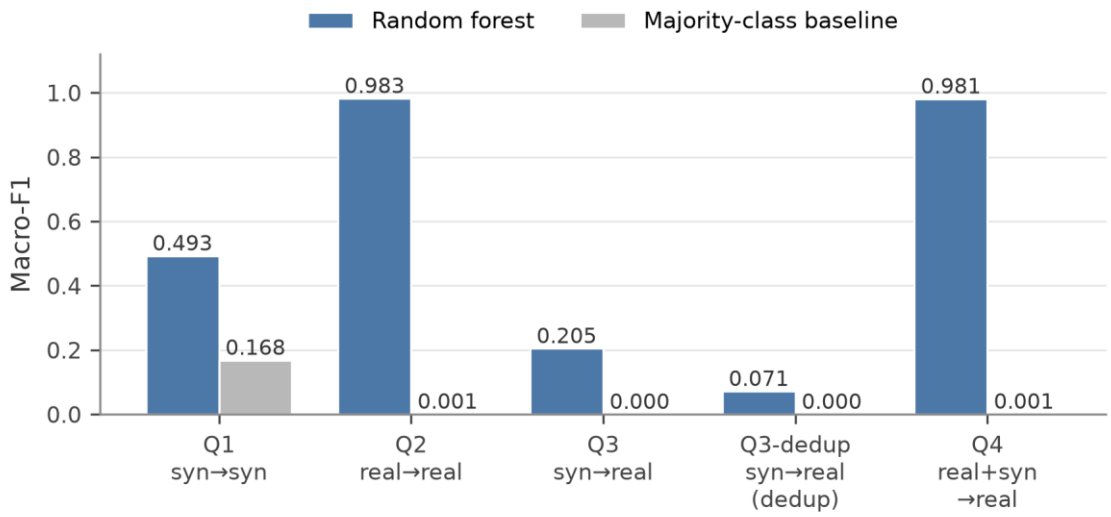


Figure 4.23: Macro-F1 of the classifier and of the majority-class baseline in each quadrant.

Q1 (synthetic → synthetic) is moderate: macro-F1 0.493 and accuracy 0.758, above its majority baseline (0.168 and 0.725). With only 60 training sessions per fold and two classes of six or fewer examples, this is roughly what one would expect.

Q2 (real → real) is easy: macro-F1 0.983 and accuracy 0.992 on templates the model never saw during training. We treat Q2 as an upper reference rather than a realistic estimate, because scripts that are close variants of each other can form different templates in different folds, and real templates with the same label often share most of their commands.

The **majority baselines** for Q2 and Q4 are close to zero (0.001) by construction. GroupKFold places the whole dota template in one test fold, whose training majority is then persistence; in the other folds the training majority is resource hijacking, which hardly occurs in their test sets.

Q3 (synthetic → real) fails. Macro-F1 is 0.205 and accuracy 0.022; with one session per template (Q3-dedup) the figures are 0.071 and 0.045. The model labels 1,636 of the 1,672 real sessions as credential access (**Figure 4.**24), a class that does not exist in the real labels. The only class it recognises is malware drop, with a per-class F1 of 0.818, and the entire macro-F1 comes from that one class (**Table 4.**15).

Q4 (real + synthetic → real) shows no improvement: 0.981 against 0.983 for Q2, with the same accuracy (0.992). Adding the synthetic sessions neither helps nor seriously harms.

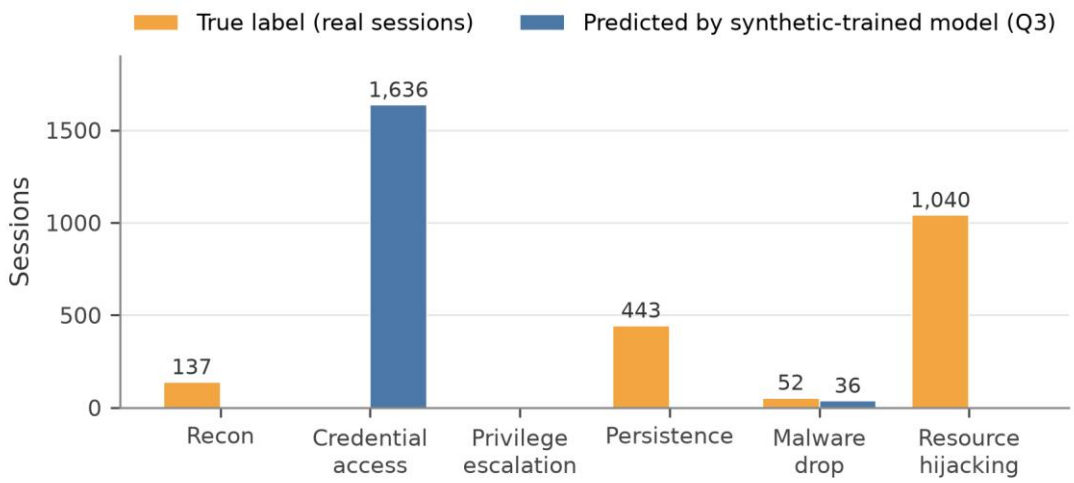


Figure 4.24: True labels of the real sessions and the labels predicted by the synthetic-trained model.

Table 4.15: Per-class F1 of the synthetic-trained model on real sessions (Q3).

Label	Real sessions with this label	F1
Recon	137	0.000
Persistence	443	0.000
Malware drop	52	0.818
Resource hijacking	1,040	0.000

Table 4.16: Number of sessions predicted in each class per quadrant (results/table3_transfer.md).

Quadrant	Recon	Credential access	Privilege esc.	Persistence	Malware drop	Resource hijacking
Q1	12	98	0	5	5	0
Q2	142	0	0	439	51	1,040
Q3	0	1,636	0	0	36	0
Q3-dedup	0	21	0	0	1	0
Q4	142	4	0	435	52	1,039

Four-quadrant transfer experiment

```
> python train_eval_classifier.py --synthetic normalized/all_synthetic_120.jsonl \
    --real normalized/real_organic.jsonl --real-labels annotation/labels_final.csv \
    --synthetic-labels annotation/labels_final.csv --label-source llm-dual
synthetic n=120, real n=1672, templates=22
| Quadrant | Train | Test | n_train | n_test | Macro-F1 | Accuracy | Majority
macro-F1 | Majority acc |
|---|---|---|---|---|---|---|---|
| Q1 | synthetic | synthetic | 60 | 120 | 0.493 | 0.758 | 0.168 | 0.725 |
| Q2 | real | real | 1337 | 1672 | 0.983 | 0.992 | 0.001 | 0.001 |
| Q3 | synthetic | real | 120 | 1672 | 0.205 | 0.022 | 0.000 | 0.000 |
| Q3_dedup | synthetic | real_dedup | 120 | 22 | 0.071 | 0.045 | 0.000 | 0.000 |
| Q4 | real+synthetic | real | 1457 | 1672 | 0.981 | 0.992 | 0.001 | 0.001 |

...
```

Figure 4.25: Console output of the transfer experiment.

4.8.2 Why transfer fails

Part of the failure is a **label-space problem**. A model trained without any resource-hijacking examples cannot predict that label, which covers most real sessions. But the failure goes beyond missing classes: *recon* and *persistence* exist in both corpora and still score an F1 of 0.000. Synthetic persistence is an intruder writing a reverse-shell cron entry or appending an SSH key; real persistence is a bot script that changes a password between fingerprinting steps. The label is the same but the behaviour is not. This is concept shift in the sense of Moreno-Torres et al. [72], on top of a strong label shift. Correcting for label shift alone [66] would not fix it.

4.9 Sensitivity and Robustness Checks

4.9.1 Does the dota label drive the result?

Because one label decides 1,038 sessions, we reran the experiment with all three resource-hijacking templates, including dota, relabelled as malware drop (**Table 4.**17, **Figure 4.**26). Q3 stayed near zero (macro-F1 0.021) and Q3-dedup at 0.067, while Q2 and Q4 reached 0.933 and 0.965. The transfer failure does not depend on this label. The small difference between Q2 and Q4 in this run (0.933 against 0.965) goes the other way from the main run, so neither run shows a consistent benefit from synthetic data.

Table 4.17: Main labels versus the dota sensitivity run (macro-F1 / accuracy).

Quadrant	Main (dual-LLM labels)	Sensitivity: dota as malware drop
Q1	0.493 / 0.758	0.493 / 0.758
Q2	0.983 / 0.992	0.933 / 0.967
Q3	0.205 / 0.022	0.021 / 0.022
Q3-dedup	0.071 / 0.045	0.067 / 0.045
Q4	0.981 / 0.992	0.965 / 0.984

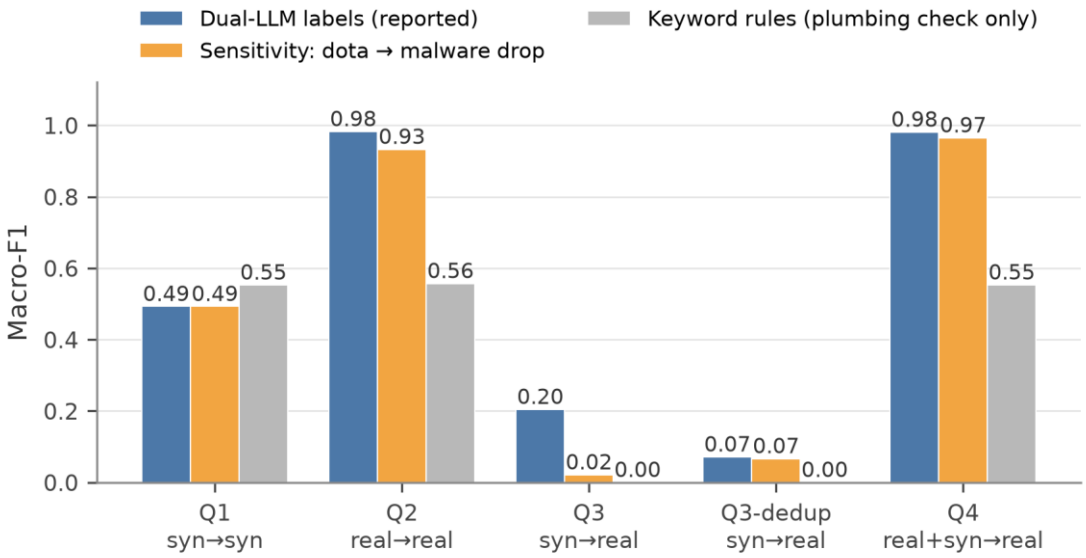


Figure 4.26: Macro-F1 per quadrant under three label sources. The keyword-rule labels are a plumbing check only and are not a result.

4.9.2 The keyword-rule plumbing check

With labels from our keyword rules (Design A), Q3 also collapsed to a macro-F1 of 0.000. We do not count this as evidence for or against transfer, because those labels share keywords with the features; it only confirms that the code path runs end to end. The rule labels also show how badly keyword rules describe real traffic: they put 1,470 of 1,672 real sessions into resource hijacking and 82 into *other*.

4.9.3 Other robustness choices

Three further choices protect the main result: the C2ST was repeated with verbs only (Section 4.6.4); the transfer test was repeated on one session per template (Q3-dedup); and all cross-validation of real data groups sessions by template. Each of these guards against a known way of inflating results [4, 58].

4.10 Answers to the Research Questions

Table 4.18: Research questions, hypotheses and outcomes.

ID	Outcome	Evidence
RQ1	The three attackers behaved very differently under the same prompt (session length, variety, interest in secrets, how sessions ended)	**Table 4.**5, **Table 4.**6; **Figure 4.**9 to **Figure 4.**14
RQ2	Synthetic and real sessions are almost perfectly separable; the gap is about intent	JSD 0.700; C2ST AUC 1.000 (verbs only 0.996); tactic profiles
RQ3	No transfer and no augmentation benefit	Q3 macro-F1 0.205 (dedup 0.071) vs Q2 0.983; Q4 0.981
RQ4	The LLM backend helped only the smallest attacker and raised credential hunting for all	Gemma JSD 0.798 → 0.746; others unchanged within CI
H1	Supported	JSD ≫ 0.5; AUC ≈ 1
H2	Supported	Q3 ≪ Q2, also after the dota sensitivity run
H3	Not supported	Only one of three attackers moved closer to real traffic

4.11 Discussion

4.11.1 The gap is about intent

The agents did not fail at their task; they did a different task. They were asked to act like a curious intruder, and they listed directories, read configuration files and searched for keys, which is exactly what a capture-the-flag player or a penetration tester does [48, 93]. The real traffic on our sample day was driven by bots that profiled the CPU, took over the account and launched a scanner and miner kit [108]. Nobody asked the agents to behave like that population, and they did not. The finding of Barron and Nikiforakis [8], that bots behave the same regardless of environment, helps explain why the LLM backend did not close the gap: making the environment richer changes how a curious agent explores, not whether it behaves like a bot.

4.11.2 What the LLM-backed honeypot changed

Qwen3-32B reduced the divergence only for the smallest attacker, Gemma-3-4B. Richer shell output may give a weak attacker more to react to, but we did not test this. Even the closest cell (JSD 0.741) remains far from real traffic, and the backend increased credential hunting for every attacker, which pushes the synthetic data further towards the one label (credential access) that real traffic never has. LLM shells are known to invent inconsistent state [56, 96], so an agent attacking one is partly reacting to the other model's imagination.

4.11.3 What this means for practitioners

Three practical messages follow. First, do not assume that agent sessions are realistic training data; measure the gap with label-free tests before using them. Second, if synthetic data is used, design the personas to match the population you expect to defend against (droppers, miners, credential stuffers), not a generic explorer. Third, filter chat-format leakage and record raw replies, or the data will contain the model's thoughts instead of commands.

4.12 Threats to Validity

Table 4.19: Threats to validity and how they were handled.

Threat	Description	Mitigation and remaining risk
Label quality	Labels were first produced by LLMs from one provider, which may share biases	Two different models, blind and independent; adjudication; author cross-verification; sensitivity run. Remaining: no independent human audit yet
Imperfect blindness	LLM-generated commands have a recognisable style, so an annotator could guess the source	Cannot be measured; our own verification was not blind either
Small synthetic sample	20 sessions per cell	Bootstrap CIs; no claim of significant ordering between cells
One day, one honeynet	Real data from 15 November 2019 only; one campaign is 62% of interactive sessions	Per-template reporting; de-duplicated tests; more days in future work
Tagger errors	Tactic regexes and keyword shares were written by us and can mislabel (for example any path containing `passwd`)	Only large differences are interpreted; tagger never used as ground truth
Confounded persona	One prompt for all sessions; persona and model effects are not separated	Conclusions limited to this persona
Glued commands	Some replies concatenated commands without separators	Cannot be repaired by leak stripping; small effect
One classifier family	C2ST and transfer use simple linear and tree models	Perfect separability shows the corpora are separable, not that no other representation could bridge them
Normaliser quirk	URL mask can swallow an adjacent `;`	Affects segment splitting of a few commands in both corpora equally

4.13 Testing, Characterisation and Data Validation

We validated the system at every stage (**Figure 4.**27). **Table 4.**20 lists the main checks, how they were run and what they showed.

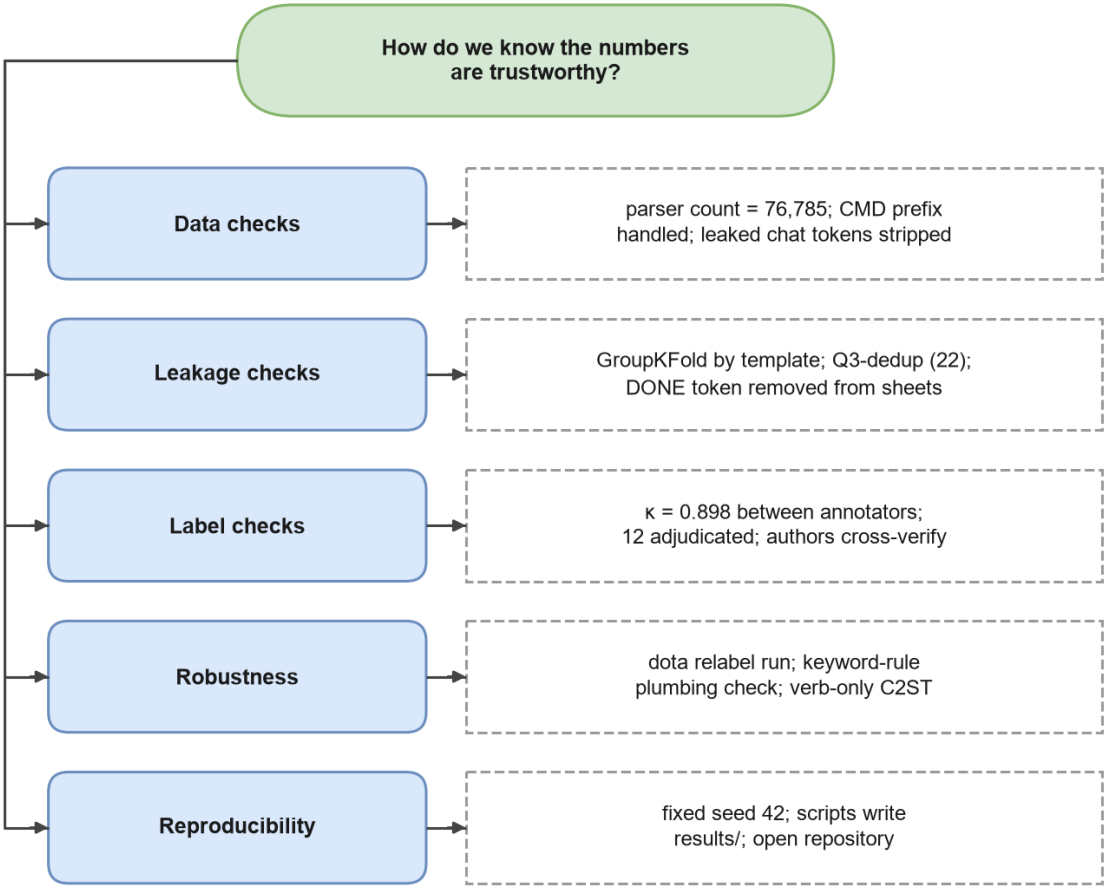


Figure 4.27: Validation checks used throughout the project.

Table 4.20: Testing and validation checks.

Check	How	Result
Honeypot reachable	`check.py` and `probe_banner()` read the SSH banner	Banner `SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5` returned
Model endpoints	`probe.py` sent a test prompt to each candidate model	Working models selected for the study
Reply boundaries	`validate_logs.py` scans logs for leaked markers and repeated prompts	Used to reject an earlier marker-based design
Recovery	Frozen-honeypot events logged in `infra_events.log`	Recovery attempts recorded with timestamps (for example during the DeepSeek static batch)
Parser correctness	Session count and field checks after parsing	76,785 sessions; 1,672 interactive; matches the dataset's structure
Normaliser	Worked examples run through the real code (**Figure 3.**10)	Masks and splits behave as designed; one URL quirk documented
Leakage	GroupKFold by template; Q3-dedup	No template in both train and test
Label reliability	Cohen's κ; disagreement file; adjudication	$\kappa = 0.898$; 12 adjudicated
Sensitivity	dota relabel run; rule-label plumbing run	Conclusions unchanged
Reproducibility	Fixed seed 42; scripts regenerate `results/`	Same numbers on rerun
Report numbers	Every number traced to `results/` or `report/data/`	Checked while writing

4.13.1 Software testing strategy

Testing followed the usual four levels (**Table 4.**21). Because this is research software, the most important tests are not about whether a function runs but about whether a number can be trusted, so several tests compare an output with an independent count or a hand-worked example.

Table 4.21: Testing levels.

Level	What was tested	How
Unit	Normalisation, segment splitting, verbs, tactic tags, macro-F1 over present classes	Worked examples run through the real functions (**Figure 3.**10)
Integration	Raw file → parser → normalised sessions → analysis scripts	Session and template counts compared across stages
System	The full pipeline from logs and raw data to `results/`	Every script run end to end; outputs checked against each other
Acceptance	Every number in the paper and this report	Traced to a file in `results/` or `report/data/`; supervisor review

4.13.2 Test cases

Table 4.22: Test cases with expected and observed results.

ID	Test	Expected	Observed	Result
TC-01	Parse the CyberLab file	All sessions read	76,785 sessions; 1,672 with commands	Pass
TC-02	Run the native Cowrie parser on the CyberLab file	Shows the format mismatch	Zero sessions, no error	Pass (pitfall confirmed)
TC-03	Normalise `echo "root:…"|chpasswd|bash`	Password masked	`echo <str>|chpasswd|bash`	Pass
TC-04	Normalise `/bin/busybox QWERT`	Random tag masked	`/bin/busybox <tag>`	Pass
TC-05	Split `cat /proc/cpuinfo | grep name | wc -l`	Three segments	Verbs `cat`, `grep`, `wc`	Pass
TC-06	Detect chat-format leakage	Leaks found only for GPT-OSS-120B	11 static + 7 dynamic sessions	Pass
TC-07	Group real sessions into templates	Far fewer templates than sessions	22 templates	Pass
TC-08	Grouped folds keep templates apart	No template in train and test	Holds by construction (GroupKFold)	Pass
TC-09	Agreement and final labels	κ computed on all items; every session labelled	$\kappa = 0.898$ on 184 items; 1,672 + 120 labelled	Pass
TC-10	Honeypot freeze is noticed	Event written to the infrastructure log	Recovery attempts logged on 27 September 2026	Pass
TC-11	Report numbers match results	No number without a source file	Checked while writing	Pass
TC-12	Report build	All references cited and linked; no broken fields	124 of 124 references linked; no field errors	Pass

4.14 Report Preparation, Project Management and Communication

The project was managed as a small research engineering effort. Work was split into the seven work packages of Chapter 1 and tracked in a shared task list (`PENDING.md`) with a done/next/future structure. The repository was the single source of truth: data generation, analysis and paper tables all ran from scripts, and generated files were never edited by hand. Results were written to `results/` as both Markdown (for reading) and JSON (for other scripts), which made it easy to check every number in the paper and in this report.

Communication followed three rules: agree on definitions before measuring (for example, the rubric was written before labelling began); report negative results as clearly as positive ones; and keep a written record of every design change and its reason, such as the move

from marker-based to idle-timeout reading. The paper was first written in IEEE format and then converted, without changing its text, to Springer's LNCS/LNNS format for submission to the International Conference on Intelligent Communication, Control and Devices (ICICCD 2026). This report and the project documentation were written from Markdown sources and built into Word documents by script, so that figures, tables and numbers stay consistent across all outputs.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

We set out to answer a practical question: can LLM red-team agents generate honeypot sessions that are good enough to train a defender's intent classifier? Under the conditions we tested, a single "explore the system" persona, three attacker models and two honeypot backends, the answer is no.

The evidence points the same way from every angle. Without any labels, the synthetic and real corpora are almost perfectly separable: their command-verb distributions have a Jensen-Shannon divergence of 0.700, and a simple classifier tells them apart with an AUC of 1.000, still 0.996 when it sees only verbs. With blind dual-LLM labels that agree well (Cohen's $\kappa = 0.898$), a classifier trained on synthetic sessions reaches a macro-F1 of only 0.205 on real sessions (0.071 when each real behaviour is counted once), against 0.983 for a classifier trained on real data. Adding the synthetic sessions to real training data gives no gain (0.981). Changing the most influential label does not change this picture.

The reason is intent, not vocabulary. The agents know the right commands (their vocabulary covers 82% of real verb occurrences), but they use them to explore files and hunt for credentials, as a penetration tester would. The real traffic on our sample day came mostly from bots that profiled the CPU, took over the account and ran a cryptominer kit: one such script alone produced 1,038 of 1,672 interactive sessions. The LLM-backed honeypot made only the smallest attacker look a little more realistic and made all attackers hunt more for credentials. Along the way we found that one attacker model leaked its chat-format control tokens and private reasoning into the shell in 18 of 40 sessions, a contamination problem that any similar pipeline has to filter.

These results do not mean that LLM agents are useless for honeypot research. They mean that *what the agent is asked to be* matters more than *which agent* it is or *how clever the honeypot* is, and that synthetic security data must be checked against real data before it is trusted. The label-free tests used here are cheap and need no annotation, so they can be run before anyone spends money on labelling synthetic data.

5.2 Deviation from Expected Results

Several outcomes differed from what we expected when the project began. **Table 5.**1 lists them with our explanation.

Table 5.1: Expected versus actual results.

Expectation at the start	What actually happened	Explanation
Agent sessions would be "somewhat" realistic, with moderate JSD	JSD 0.700 and AUC 1.000: almost no overlap	The agents pursued a different goal from the dominant real population
An LLM-backed honeypot would make every agent more realistic	Only Gemma-3-4B moved closer; credential hunting rose for all	A richer environment changes how a curious agent explores, not whether it behaves like a bot
Synthetic data would at least help when mixed with real data	Q4 (0.981) ≈ Q2 (0.983); no consistent gain	Synthetic labels and behaviours barely overlap with real ones
Larger models would look more realistic than small ones	No clear size ordering; the 4B model was the one that moved most	Persona dominates model size
Refusals would be a major problem	One refusal in 120 sessions	The research framing in the prompt worked; leakage was the bigger problem
A standard Cowrie parser would read the public data	It read zero sessions without any error	The CyberLab export uses a different structure
Keyword rules would be good enough as labels	They mislabelled bot behaviour and were circular	Led to the switch to blind dual-LLM labels

5.3 Lessons Learned

1. **Measure before you label.** Label-free tests (JSD, C2ST) showed the core result before a single label existed, at almost no cost.
2. **Group by behaviour, not by session.** With 1,672 sessions but only 22 templates, random splits would have given meaningless scores.
3. **Silent failures are the dangerous ones.** The parser that read zero sessions and the model that leaked its reasoning both produced plausible-looking output.
4. **Keep failed sessions.** Connection losses and LLM errors told us as much about the setup as the completed sessions.
5. **Write the rubric first.** Agreeing on definitions before labelling made the disagreements few and easy to discuss.

5.4 Future Work

The limitations in Section 4.12 point directly to the next steps, summarised in **Table 5.**2.

Table 5.2: Future work.

Direction	What to do	Expected benefit
Bot-like personas	Prompt agents as a dropper, a cryptominer, a credential stuffer and a Mirai-style scanner, then rerun the transfer matrix	Tests whether persona design can close the intent gap
Persona randomisation	Vary goals, skill and style across sessions, in the spirit of domain randomisation [106]	Wider coverage of real behaviour
More real data	More days of CyberLab and other honeynets, including HoneyCloud-style IoT data [22]	Less dependence on one day and one campaign
Independent human audit	Two security analysts outside the author team label the same 184 items	Measures human–human and human–LLM agreement (Design C)
Better LLM honeypots	Fine-tuned or state-tracking shells [79] evaluated with a unified protocol [113]	Separates backend quality from attacker behaviour
Stronger agents and benchmarks	Agents from CTF benchmarks [93, 122] and exploit-capable agents [33, 49]	Tests whether capability changes footprints
Generative baselines	Compare agent data with GAN-style generators where enough real data exists [65, 90]	Puts agent data in context with other synthetic methods
Sequence models	Classifiers that use command order, not just bags of tokens	May capture behaviour beyond vocabulary
Contamination checks as standard	Report chat-token leak rates and C2ST AUC with every synthetic honeypot dataset	Makes synthetic data quality visible
Datasheets	Publish a datasheet for the synthetic corpus [39]	Responsible reuse

5.5 Final Remarks

The idea that AI agents can manufacture realistic attack data is attractive, and it may yet work. This project shows that it does not work automatically. An agent asked to "explore the system" behaves like an explorer, and the internet's honeypots are visited mostly by bots that do not explore at all. Training data has to come from, or be checked against, the population we actually want to defend against. We hope that the open pipeline, the label-free tests and the documentation in this report make that check easy for the next team that tries.

REFERENCES

1. ACM (2018) *ACM Code of Ethics and Professional Conduct*. New York: Association for Computing Machinery. Available at: https://www.acm.org/code-of-ethics.

2. Anderson, L.W. and Krathwohl, D.R. (eds.) (2001) *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York: Longman.

3. Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., Durumeric, Z., Halderman, J.A., Invernizzi, L., Kallitsis, M., Kumar, D., Lever, C., Ma, Z., Mason, J., Menscher, D., Seaman, C., Sullivan, N., Thomas, K. and Zhou, Y. (2017) 'Understanding the Mirai Botnet', *Proc. 26th USENIX Security Symposium*, Vancouver, pp.1093–1110.

4. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L. and Rieck, K. (2022) 'Dos and Don'ts of Machine Learning in Computer Security', *Proc. 31st USENIX Security Symposium*, Boston, pp.3971–3988.

5. Artstein, R. and Poesio, M. (2008) 'Inter-Coder Agreement for Computational Linguistics', *Computational Linguistics*, Vol.34, No.4, pp.555–596.

6. Bai, Y., Kadavath, S., Kundu, S., Askell, A. et al. (2022) 'Constitutional AI: Harmlessness from AI Feedback', *arXiv preprint* arXiv:2212.08073.

7. Bailey, M., Cooke, E., Jahanian, F., Nazario, J. and Watson, D. (2005) 'The Internet Motion Sensor: A Distributed Blackhole Monitoring System', *Proc. Network and Distributed System Security Symposium (NDSS 2005)*, San Diego.

8. Barron, T. and Nikiforakis, N. (2017) 'Picky Attackers: Quantifying the Role of System Properties on Intruder Behavior', *Proc. 33rd Annual Computer Security Applications Conference (ACSAC 2017)*, San Juan. doi:10.1145/3134600.3134614.

9. Ben-David, S., Blitzer, J., Crammer, K., Kulesza, A., Pereira, F. and Vaughan, J.W. (2010) 'A Theory of Learning from Different Domains', *Machine Learning*, Vol.79, No.1–2, pp.151–175.

10. Bilge, L. and Dumitraş, T. (2012) 'Before We Knew It: An Empirical Study of Zero-Day Attacks in the Real World', *Proc. 2012 ACM Conference on Computer and Communications Security (CCS)*, Raleigh, pp.833–844.

11. Bloom, B.S. (ed.) (1956) *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain*. New York: David McKay.

12. Bommasani, R., Hudson, D.A., Adeli, E. et al. (2021) 'On the Opportunities and Risks of Foundation Models', *arXiv preprint* arXiv:2108.07258.

13. Breiman, L. (2001) 'Random Forests', *Machine Learning*, Vol.45, No.1, pp.5–32.

14. Bringer, M.L., Chelmecki, C.A. and Fujinoki, H. (2012) 'A Survey: Recent Advances and Future Trends in Honeypot Research', *International Journal of Computer Network and Information Security*, Vol.4, No.10, pp.63–75.

15. Brown, T.B., Mann, B., Ryder, N., Subbiah, M. et al. (2020) 'Language Models are Few-Shot Learners', *Advances in Neural Information Processing Systems*, Vol.33, pp.1877–1901.

16. Buczak, A.L. and Guven, E. (2016) 'A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection', *IEEE Communications Surveys & Tutorials*, Vol.18, No.2, pp.1153–1176.

17. Carlini, N., Tramèr, F., Wallace, E., Jagielski, M. et al. (2021) 'Extracting Training Data from Large Language Models', *Proc. 30th USENIX Security Symposium*, pp.2633–2650.

18. CERT-In (2022) *Directions under sub-section (6) of section 70B of the Information Technology Act, 2000 relating to information security practices, procedure, prevention, response and reporting of cyber incidents*. New Delhi: Indian Computer Emergency Response Team, 28 April 2022.

19. Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic Minority Over-sampling Technique', *Journal of Artificial Intelligence Research*, Vol.16, pp.321–357.

20. Cheswick, B. (1992) 'An Evening with Berferd, in which a Cracker is Lured, Endured, and Studied', *Proc. Winter USENIX Conference*, San Francisco.

21. Cohen, J. (1960) 'A Coefficient of Agreement for Nominal Scales', *Educational and Psychological Measurement*, Vol.20, No.1, pp.37–46.

22. Dang, F., Li, Z., Liu, Y., Zhai, E., Chen, Q.A., Xu, T., Chen, Y. and Yang, J. (2019) 'Understanding Fileless Attacks on Linux-based IoT Devices with HoneyCloud', *Proc. 17th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys 2019)*, pp.482–493.

23. DeepSeek-AI (2025) 'DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models', *arXiv preprint* arXiv:2512.02556.

24. Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M. and Rass, S. (2024) 'PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing', *Proc. 33rd USENIX Security Symposium*, Philadelphia, pp.847–864.

25. Deutsche Telekom Security (n.d.) *T-Pot: The All In One Multi Honeypot Platform*. GitHub repository. Available at: https://github.com/telekom-security/tpotce (Accessed: October 2026).

26. Dittrich, D. and Kenneally, E. (2012) *The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research*. Washington, DC: U.S. Department of Homeland Security.

27. Durumeric, Z., Wustrow, E. and Halderman, J.A. (2013) 'ZMap: Fast Internet-wide Scanning and Its Security Applications', *Proc. 22nd USENIX Security Symposium*, Washington, DC, pp.605–620.

28. Efron, B. and Tibshirani, R.J. (1993) *An Introduction to the Bootstrap*. New York: Chapman & Hall.

29. Endres, D.M. and Schindelin, J.E. (2003) 'A New Metric for Probability Distributions', *IEEE Transactions on Information Theory*, Vol.49, No.7, pp.1858–1860.

30. Engelen, G., Rimmer, V. and Joosen, W. (2021) 'Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study', *Proc. 2021 IEEE Security and Privacy Workshops (SPW)*, pp.7–12.

31. European Union (2024) *Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, 12 July 2024.

32. Fan, W., Du, Z., Fernández, D. and Villagrá, V.A. (2018) 'Enabling an Anatomic View to Investigate Honeypot Systems: A Survey', *IEEE Systems Journal*, Vol.12, No.4, pp.3906–3919.

33. Fang, R., Bindu, R., Gupta, A. and Kang, D. (2024a) 'LLM Agents can Autonomously Exploit One-day Vulnerabilities', *arXiv preprint* arXiv:2404.08144.

34. Fang, R., Bindu, R., Gupta, A., Zhan, Q. and Kang, D. (2024b) 'LLM Agents can Autonomously Hack Websites', *arXiv preprint* arXiv:2402.06664.

35. Fawcett, T. (2006) 'An Introduction to ROC Analysis', *Pattern Recognition Letters*, Vol.27, No.8, pp.861–874.

36. Fleiss, J.L. (1971) 'Measuring Nominal Scale Agreement among Many Raters', *Psychological Bulletin*, Vol.76, No.5, pp.378–382.

37. Franco, J., Aris, A., Canberk, B. and Uluagac, A.S. (2021) 'A Survey of Honeypots and Honeynets for Internet of Things, Industrial Internet of Things, and Cyber-Physical Systems', *IEEE Communications Surveys & Tutorials*, Vol.23, No.4, pp.2351–2383.

38. Ganguli, D., Lovitt, L., Kernion, J., Askell, A. et al. (2022) 'Red Teaming Language Models to Reduce Harms: Methods, Scaling Behaviors, and Lessons Learned', *arXiv preprint* arXiv:2209.07858.

39. Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J.W., Wallach, H., Daumé III, H. and Crawford, K. (2021) 'Datasheets for Datasets', *Communications of the ACM*, Vol.64, No.12, pp.86–92.

40. Gemma Team (2025) 'Gemma 3 Technical Report', *arXiv preprint* arXiv:2503.19786.

41. Gilardi, F., Alizadeh, M. and Kubli, M. (2023) 'ChatGPT Outperforms Crowd Workers for Text-Annotation Tasks', *Proceedings of the National Academy of Sciences*, Vol.120, No.30, e2305016120.

42. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014) 'Generative Adversarial Nets', *Advances in Neural Information Processing Systems*, Vol.27, pp.2672–2680.

43. Government of India (2000) *The Information Technology Act, 2000* (Act No. 21 of 2000). New Delhi: Ministry of Law, Justice and Company Affairs.

44. Government of India (2023) *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023). New Delhi: Ministry of Law and Justice.

45. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T. and Fritz, M. (2023) 'Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection', *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, pp.79–90.

46. Gretton, A., Borgwardt, K.M., Rasch, M.J., Schölkopf, B. and Smola, A. (2012) 'A Kernel Two-Sample Test', *Journal of Machine Learning Research*, Vol.13, pp.723–773.

47. Hanley, J.A. and McNeil, B.J. (1982) 'The Meaning and Use of the Area under a Receiver Operating Characteristic (ROC) Curve', *Radiology*, Vol.143, No.1, pp.29–36.

48. Happe, A. and Cito, J. (2023) 'Getting pwn'd by AI: Penetration Testing with Large Language Models', *Proc. 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2023)*, San Francisco, pp.2082–2086.

49. Happe, A., Kaplan, A. and Cito, J. (2023) 'LLMs as Hackers: Autonomous Linux Privilege Escalation Attacks', *arXiv preprint* arXiv:2310.11409.

50. Harris, C.R., Millman, K.J., van der Walt, S.J. et al. (2020) 'Array Programming with NumPy', *Nature*, Vol.585, pp.357–362.

51. He, H. and Garcia, E.A. (2009) 'Learning from Imbalanced Data', *IEEE Transactions on Knowledge and Data Engineering*, Vol.21, No.9, pp.1263–1284.

52. Herwig, S., Harvey, K., Hughey, G., Roberts, R. and Levin, D. (2019) 'Measurement and Analysis of Hajime, a Peer-to-peer IoT Botnet', *Proc. Network and Distributed System Security Symposium (NDSS 2019)*, San Diego.

53. Hunter, J.D. (2007) 'Matplotlib: A 2D Graphics Environment', *Computing in Science & Engineering*, Vol.9, No.3, pp.90–95.

54. Hutchins, E.M., Cloppert, M.J. and Amin, R.M. (2011) 'Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains', *Leading Issues in Information Warfare & Security Research*, Vol.1, No.1, pp.80–106.

55. Jaccard, P. (1912) 'The Distribution of the Flora in the Alpine Zone', *New Phytologist*, Vol.11, No.2, pp.37–50.

56. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A. and Fung, P. (2023) 'Survey of Hallucination in Natural Language Generation', *ACM Computing Surveys*, Vol.55, No.12, Article 248.

57. Jordaney, R., Sharad, K., Dash, S.K., Wang, Z., Papini, D., Nouretdinov, I. and Cavallaro, L. (2017) 'Transcend: Detecting Concept Drift in Malware Classification Models', *Proc. 26th USENIX Security Symposium*, Vancouver, pp.625–642.

58. Kapoor, S. and Narayanan, A. (2023) 'Leakage and the Reproducibility Crisis in Machine-Learning-Based Science', *Patterns*, Vol.4, No.9, 100804.

59. Kohavi, R. (1995) 'A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection', *Proc. 14th International Joint Conference on Artificial Intelligence (IJCAI)*, Montreal, pp.1137–1143.

60. Kolias, C., Kambourakis, G., Stavrou, A. and Voas, J. (2017) 'DDoS in the IoT: Mirai and Other Botnets', *Computer*, Vol.50, No.7, pp.80–84.

61. Kreibich, C. and Crowcroft, J. (2004) 'Honeycomb: Creating Intrusion Detection Signatures Using Honeypots', *ACM SIGCOMM Computer Communication Review*, Vol.34, No.1, pp.51–56.

62. Kullback, S. and Leibler, R.A. (1951) 'On Information and Sufficiency', *The Annals of Mathematical Statistics*, Vol.22, No.1, pp.79–86.

63. Landis, J.R. and Koch, G.G. (1977) 'The Measurement of Observer Agreement for Categorical Data', *Biometrics*, Vol.33, No.1, pp.159–174.

64. Lin, J. (1991) 'Divergence Measures Based on the Shannon Entropy', *IEEE Transactions on Information Theory*, Vol.37, No.1, pp.145–151.

65. Lin, Z., Shi, Y. and Xue, Z. (2022) 'IDSGAN: Generative Adversarial Networks for Attack Generation against Intrusion Detection', *Advances in Knowledge Discovery and Data Mining (PAKDD 2022)*, Lecture Notes in Computer Science. Cham: Springer. doi:10.1007/978-3-031-05981-0_7.

66. Lipton, Z.C., Wang, Y.-X. and Smola, A.J. (2018) 'Detecting and Correcting for Label Shift with Black Box Predictors', *Proc. 35th International Conference on Machine Learning (ICML)*, PMLR 80, pp.3122–3130.

67. Lopez-Paz, D. and Oquab, M. (2017) 'Revisiting Classifier Two-Sample Tests', *Proc. 5th International Conference on Learning Representations (ICLR 2017)*, Toulon.

68. McHugh, J. (2000) 'Testing Intrusion Detection Systems: A Critique of the 1998 and 1999 DARPA Intrusion Detection System Evaluations as Performed by Lincoln Laboratory', *ACM Transactions on Information and System Security*, Vol.3, No.4, pp.262–294.

69. McKee, F. and Noever, D. (2023) 'Chatbots in a Honeypot World', *arXiv preprint* arXiv:2301.03771.

70. Merkel, D. (2014) 'Docker: Lightweight Linux Containers for Consistent Development and Deployment', *Linux Journal*, Vol.2014, No.239, Article 2.

71. Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I.D. and Gebru, T. (2019) 'Model Cards for Model Reporting', *Proc. Conference on Fairness, Accountability, and Transparency (FAT* '19)*, Atlanta, pp.220–229.

72. Moreno-Torres, J.G., Raeder, T., Alaiz-Rodríguez, R., Chawla, N.V. and Herrera, F. (2012) 'A Unifying View on Dataset Shift in Classification', *Pattern Recognition*, Vol.45, No.1, pp.521–530.

73. Moustafa, N. and Slay, J. (2015) 'UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems', *Proc. Military Communications and Information Systems Conference (MilCIS 2015)*, Canberra, pp.1–6.

74. Nawrocki, M., Wählisch, M., Schmidt, T.C., Keil, C. and Schönfelder, J. (2016) 'A Survey on Honeypot Software and Data Analysis', *arXiv preprint* arXiv:1608.06249.

75. NIST (2023) *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1. Gaithersburg, MD: National Institute of Standards and Technology.

76. Okabe, M. and Ito, K. (2008) *Color Universal Design (CUD): How to Make Figures and Presentations That Are Friendly to Colorblind People*. Available at: https://jfly.uni-koeln.de/color/ (Accessed: October 2026).

77. OpenAI (2025) 'gpt-oss-120b & gpt-oss-20b Model Card', *arXiv preprint* arXiv:2508.10925.

78. Oosterhof, M. (2015) *Cowrie SSH/Telnet Honeypot*. GitHub repository. Available at: https://github.com/cowrie/cowrie (Accessed: October 2026).

79. Otal, H.T. and Canbaz, M.A. (2024) 'LLM Honeypot: Leveraging Large Language Models as Advanced Interactive Honeypot Systems', *Proc. 2024 IEEE Conference on Communications and Network Security (CNS)*. arXiv:2409.08234.

80. Ouyang, L., Wu, J., Jiang, X., Almeida, D. et al. (2022) 'Training Language Models to Follow Instructions with Human Feedback', *Advances in Neural Information Processing Systems*, Vol.35, pp.27730–27744.

81. Pa, Y.M.P., Suzuki, S., Yoshioka, K., Matsumoto, T., Kasama, T. and Rossow, C. (2015) 'IoTPOT: Analysing the Rise of IoT Compromises', *Proc. 9th USENIX Workshop on Offensive Technologies (WOOT 15)*, Washington, DC.

82. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V. et al. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, Vol.12, pp.2825–2830.

83. Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J. and Cavallaro, L. (2019) 'TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time', *Proc. 28th USENIX Security Symposium*, Santa Clara, pp.729–746.

84. Peng, R.D. (2011) 'Reproducible Research in Computational Science', *Science*, Vol.334, No.6060, pp.1226–1227.

85. Perez, E., Huang, S., Song, F., Cai, T., Ring, R., Aslanides, J., Glaese, A., McAleese, N. and Irving, G. (2022) 'Red Teaming Language Models with Language Models', *Proc. 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Abu Dhabi, pp.3419–3448.

86. Provos, N. (2004) 'A Virtual Honeypot Framework', *Proc. 13th USENIX Security Symposium*, San Diego, pp.1–14.

87. Provos, N. and Holz, T. (2007) *Virtual Honeypots: From Botnet Tracking to Intrusion Detection*. Upper Saddle River, NJ: Addison-Wesley.

88. Quiñonero-Candela, J., Sugiyama, M., Schwaighofer, A. and Lawrence, N.D. (eds.) (2009) *Dataset Shift in Machine Learning*. Cambridge, MA: MIT Press.

89. Rabanser, S., Günnemann, S. and Lipton, Z.C. (2019) 'Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift', *Advances in Neural Information Processing Systems*, Vol.32.

90. Ring, M., Schlör, D., Landes, D. and Hotho, A. (2019) 'Flow-based Network Traffic Generation Using Generative Adversarial Networks', *Computers & Security*, Vol.82, pp.156–172.

91. Salton, G. and Buckley, C. (1988) 'Term-Weighting Approaches in Automatic Text Retrieval', *Information Processing & Management*, Vol.24, No.5, pp.513–523.

92. Sedlar, U., Kren, M., Štefanič Južnič, L. and Volk, M. (2020) *CyberLab Honeynet Dataset*, Version 1.0. Zenodo. doi:10.5281/zenodo.3687527.

93. Shao, M., Jancheska, S., Udeshi, M., Dolan-Gavitt, B., Xi, H., Milner, K., Chen, B., Yin, M., Garg, S., Krishnamurthy, P., Khorrami, F., Karri, R. and Shafique, M. (2024) 'NYU CTF Bench: A Scalable Open-Source Benchmark Dataset for Evaluating LLMs in Offensive Security', *Advances in Neural Information Processing Systems 37 (Datasets and Benchmarks Track)*.

94. Sharafaldin, I., Lashkari, A.H. and Ghorbani, A.A. (2018) 'Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization', *Proc. 4th International Conference on Information Systems Security and Privacy (ICISSP 2018)*, Funchal, pp.108–116.

95. Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R. and Gal, Y. (2024) 'AI Models Collapse when Trained on Recursively Generated Data', *Nature*, Vol.631, pp.755–759.

96. Sladić, M., Valeros, V., Catania, C. and Garcia, S. (2023) 'LLM in the Shell: Generative Honeypots', *arXiv preprint* arXiv:2309.00155.

97. Sokolova, M. and Lapalme, G. (2009) 'A Systematic Analysis of Performance Measures for Classification Tasks', *Information Processing & Management*, Vol.45, No.4, pp.427–437.

98. Sommer, R. and Paxson, V. (2010) 'Outside the Closed World: On Using Machine Learning for Network Intrusion Detection', *Proc. 2010 IEEE Symposium on Security and Privacy*, Oakland, pp.305–316.

99. Spärck Jones, K. (1972) 'A Statistical Interpretation of Term Specificity and Its Application in Retrieval', *Journal of Documentation*, Vol.28, No.1, pp.11–21.

100. Spitzner, L. (2003a) *Honeypots: Tracking Hackers*. Boston: Addison-Wesley.

101. Spitzner, L. (2003b) 'The Honeynet Project: Trapping the Hackers', *IEEE Security & Privacy*, Vol.1, No.2, pp.15–23.

102. Stoll, C. (1989) *The Cuckoo's Egg: Tracking a Spy Through the Maze of Computer Espionage*. New York: Doubleday.

103. Strom, B.E., Applebaum, A., Miller, D.P., Nickels, K.C., Pennington, A.G. and Thomas, C.B. (2018) *MITRE ATT&CK: Design and Philosophy*, Technical Report MP180360R1. McLean, VA: The MITRE Corporation.

104. Tamminen, U. (2009) *Kippo – SSH Honeypot*. Software repository. Available at: https://github.com/desaster/kippo (Accessed: October 2026).

105. Tavallaee, M., Bagheri, E., Lu, W. and Ghorbani, A.A. (2009) 'A Detailed Analysis of the KDD CUP 99 Data Set', *Proc. 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, Ottawa, pp.1–6.

106. Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W. and Abbeel, P. (2017) 'Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World', *Proc. 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vancouver, pp.23–30.

107. Törnberg, P. (2023) 'ChatGPT-4 Outperforms Experts and Crowd Workers in Annotating Political Twitter Messages with Zero-Shot Learning', *arXiv preprint* arXiv:2304.06588.

108. Trend Micro (2018) *Outlaw Group Distributes Botnet for Cryptocurrency-Mining, Scanning, and Brute-Force*. Trend Micro Research blog, 19 November 2018.

109. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I. (2017) 'Attention Is All You Need', *Advances in Neural Information Processing Systems*, Vol.30, pp.5998–6008.

110. Vetterl, A. and Clayton, R. (2018) 'Bitter Harvest: Systematically Fingerprinting Low- and Medium-interaction Honeypots at Internet Scale', *Proc. 12th USENIX Workshop on Offensive Technologies (WOOT 18)*, Baltimore.

111. Virtanen, P., Gommers, R., Oliphant, T.E. et al. (2020) 'SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python', *Nature Methods*, Vol.17, pp.261–272.

112. Wang, Z., You, J., Wang, H., Yuan, T., Lv, S., Wang, Y. and Sun, L. (2024) 'HoneyGPT: Breaking the Trilemma in Terminal Honeypots with Large Language Model', *arXiv preprint* arXiv:2406.01882.

113. Weber, S.B., Feger, M. and Pilgermann, M. (2024) 'Don't Stop Believin': A Unified Evaluation Approach for LLM Honeypots', *IEEE Access*, Vol.12, pp.144579–144587.

114. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. and Zhou, D. (2022) 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models', *Advances in Neural Information Processing Systems*, Vol.35, pp.24824–24837.

115. Weidinger, L., Mellor, J., Rauh, M., Griffin, C. et al. (2021) 'Ethical and Social Risks of Harm from Language Models', *arXiv preprint* arXiv:2112.04359.

116. Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J. et al. (2016) 'The FAIR Guiding Principles for Scientific Data Management and Stewardship', *Scientific Data*, Vol.3, 160018.

117. Xu, H., Wang, S. et al. (2024) 'Large Language Models for Cyber Security: A Systematic Literature Review', *arXiv preprint* arXiv:2405.04760.

118. Xu, J., Stokes, J.W., McDonald, G., Bai, X., Marshall, D., Wang, S., Swaminathan, A. and Li, Z. (2024) 'AutoAttacker: A Large Language Model Guided System to Implement Automatic Cyber-attacks', *arXiv preprint* arXiv:2403.01038.

119. Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B. et al. (2025) 'Qwen3 Technical Report', *arXiv preprint* arXiv:2505.09388.

120. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023) 'ReAct: Synergizing Reasoning and Acting in Language Models', *Proc. 11th International Conference on Learning Representations (ICLR 2023)*, Kigali.

121. Yao, Y., Duan, J., Xu, K., Cai, Y., Sun, Z. and Zhang, Y. (2024) 'A Survey on Large Language Model (LLM) Security and Privacy: The Good, The Bad, and The Ugly', *High-Confidence Computing*, Vol.4, No.2, 100211.

122. Zhang, A.K., Perry, N., Dulepet, R. et al. (2025) 'Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models', *Proc. 13th International Conference on Learning Representations (ICLR 2025)*.

123. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E. and Stoica, I. (2023) 'Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena', *Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track)*.

124. Ziems, C., Held, W., Shaikh, O., Chen, J., Zhang, Z. and Yang, D. (2024) 'Can Large Language Models Transform Computational Social Science?', *Computational Linguistics*, Vol.50, No.1, pp.237–291.

APPENDICES

Appendix 1: User Manual

This appendix is a complete, step-by-step guide to rebuilding the project from an empty folder, running every stage and reading the outputs. It is written so that a reader who has never seen the project can reproduce it. Commands are shown for Windows (PowerShell) because that is where the project was developed; the Linux and macOS equivalents differ only in path separators.

A1.1 What you need

Table A1.1: Prerequisites for running the project.

Requirement	Version	Notes
Python	3.11	Add it to the PATH during installation
Git	any recent	To clone the repository
Docker Desktop	any recent	Only needed to run the live honeypot; the analysis works without it
Disk space	about 2 GB	The raw real dataset is about 600 MB uncompressed
API access (optional)	–	Only to generate new synthetic sessions; the 120 existing logs are included

> **Safety note.** The attacker harness is for use only against a honeypot you own and have configured. It has no scanning or target-discovery capability and connects to exactly one host that you set. Never point it at a system you do not control.

A1.2 Step 1 — Get the code

```
git clone https://github.com/parthkansal823/Honeypot_RedTeaming-paper.git
cd Honeypot_RedTeaming-paper
python -m venv venv
venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

The requirements file installs Paramiko (SSH), scikit-learn, SciPy, NumPy, pandas and the provider SDKs. If you only want to reproduce the analysis, this is all you need: the 120 synthetic session logs are already in `logs/`, and the normalised corpora are in `normalized/`.

A1.3 Step 2 — Get the real dataset

The real data is not stored in the repository because of its size. Download it from Zenodo and extract it:

```
# Download cyberlab_2019-11-15.json.gz from https://doi.org/10.5281/zenodo.3687527
# Place it in the project root, then:
```

```
tar -xzf cyberlab_2019-11-15.json.gz
```

You should now have a folder `cyberlab_2019-11-15.json/` containing `cowrie_2019-11-15.json`.

A1.4 Step 3 — (Optional) Run the live honeypot

This step is only needed to generate *new* synthetic sessions. To reproduce the reported results you can skip to Step 5.

```
# Start the Cowrie honeypot in Docker (listens on localhost:2222)
.\run_docker.bat start-honeypot

# Check that it answers with an SSH banner
python check.py
```

Figure 4.1 in Chapter 4 shows the expected output of these two commands. To switch to the LLM-backed honeypot, edit `cowrie_etc/cowrie.cfg`, change `backend = shell` to `backend = llm`, set the `[llm]` model and endpoint, and restart the container.

A1.5 Step 4 — (Optional) Generate synthetic sessions

```
# One quick test session against the static honeypot
python better_ai_attacker.py --host 127.0.0.1 --port 2222 `
  --username root --password root `
  --provider bedrock_openai --model deepseek.v3.2 `
  --sessions 1 --output logs\test

# A full batch from a config file (20 sessions per model/backend)
python better_ai_attacker.py --config batch_config.json
```

Each session is saved as one JSON file under the output folder, and a line is printed for each session with its final status (**Figure 4.**2). Keep the failed sessions; their counts are part of the result.

A1.6 Step 5 — Parse and normalise the real data

```
python parse_cyberlab.py --in cyberlab_2019-11-15.json\cowrie_2019-11-15.json `
  --out normalized\real_organic.jsonl
```

The script prints a summary that should read `Parsed 76785 sessions` with `with_commands: 1672` (**Figure 4.**8). If it instead reports zero sessions, you are almost certainly using the wrong parser: `cowrie_session_parser.py` is for Cowrie's native logs, while the CyberLab export needs `parse_cyberlab.py`.

A1.7 Step 6 — Run the analysis

```
# Label-free comparison (JSD, Jaccard, C2ST, tactic profiles)
python domain_shift.py

# Attacker tables and paper figures
python generate_paper_tables.py
python make_figures.py

# Four-quadrant transfer experiment with the dual-LLM labels
python train_eval_classifier.py `
  --synthetic normalized\all_synthetic_120.jsonl `
  --real normalized\real_organic.jsonl `
  --real-labels annotation\labels_final.csv `
  --synthetic-labels annotation\labels_final.csv `
  --label-source llm-dual
```

Each script writes its results into the `results/` folder as both a readable `.md` file and a machine-readable `.json` file. Figures are written to `figures/`.

A1.8 Step 7 — (Optional) Reproduce the labelling

```
# Build the blind annotation sheets
python make_annotation_sheet.py

# After both annotators fill annotation\sheet_annotator1.csv and
# annotation\sheet_annotator2.csv, and the adjudicator fills
# annotation\adjudicated.csv, compute agreement and the final labels:
python agreement.py
```

This regenerates `annotation/labels_final.csv` and `results/annotation_agreement.md` (**Figure 4.**21). To run the human audit described in future work, hand the two blank sheets to two analysts and follow the same steps.

A1.9 Step 8 — Build this report

```
# Regenerate the descriptive statistics, diagrams and charts
python report\src\compute_stats.py
python report\src\make_diagrams.py
python report\src\make_charts.py
# Build the Word document from the Markdown chapters
python report\src\build_report.py
```

A1.10 Where every output lives

Table A1.2: Where to find each output.

You want…	Look in…
The raw synthetic sessions	`logs/<attacker>[_vs_qwen.qwen3-32b]/*.json`
The normalised corpora	`normalized/all_synthetic_120.jsonl`, `normalized/real_organic.jsonl`
Attacker behaviour (**Table 4.**5, **Table 4.**6)	`results/tables_1_2.md` / `.json`
Label-free comparison (**Table 4.**7 to **Table 4.**11)	`results/domain_shift.md` / `.json`
Annotation agreement	`results/annotation_agreement.md`; `annotation/labels_final.csv`
Transfer results (**Table 4.**14)	`results/table3_transfer.md` / `.json`
Sensitivity run	`results/sensitivity_dota/`
Keyword-rule plumbing check	`results/rule_labels/`
Figures	`figures/` (paper), `report/images/` (report)
Editable diagrams	`report/diagrams/*.drawio`

A1.11 Troubleshooting

Table A1.3: Common problems and fixes.

Symptom	Likely cause and fix
`parse_cyberlab.py` reports 0 sessions	Wrong input path, or the file is still gzip-compressed; extract it first
The honeypot stops responding	A heavy command (for example `find /`) froze Cowrie; the harness recovers automatically, or restart the container
Attacker replies are cut off	The LLM backend is slow; raise `--response-wait` to 3–5 seconds
Commands contain English sentences	Chat-format leakage from the model; confirm the `<|` stripping is active
Scores look suspiciously high	Make sure real folds are grouped by template (GroupKFold), not split randomly
A model refuses the task	Check the research framing in the system prompt; refusals are recorded as a status, not an error

Appendix 2: Annotation Rubric and Protocol

This appendix reproduces the labelling rubric used by both annotators and the adjudicator, so that the labels can be audited or reproduced.

A2.1 Instructions to annotators

Label each item with exactly **one** primary-intent label. Judge the whole command sequence, not single keywords. Ignore any leaked `<|channel|>…` text (it has already been stripped). Do not open the key file, and do not discuss items with the other annotator until agreement has been computed. Record a confidence of 1 (guess), 2 (fairly sure) or 3 (certain), and use the notes column for anything unusual.

A2.2 The eight labels

Table 3.14 in Chapter 3 gives the definitions. In short: **recon** is looking without changing anything; **credential access** is hunting for secrets, keys or password files; **privilege escalation** is trying to gain more rights; **persistence** is keeping access through cron, SSH keys or account changes; **malware drop** is downloading or running a payload; **resource hijacking** is deploying or inspecting a cryptominer; **defence evasion** is hiding or disabling defences; and **other** is for benign, unclear or noise sessions.

A2.3 Tie-break order

When several behaviours appear in one session, choose the goal that progressed furthest, in this order (highest first):

resource hijacking → malware drop → persistence → privilege escalation → credential access → defence evasion → recon → other.

A2.4 Worked rubric examples

Table A2.1: Example sessions and their labels under the rubric.

Command sequence (shortened)	Label	Reason
`whoami; ls -la /root; cat /etc/passwd; sudo -l`	recon	Only looking; `sudo -l` alone is recon
`cat /etc/shadow; find / -name id_rsa; cat ~/.ssh/*`	credential access	Actively hunting for secrets and keys
`echo "ssh-rsa …" >> ~/.ssh/authorized_keys`	persistence	Adds a key to keep access
`cat /proc/cpuinfo; echo …|chpasswd; wget <url>; sh …` (dota)	resource hijacking	Fingerprint + account takeover + miner kit; top of the tie-break
`cd /tmp; wget <url>/x.sh; chmod +x x.sh; ./x.sh`	malware drop	Download and run a payload, no miner identified
`/bin/busybox QWERT; cat /bin/busybox`	recon	Capability probe only, no download

A2.5 The adjudication record

The adjudicator resolved the 12 items on which the two annotators disagreed, writing a one-line reason for each. The most consequential was the dota template (item A036): one annotator chose malware drop, the other resource hijacking; the adjudicator chose resource hijacking because the session both drops a payload and deploys a miner, and resource hijacking is higher in the tie-break order. That decision sets the label of 1,038 real sessions and was reviewed and confirmed by the authors, and tested in the sensitivity run of Section 4.9.

Appendix 3: Data Schemas

This appendix documents the main record formats, so that the released files can be read without reverse-engineering them. The relationships between these records are shown in **Figure 3.**15 in Chapter 3.

A3.1 Raw synthetic session log

One JSON file per session under `logs/`. Key fields:

Table A3.1: Fields of a raw synthetic session log.

Field	Type	Meaning
session_id	string	Unique id: provider-model-index-hex
attacker_type	string	Always `synthetic_llm`
provider, model	string	How the attacker was reached
honeypot_backend_label	string	`static` or `dynamic_qwen.qwen3-32b`
status	string	One of the eight outcomes in **Table 3.**11
total_turns	integer	Number of turns recorded
recovery_attempts	integer	Infrastructure retries before this attempt succeeded
turns	array	One object per turn (below)

Each turn object has: `turn` (index), `timestamp`, `command`, `output`, `latency_sec`, and an optional `note` or `raw_response` for special cases (for example a refusal or a lost connection).

A3.2 Normalised session (both corpora)

The synthetic logs are converted, and the real data is parsed, into one common per-session shape, stored as JSON Lines.

Table A3.2: Fields of a normalised session.

Field	Type	Meaning
session_id	string	Unique id
source	string	`cyberlab_2019-11-15` (real) or the synthetic provider
attacker_model, honeypot_backend	string	Present for synthetic sessions
src_ip	string	Hashed source identifier (real only)
country, protocol	string	Geolocation and `ssh`/`telnet` (real only)
start, end, duration_s	string / number	Session timing (real only)
login_attempts	array	`{username, password, success}` (real only)
login_success	boolean	Whether any login succeeded (real only)
commands	array of strings	The commands typed, in order
downloads	array	`{url, shasum, outfile}` (real only)
command_count	integer	Number of commands

A3.3 Annotation files

Table A3.3: Annotation files and their columns.

File	Columns
`annotation_sheet.csv`	item_id, commands (the blind item shown to annotators)
`sheet_annotator1.csv`, `sheet_annotator2.csv`	item_id, n_commands, commands, label, confidence, notes
`key.csv` (hidden during labelling)	item_id, session_id, source, attacker_model, honeypot_backend, template_size
`adjudicated.csv`	item_id, label, rationale
`disagreements.csv`	item_id, label_annotator1, label_annotator2, notes1, notes2, commands, label_final
`labels_final.csv`	session_id, label, source

A3.4 Result files

Every reported number is written by a script into `results/`. Each analysis produces a readable `.md` file and a `.json` file with the same content; the `.json` files are what the report's own charts read, so the numbers in the charts and in the text cannot drift apart.

Appendix 4: List of Scripts

Table A4.1: The project's scripts and what they do.

Script	Purpose
`better_ai_attacker.py`	The attacker harness used for the study (recovery, retries, multiple providers)
`ai_attacker.py`, `another_ai_attacker.py`	Earlier harness versions, kept for history
`probe.py`	Check which model endpoints answer
`check.py`	Read the honeypot's SSH banner a few times
`parse_cyberlab.py`	Parse the CyberLab export into normalised sessions
`cowrie_session_parser.py`	Parse Cowrie's *native* logs (not the CyberLab format)
`convert_live_sessions.py`	Convert harness logs into the normalised schema
`session_features.py`	Normalisation, segments, verbs, tactic tags, template keys
`domain_shift.py`	Label-free comparison
`make_annotation_sheet.py`	Build the blind annotation sheets and key
`agreement.py`	Cohen's kappa, disagreements, final labels
`train_eval_classifier.py`	Four-quadrant transfer experiment
`generate_paper_tables.py`	Attacker behaviour tables
`make_figures.py`	Paper figures
`validate_logs.py`	Scan logs for corruption from the old marker design
`analyze.py`, `analyze_logs.ps1`	Quick per-model summaries
`report/src/compute_stats.py`	Descriptive statistics for this report
`report/src/make_diagrams.py`	Flowcharts and block diagrams (draw.io + PNG)
`report/src/make_charts.py`	Report charts and console pictures
`report/src/build_report.py`	Assemble the Word document from the Markdown chapters

Appendix 5: Glossary

Table A5.1: Glossary of key terms.

Term	Meaning in this report
Honeypot	A decoy system whose only purpose is to be attacked and observed
Cowrie	A medium-interaction SSH/Telnet honeypot that emulates a shell
Static backend	Cowrie's built-in command emulation
LLM backend	A Cowrie mode in which an LLM writes the shell's replies
Synthetic session	A session produced by an LLM attacker against our honeypot
Real session	A session from the public CyberLab honeynet data
Interactive session	A session in which at least one command was typed
Behaviour template	A group of sessions with identical normalised command sequences
Normalisation	Masking session-specific values so similar behaviours collapse together
Verb	The first word of a command segment (for example `ls`, `wget`)
Tactic	An ATT&CK-style category of attacker goal (discovery, persistence, …)
Intent label	The single primary goal assigned to a session by annotators
JSD	Jensen-Shannon divergence; a 0–1 distance between two distributions
C2ST	Classifier two-sample test; uses a classifier to measure how separable two sets are
AUC	Area under the ROC curve; 0.5 is random, 1.0 is perfect separation
Macro-F1	Average of per-class F1 scores, over the classes present in the test set
GroupKFold	Cross-validation that keeps all sessions of one template in the same fold
Transfer	Training on one corpus and testing on another
dota	The Outlaw-style cryptominer dropper that dominates the real data
Chat-format leakage	A model typing its own control tokens and reasoning into the shell

Appendix 6: Catalogue of the Real Behaviour Templates

The 1,672 interactive real sessions collapse to 22 behaviour templates after normalisation (Section 3.7.8). The table lists every template with its size, its share of the interactive sessions, the running total, its length, its final intent label and the descriptive tactics its commands touch.

Table A6.1: The real behaviour templates, largest first.

#	Sessions	Share	Cumulative	Commands	Final label	Tactics touched
1	1,038	62.1%	62.1%	23	Resource hijacking	account manipulation, discovery, execution, resource hijacking
2	429	25.7%	87.7%	19	Persistence	account manipulation, discovery, execution, resource hijacking
3	63	3.8%	91.5%	6	Recon	execution
4	43	2.6%	94.1%	10	Recon	discovery, execution
5	36	2.2%	96.2%	2	Malware drop	execution, ingress tool transfer
6	16	1.0%	97.2%	6	Recon	execution
7	8	0.5%	97.7%	1	Persistence	account manipulation, credential access, defense evasion, discovery
8	6	0.4%	98.0%	1	Recon	discovery
9	6	0.4%	98.4%	50	Malware drop	account manipulation, credential access, defense evasion, ingress tool transfer, persistence
10	5	0.3%	98.7%	46	Malware drop	account manipulation, credential access, defense evasion, ingress tool transfer
11	4	0.2%	98.9%	8	Recon	discovery
12	3	0.2%	99.1%	13	Persistence	account manipulation, discovery, execution
13	3	0.2%	99.3%	1	Malware drop	discovery, execution, ingress tool transfer
14	2	0.1%	99.4%	14	Persistence	account manipulation, discovery, execution
15	2	0.1%	99.5%	1	Recon	discovery
16	2	0.1%	99.6%	6	Recon	execution
17	1	0.1%	99.7%	22	Resource hijacking	account manipulation, discovery, execution, resource hijacking
18	1	0.1%	99.8%	48	Malware drop	account manipulation, credential access, defense evasion, ingress tool transfer
19	1	0.1%	99.8%	21	Persistence	account manipulation, discovery, execution, resource hijacking
20	1	0.1%	99.9%	2	Malware drop	execution, resource hijacking

#	Sessions	Share	Cumulative	Commands	Final label	Tactics touched
21	1	0.1%	99.9%	27	Resource hijacking	account manipulation, defense evasion, discovery, execution, ingress tool transfer, privilege escalation
22	1	0.1%	100.0%	6	Recon	execution

The two largest templates alone contain 1,467 of 1,672 sessions. Both begin with the same CPU fingerprinting and differ mainly in whether a payload kit is unpacked afterwards, which is why one is labelled resource hijacking and the other persistence.

Appendix 7: Adjudicated Annotation Disagreements

The two blind annotators disagreed on 12 of 184 items. A separate blind adjudicator chose the final label using the rubric and wrote a one-line reason. The source column comes from the hidden key file and was not visible to anyone during labelling or adjudication.

Table A7.1: Every disagreement, the final label and the adjudicator's reason.

Item	Source	Annotator A	Annotator B	Final	Adjudicator's reason
A003	Gemma-3-4B / Dynamic Qwen-32B	Privilege escalation	Recon	Recon	Dominated by recon: whoami, ls, cat passwd, sudo -l (recon); nano edits interactive and unconfirmed, no write committed.
A011	GPT-OSS-120B / Static Cowrie	Persistence	Credential access	Persistence	Heavy credential harvesting, but attacker writes @reboot root reverse-shell cron entry; persistence outranks credential_access.
A019	GPT-OSS-120B / Dynamic Qwen-32B	Credential access	Persistence	Persistence	Steals shadow/keys/databases then installs reverse-shell cron via crontab; persistence outranks credential_access in tie-break.
A036	real template (1 sessions)	Resource hijacking	Malware drop	Resource hijacking	Dota campaign: repeated CPU profiling plus miner/bot download and execution; resource_hijacking outranks malware_drop.
A042	Gemma-3-4B / Dynamic Qwen-32B	Recon	Credential access	Credential access	Recon plus grep for password in bashrc; searching for secrets makes credential_access outrank recon.
A052	real template (1 sessions)	Persistence	Malware drop	Resource hijacking	Dota miner campaign: CPU profiling, chpasswd, miner download; resource_hijacking outranks persistence and malware_drop.
A058	Gemma-3-4B / Static Cowrie	Recon	Credential access	Credential access	Reads /etc/shadow content via strings/grep for hashes; credential_access outranks surrounding recon.
A064	Gemma-3-4B / Static Cowrie	Recon	Credential access	Credential access	Directly cats /etc/shadow to steal hashes; credential_access outranks recon despite apt noise.
A073	Gemma-3-4B / Dynamic Qwen-32B	Credential access	Privilege escalation	Privilege escalation	Reads shadow then sudo cat and sudo -i to gain root shell; privilege_escalation outranks credential_access.
A079	DeepSeek-V3.2 / Dynamic Qwen-32B	Recon	Credential access	Credential access	Reads shadow hashes and SSH authorized_keys; credential harvesting outranks recon and unused SUID hunt.
A093	GPT-OSS-120B / Dynamic Qwen-32B	Malware drop	Persistence	Persistence	Appends attacker SSH key and installs payload cron; persistence achieved in-session, outranks credential_access.
A155	real template (1038 sessions)	Resource hijacking	Malware drop	Resource hijacking	Dota miner campaign identical to A052: CPU profiling, chpasswd, miner download; resource_hijacking wins.

Appendix 8: Summary of All Synthetic Sessions

This appendix lists every one of the 120 synthetic sessions with its cell, start time (UTC), how it ended, how many commands it issued and the final intent label it received. Sessions are numbered within each cell in the order they were run.

DeepSeek-V3.2 against Dynamic Qwen-32B

Table A8.1: Synthetic sessions of DeepSeek-V3.2 against Dynamic Qwen-32B.

#	Started (UTC)	Status	Commands	Final label
1	2026-09-27 13:03:59	completed	6	Credential access
2	2026-09-27 13:10:48	completed	18	Credential access
3	2026-09-27 13:12:30	completed	25	Credential access
4	2026-09-27 13:14:59	completed	14	Credential access
5	2026-09-27 13:28:05	completed	19	Credential access
6	2026-09-27 13:30:36	completed	20	Credential access
7	2026-09-27 13:33:11	completed	17	Credential access
8	2026-09-27 13:35:29	completed	20	Credential access
9	2026-09-27 13:38:19	completed	13	Credential access
10	2026-09-27 17:21:00	completed	15	Credential access
11	2026-09-27 17:22:33	completed	21	Credential access
12	2026-09-27 17:24:38	completed	14	Credential access
13	2026-09-27 17:26:08	completed	23	Credential access
14	2026-09-27 17:29:18	completed	16	Credential access
15	2026-09-27 17:31:04	completed	21	Credential access
16	2026-09-27 17:33:10	completed	9	Credential access
17	2026-09-27 17:34:21	completed	9	Recon
18	2026-09-27 17:36:43	completed	23	Credential access
19	2026-09-27 17:39:09	completed	16	Credential access
20	2026-09-27 17:40:32	completed	18	Credential access

DeepSeek-V3.2 against Static Cowrie

Table A8.2: Synthetic sessions of DeepSeek-V3.2 against Static Cowrie.

#	Started (UTC)	Status	Commands	Final label
1	2026-09-24 07:26:49	attacker refused	7	Credential access
2	2026-09-24 23:08:00	completed	13	Recon
3	2026-09-24 23:15:07	completed	8	Credential access
4	2026-09-24 23:51:40	completed	22	Recon

#	Started (UTC)	Status	Commands	Final label
5	2026-09-24 23:53:43	completed	12	Credential access
6	2026-09-24 23:54:40	completed	12	Credential access
7	2026-09-27 00:42:35	completed	23	Malware drop
8	2026-09-27 00:44:38	completed	21	Credential access
9	2026-09-27 01:29:06	completed	13	Credential access
10	2026-09-27 01:45:31	completed	13	Credential access
11	2026-09-27 01:46:33	completed	9	Credential access
12	2026-09-27 01:47:16	completed	18	Credential access
13	2026-09-27 02:00:24	completed	11	Credential access
14	2026-09-27 02:05:01	completed	15	Credential access
15	2026-09-27 02:06:16	completed	9	Credential access
16	2026-09-27 02:07:00	completed	9	Credential access
17	2026-09-27 02:10:13	completed	18	Recon
18	2026-09-27 02:11:50	completed	9	Credential access
19	2026-09-27 02:12:35	completed	18	Credential access
20	2026-09-27 02:15:02	completed	10	Credential access

GPT-OSS-120B against Dynamic Qwen-32B

Table A8.3: Synthetic sessions of GPT-OSS-120B against Dynamic Qwen-32B.

#	Started (UTC)	Status	Commands	Final label
1	2026-09-27 18:10:05	llm error	9	Credential access
2	2026-09-27 18:13:01	llm error	20	Credential access
3	2026-09-27 18:15:42	llm error	14	Credential access
4	2026-09-27 18:52:25	llm error	11	Credential access
5	2026-09-27 19:46:28	completed	11	Credential access
6	2026-09-27 19:57:03	connection lost	22	Credential access
7	2026-09-27 20:00:40	llm error	18	Credential access
8	2026-09-27 20:04:43	connection lost	15	Credential access
9	2026-09-27 20:08:33	connection lost	14	Credential access
10	2026-09-27 20:12:18	connection lost	17	Credential access
11	2026-09-29 16:48:59	completed	14	Persistence
12	2026-09-29 17:13:11	connection lost	22	Persistence
13	2026-09-29 17:17:02	connection lost	16	Credential access
14	2026-09-29 17:21:26	connection lost	10	Credential access
15	2026-09-29 17:25:57	connection lost	20	Credential access
16	2026-09-29 17:30:02	connection lost	21	Credential access
17	2026-09-29 17:33:45	llm error	17	Credential access

#	Started (UTC)	Status	Commands	Final label
18	2026-09-29 17:38:05	completed	16	Persistence
19	2026-09-29 17:41:51	connection lost	16	Persistence
20	2026-09-29 17:45:35	completed	22	Credential access

GPT-OSS-120B against Static Cowrie

Table A8.4: Synthetic sessions of GPT-OSS-120B against Static Cowrie.

#	Started (UTC)	Status	Commands	Final label
1	2026-09-27 04:47:23	completed	15	Credential access
2	2026-09-27 05:36:51	max turns reached	25	Persistence
3	2026-09-27 05:39:26	max turns reached	25	Credential access
4	2026-09-27 05:42:12	completed	12	Credential access
5	2026-09-27 05:47:59	max turns reached	20	Credential access
6	2026-09-27 09:47:11	max turns reached	20	Credential access
7	2026-09-27 09:49:26	max turns reached	20	Persistence
8	2026-09-27 09:51:25	max turns reached	20	Credential access
9	2026-09-27 09:54:07	completed	13	Credential access
10	2026-09-27 10:05:25	max turns reached	20	Credential access
11	2026-09-27 10:18:09	max turns reached	20	Credential access
12	2026-09-27 10:20:49	max turns reached	20	Credential access
13	2026-09-27 10:27:02	max turns reached	20	Credential access
14	2026-09-27 10:38:09	max turns reached	20	Credential access
15	2026-09-27 10:40:30	max turns reached	20	Credential access
16	2026-09-27 10:56:45	max turns reached	20	Persistence
17	2026-09-27 11:07:33	max turns reached	20	Persistence
18	2026-09-27 11:12:41	max turns reached	20	Credential access
19	2026-09-27 11:16:04	max turns reached	20	Credential access
20	2026-09-27 11:22:30	max turns reached	20	Credential access

Gemma-3-4B against Dynamic Qwen-32B

Table A8.5: Synthetic sessions of Gemma-3-4B against Dynamic Qwen-32B.

#	Started (UTC)	Status	Commands	Final label
1	2026-09-27 19:12:09	completed	7	Credential access
2	2026-09-27 19:13:21	completed	19	Malware drop
3	2026-09-27 19:14:42	completed	19	Malware drop
4	2026-09-27 19:16:29	max turns reached	25	Recon

#	Started (UTC)	Status	Commands	Final label
5	2026-09-27 19:19:00	completed	13	Persistence
6	2026-09-27 19:19:50	completed	15	Credential access
7	2026-09-27 19:21:21	completed	22	Recon
8	2026-09-27 19:22:53	completed	15	Credential access
9	2026-09-27 19:24:12	completed	16	Privilege escalation
10	2026-09-27 19:25:53	completed	11	Credential access
11	2026-09-27 19:27:29	completed	24	Persistence
12	2026-09-27 19:29:25	completed	22	Persistence
13	2026-09-27 19:31:18	completed	19	Credential access
14	2026-09-27 19:32:44	completed	22	Recon
15	2026-09-27 19:34:51	completed	19	Credential access
16	2026-09-27 19:37:06	completed	18	Credential access
17	2026-09-27 19:38:49	completed	17	Privilege escalation
18	2026-09-27 19:41:08	completed	17	Credential access
19	2026-09-27 19:42:12	completed	25	Credential access
20	2026-09-27 19:43:54	max turns reached	25	Malware drop

Gemma-3-4B against Static Cowrie

Table A8.6: Synthetic sessions of Gemma-3-4B against Static Cowrie.

#	Started (UTC)	Status	Commands	Final label
1	2026-09-27 02:39:32	completed	22	Credential access
2	2026-09-27 02:42:41	completed	18	Credential access
3	2026-09-27 02:44:14	max turns reached	25	Recon
4	2026-09-27 02:46:32	completed	15	Credential access
5	2026-09-27 02:54:26	max turns reached	25	Recon
6	2026-09-27 02:56:28	max turns reached	25	Recon
7	2026-09-27 02:58:22	completed	24	Credential access
8	2026-09-27 03:01:17	completed	20	Credential access
9	2026-09-27 03:02:52	completed	16	Recon
10	2026-09-27 03:04:03	timed out	16	Credential access
11	2026-09-27 03:31:33	max turns reached	25	Recon
12	2026-09-27 03:33:33	max turns reached	25	Recon
13	2026-09-27 03:54:55	max turns reached	25	Credential access
14	2026-09-27 03:56:58	completed	20	Credential access
15	2026-09-27 04:01:58	completed	18	Credential access
16	2026-09-27 04:03:22	max turns reached	25	Recon
17	2026-09-27 04:05:25	completed	12	Malware drop

#	Started (UTC)	Status	Commands	Final label
18	2026-09-27 04:09:58	completed	22	Malware drop
19	2026-09-27 04:17:38	max turns reached	25	Credential access
20	2026-09-27 04:24:59	completed	17	Credential access

Appendix 9: Configuration Files

These are the configuration files used to run the testbed. API keys and other secrets are never stored in them; where a key field exists it is shown as `<redacted>`.

Cowrie honeypot configuration (cowrie_etc/cowrie.cfg)

```
# Cowrie Honeypot Configuration for Research Paper Testbed

[honeypot]
hostname = svr04
log_path = var/log/cowrie
download_path = var/lib/cowrie/downloads
contents_path = honeyfs
txtcmds_path = txtcmds
ttylog = true
interactive_timeout = 300
authentication_timeout = 120

[ssh]
enabled = true
listen_endpoints = tcp:2222:interface=0.0.0.0
version = SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5

[telnet]
enabled = false

# ============================================================================
# BACKEND CONFIGURATION
# Set backend to 'shell' for Condition A (Static Honeypot Baseline)
# Set backend to 'llm' for Condition B (Dynamic LLM-driven Honeypot)
# ============================================================================
backend = shell

[shell]
filesystem = share/cowrie/fs.pickle
arch = linux-x64-lsb
kernel_version = 5.4.0-42-generic
kernel_build_string = #46-Ubuntu SMP Fri Jul 10 00:24:02 UTC 2020

# If switching to backend = llm, configure the LLM endpoint below:
[llm]
model = qwen.qwen3-32b
base_url = https://bedrock-mantle.us-east-1.api.aws/v1
api_key = <redacted>
temperature = 0.7
max_tokens = 512
```

Docker Compose file (docker-compose.yml)

```
version: '3.8'

services:
  # ============================================================================
  # Cowrie Honeypot Service (Isolated Target Environment)
  # ============================================================================
  cowrie:
    image: cowrie/cowrie:latest
    container_name: cowrie-honeypot
    ports:
      - "2222:2222"
    volumes:
```

```
      - ./cowrie_etc/cowrie.cfg:/cowrie/cowrie-git/etc/cowrie.cfg:ro
      - cowrie-data:/cowrie/cowrie-git/var
      - ./var/log/cowrie:/cowrie/cowrie-git/var/log/cowrie
    environment:
      - BEDROCK_API_KEY=${BEDROCK_API_KEY:-}
      - OPENAI_API_KEY=${OPENAI_API_KEY:-}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY:-}
      - GEMINI_API_KEY=${GEMINI_API_KEY:-}
    restart: unless-stopped
    networks:
      - honeypot-net

  # ============================================================================
  # AI Attacker & Evaluation Pipeline Service
  # ============================================================================
  attacker:
    build:
      context: .
      dockerfile: Dockerfile.attacker
    container_name: ai-attacker-evaluator
    depends_on:
      - cowrie
    volumes:
      - ./logs:/app/logs
      - ./normalized:/app/normalized
      - ./results:/app/results
      - ./var/log/cowrie:/app/var/log/cowrie:ro
      - .:/app
    environment:
      - BEDROCK_API_KEY=${BEDROCK_API_KEY:-}
      - OPENAI_API_KEY=${OPENAI_API_KEY:-}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY:-}
      - GEMINI_API_KEY=${GEMINI_API_KEY:-}
    networks:
      - honeypot-net
    command: python generate_paper_tables.py

networks:
  honeypot-net:
    driver: bridge

volumes:
  cowrie-data:
```

Analysis container (Dockerfile.attacker)

```
# Base image for AI Attacker & Evaluation Pipeline
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies (SSH utilities, build tools)
RUN apt-get update && apt-get install -y --no-install-recommends \
    openssh-client \
    curl \
    git \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements and install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application source code
COPY . .

# Default entrypoint
```

```
CMD ["python", "generate_paper_tables.py"]
```

Python dependencies (requirements.txt)

```
paramiko>=3.4.0
boto3>=1.34.0
anthropic>=0.40.0
openai>=1.50.0
requests>=2.31.0
scikit-learn>=1.4.0
scipy>=1.12.0
numpy>=1.26.0
pandas>=2.2.0
```

Appendix 10: Source Code of the Analysis Scripts

This appendix reproduces the analysis code that produced the reported results, so that every method described in Chapter 3 can be checked line by line. The full repository, including the testbed scripts, is available on GitHub.

Normalisation, verbs, tactic tags and templates (session_features.py)

```
"""
Shared command normalization and descriptive ATT&CK-tactic tagging, used by
domain_shift.py, train_eval_classifier.py and make_annotation_sheet.py so all
three see sessions the same way.

The tactic tagger is DESCRIPTIVE (what share of sessions touch each tactic).
It is deliberately not used as a training label for the reported numbers -
that would be the circular weak-labeling problem described in
honeypot_research_paper_plan.md.
"""

import json
import re

_URL = re.compile(r"\b(?:https?|ftp|tftp)://\S+", re.I)
_IP = re.compile(r"\b\d{1,3}(?:\.\d{1,3}){3}(?::\d+)?\b")
_QUOTED = re.compile(r"\"(?:[^\"\\]|\\.)*\"|'[^']*'")
_HEX = re.compile(r"\b[0-9a-f]{16,}\b", re.I)
_RANDTOK = re.compile(r"\b(?=[A-Za-z0-9]*\d)(?=[A-Za-z0-9]*[A-Za-z])[A-Za-z0-9]{8,}\b")
_NUM = re.compile(r"\b\d+\b")
# Mirai-family bots echo a random 5-letter tag via `/bin/busybox ABCDE` to fingerprint
# the shell; mask it (but keep real applet names) so variants collapse to one template.
_BUSYBOX_APPLETS = {"wget", "tftp", "ftpget", "chmod", "cat", "echo", "ps", "rm", "cp",
"mv",
                    "mkdir", "sh", "ls", "nc", "dd", "kill", "killall", "mount", "uname",
"sleep",
                    "touch", "ln", "grep", "head", "tail", "ifconfig", "ip", "id",
"telnetd",
                    "httpd", "chown", "find", "top", "free", "df", "ping", "route",
"netstat",
                    "hostname", "awk", "sed", "tar", "gzip", "unzip", "whoami", "passwd"}
_BUSYBOX_TAG = re.compile(r"\bbusybox ([a-z]{3,12})\b")
_SPLIT = re.compile(r"\|\||&&|[;|&\n]|\$\(|`")

# Command-level tactic patterns, matched against a normalized segment.
TACTIC_PATTERNS = {
    "discovery":
r"^(uname|whoami|id|hostname|ifconfig|ip|ps|netstat|ss|ls|pwd|w|who|uptime|df|free|"

r"lscpu|nproc|top|env|which|lsb_release|dmesg|mount|arp|route|last)\b|/proc/(cpuinfo|meminf
o|version)"
                r"|^cat /etc/(os-release|issue|hostname)|^crontab -l|^find\b(?!.*-
perm)|^(wget|tftp|curl|ftpget)$",
    "credential_access":
r"/etc/shadow|/etc/passwd\b(?!.*chpasswd)|\.ssh|id_rsa|\.bash_history|^history\b|"
                        r"password|passwd\.|secret|credential|\.aws|\.my\.cnf|wp-config",
    "account_manipulation":
r"chpasswd|^passwd\b|\|\s*passwd|useradd|adduser|usermod|authorized_keys",
    "persistence": r"^crontab (?!-l)|/etc/cron|systemctl
enable|rc\.local|>>?\s*\S*\.(bashrc|profile)|/etc/init\.d",
    "privilege_escalation": r"^sudo\b|^su\b|-perm -[42]000|getcap|pkexec|/etc/sudoers",
    "defense_evasion": r"history -c|unset hist|histfile|rm
.*(\.bash_history|/var/log)|chattr|^iptables|"
                       r"^pkill|^kill(all)?\b|ufw disable|setenforce",
    # Bare `wget`/`tftp` with no argument is Mirai's capability probe, not a download.
```

```
    "ingress_tool_transfer": r"^(wget|curl|tftp|ftpget|scp|nc|ncat)\s+\S|busybox
(wget|tftp)\s+\S",
    "execution": r"chmod \+?[0-7]*x|^\./|^(sh|bash)\b|base64 (-d|--
decode)|^nohup\b|^perl\b|^python",
    "resource_hijacking": r"xmrig|minerd|stratum|cryptonight|\bminer\b|\.rsync/|dota",
}
_TACTIC_RE = {t: re.compile(p, re.I) for t, p in TACTIC_PATTERNS.items()}
TACTICS = list(TACTIC_PATTERNS)


def load_jsonl(path):
    with open(path, encoding="utf-8-sig") as f:
        return [json.loads(line) for line in f if line.strip()]


def has_chat_leak(cmd):
    """GPT-OSS sometimes emitted harmony-format tokens plus its own reasoning
    ("<|channel|>analysis<|message|>...") into the shell command."""
    return "<|" in cmd


def strip_chat_leak(cmd):
    return cmd.split("<|", 1)[0]


def normalize_cmd(cmd):
    """Mask session-specific values (URLs, IPs, quoted args, random passwords) so
    the same bot script run from different IPs normalizes to one template."""
    c = strip_chat_leak(cmd).strip().lower()
    c = _URL.sub("<url>", c)
    c = _IP.sub("<ip>", c)
    c = _QUOTED.sub("<str>", c)
    c = _HEX.sub("<hex>", c)
    c = _RANDTOK.sub("<tok>", c)
    c = _NUM.sub("<n>", c)
    c = _BUSYBOX_TAG.sub(lambda m: m.group(0) if m.group(1) in _BUSYBOX_APPLETS else
"busybox <tag>", c)
    return re.sub(r"\s+", " ", c)


def segments(cmd):
    """Split a normalized command line into its chained/piped sub-commands."""
    return [s.strip() for s in _SPLIT.split(normalize_cmd(cmd)) if s.strip()]


def verb(segment):
    toks = [t for t in segment.split() if "=" not in t]  # drop VAR=x prefixes
    return toks[0].rsplit("/", 1)[-1] if toks else None


def session_verbs(session):
    return [v for c in session.get("commands") or [] for v in map(verb, segments(c)) if v]


def session_tactics(session):
    found = set()
    for c in session.get("commands") or []:
        for seg in segments(c):
            found.update(t for t, rx in _TACTIC_RE.items() if rx.search(seg))
    return found


def template_key(session):
    return tuple(normalize_cmd(c) for c in session.get("commands") or [])
```

CyberLab parser (parse_cyberlab.py)

```
"""
Parse the CyberLab Cowrie dataset (Zenodo DOI 10.5281/zenodo.3687527) into the
same per-session JSONL schema that train_eval_classifier.py consumes.

The CyberLab export is NOT Cowrie's native JSON-lines log, so
cowrie_session_parser.py silently parses 0 sessions from it:
  - the file is one big JSON array, one `{"<session_id>": [events...]},` per line
  - IPs are replaced by `src_ip_identifier` hashes
  - command text lives in `message` as "CMD: <command>" (there is no `input` field)

Streams line-by-line, so the 600 MB file never has to fit in memory.

Usage:
    python parse_cyberlab.py [--in cyberlab_2019-11-15.json/cowrie_2019-11-15.json]
                             [--out normalized/real_organic.jsonl]
"""

import argparse
import json
from collections import Counter, defaultdict

from cowrie_session_parser import flag_findings

DEFAULT_IN = "cyberlab_2019-11-15.json/cowrie_2019-11-15.json"
DEFAULT_OUT = "normalized/real_organic.jsonl"


def iter_session_blocks(path):
    """Yield (session_id, events) from the array-of-single-key-objects format."""
    with open(path, "r", encoding="utf-8", errors="replace") as f:
        for line in f:
            line = line.strip().lstrip("[").rstrip(",").rstrip("]")
            if not line:
                continue
            try:
                obj = json.loads(line)
            except json.JSONDecodeError:
                continue
            for sid, events in obj.items():
                yield sid, events


def new_session(sid):
    return {
        "session_id": sid, "source": "cyberlab_2019-11-15", "src_ip": None,
        "country": None, "protocol": None, "start": None, "end": None,
        "duration_s": None, "login_attempts": [], "login_success": False,
        "commands": [], "downloads": [], "client_version": None,
    }


def apply_event(s, ev):
    s["src_ip"] = s["src_ip"] or ev.get("src_ip_identifier")
    geo = ev.get("geolocation_data") or {}
    s["country"] = s["country"] or geo.get("country_code2")
    ts = ev.get("timestamp")
    if ts:
        s["start"] = ts if s["start"] is None or ts < s["start"] else s["start"]
        s["end"] = ts if s["end"] is None or ts > s["end"] else s["end"]

    eid = ev.get("eventid", "")
    if eid == "cowrie.session.connect":
        s["protocol"] = ev.get("protocol")
    elif eid in ("cowrie.login.success", "cowrie.login.failed"):
        success = eid.endswith("success")
        s["login_attempts"].append({"username": ev.get("username"),
```

```
                                    "password": ev.get("password"), "success": success})
        s["login_success"] = s["login_success"] or success
    elif eid == "cowrie.command.input":
        msg = ev.get("input") or ev.get("message") or ""
        s["commands"].append(msg[5:] if msg.startswith("CMD: ") else msg)
    elif eid in ("cowrie.session.file_download", "cowrie.session.file_upload"):
        s["downloads"].append({"url": ev.get("url"), "shasum": ev.get("shasum"),
                               "outfile": ev.get("outfile")})
    elif eid == "cowrie.client.version":
        s["client_version"] = ev.get("ssh_client_version") or ev.get("version")
    elif eid == "cowrie.session.closed":
        s["duration_s"] = ev.get("duration")


def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--in", dest="inp", default=DEFAULT_IN)
    ap.add_argument("--out", default=DEFAULT_OUT)
    args = ap.parse_args()

    # A session can be split across several array entries, so merge by id.
    sessions = {}
    for sid, events in iter_session_blocks(args.inp):
        s = sessions.setdefault(sid, new_session(sid))
        for ev in sorted(events, key=lambda e: e.get("timestamp") or ""):
            apply_event(s, ev)

    stats = Counter()
    with open(args.out, "w", encoding="utf-8") as out:
        for s in sessions.values():
            s["findings"] = flag_findings(s)
            s["command_count"] = len(s["commands"])
            s["login_attempt_count"] = len(s["login_attempts"])
            stats["sessions"] += 1
            stats["with_login_success"] += s["login_success"]
            stats["with_commands"] += s["command_count"] > 0
            stats["with_downloads"] += bool(s["downloads"])
            stats[f"proto_{s['protocol']}"] += 1
            out.write(json.dumps(s) + "\n")

    print(f"Parsed {stats['sessions']} sessions -> {args.out}")
    for k, v in sorted(stats.items()):
        print(f"  {k}: {v}")


if __name__ == "__main__":
    main()
```

Label-free comparison (domain_shift.py)

```
"""
Label-free comparison of synthetic LLM-attacker sessions vs real CyberLab
honeypot sessions. Needs no annotation, so it is immune to the circular
weak-label problem and is the paper's primary evidence on realism.

Measures:
  1. corpus structure (effective number of distinct behaviors after template dedup)
  2. ATT&CK tactic coverage per corpus
  3. command-verb distributions: Jensen-Shannon divergence, vocabulary overlap
  4. classifier two-sample test (C2ST): can a simple model tell the corpora apart?
  5. per attacker x backend condition: which synthetic source is closest to real?

Usage:
    python domain_shift.py [--synthetic normalized/all_synthetic_120.jsonl]
                           [--real normalized/real_organic.jsonl]
```

```
Writes results/domain_shift.md and results/domain_shift.json.
"""

import argparse
import json
import random
import statistics
from collections import Counter
from pathlib import Path

import numpy as np
from scipy.spatial.distance import jensenshannon
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import RepeatedStratifiedKFold

from session_features import (TACTICS, has_chat_leak, load_jsonl, normalize_cmd, segments,
                              session_tactics, session_verbs, template_key)

SEED = 42
N_BOOT = 500


def dedup_by_template(sessions):
    seen, out = set(), []
    for s in sessions:
        k = template_key(s)
        if k not in seen:
            seen.add(k)
            out.append(s)
    return out


def corpus_stats(sessions):
    cmds = [c for s in sessions for c in s["commands"]]
    templates = Counter(map(template_key, sessions))
    return {
        "sessions": len(sessions),
        "distinct_templates": len(templates),
        "top5_template_share": sum(n for _, n in templates.most_common(5)) / len(sessions),
        "median_cmds_per_session": statistics.median(len(s["commands"]) for s in sessions),
        "mean_chars_per_cmd": statistics.mean(len(c) for c in cmds),
        "pct_cmds_chained": sum(len(segments(c)) > 1 for c in cmds) / len(cmds),
    }


def tactic_shares(sessions):
    counts = Counter(t for s in sessions for t in session_tactics(s))
    return {t: counts[t] / len(sessions) for t in TACTICS}


def verbs_of(session):
    """session_verbs() is regex-heavy; cache it on the session for the bootstrap."""
    if "_verbs" not in session:
        session["_verbs"] = Counter(session_verbs(session))
    return session["_verbs"]


def jsd(a_sessions, b_sessions):
    a, b = Counter(), Counter()
    for s in a_sessions:
        a.update(verbs_of(s))
    for s in b_sessions:
        b.update(verbs_of(s))
    vocab = sorted(a.keys() | b.keys())
    pa = np.array([a[v] for v in vocab], dtype=float)
    pb = np.array([b[v] for v in vocab], dtype=float)
    return float(jensenshannon(pa / pa.sum(), pb / pb.sum(), base=2))
```

```
def jsd_with_ci(a_sessions, b_sessions, rng):
    point = jsd(a_sessions, b_sessions)
    boots = sorted(jsd(rng.choices(a_sessions, k=len(a_sessions)), b_sessions) for _ in
range(N_BOOT))
    return point, boots[int(0.025 * N_BOOT)], boots[int(0.975 * N_BOOT)]


def c2st_auc(synthetic, real, text_fn):
    """5x5 repeated stratified CV AUC of a logistic regression separating the two
    corpora. 0.5 = indistinguishable, 1.0 = trivially separable."""
    docs = [text_fn(s) for s in synthetic + real]
    y = np.array([1] * len(synthetic) + [0] * len(real))
    aucs = []
    cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=SEED)
    for tr, te in cv.split(docs, y):
        vec = TfidfVectorizer(token_pattern=r"[^\s]+", min_df=1)
        X_tr = vec.fit_transform([docs[i] for i in tr])
        X_te = vec.transform([docs[i] for i in te])
        clf = LogisticRegression(max_iter=2000, class_weight="balanced").fit(X_tr, y[tr])
        aucs.append(roc_auc_score(y[te], clf.predict_proba(X_te)[:, 1]))
    return float(np.mean(aucs)), float(np.std(aucs))


def fmt_pct(x):
    return f"{100 * x:.1f}%"


def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--synthetic", default="normalized/all_synthetic_120.jsonl")
    ap.add_argument("--real", default="normalized/real_organic.jsonl")
    ap.add_argument("--out-dir", default="results")
    args = ap.parse_args()
    rng = random.Random(SEED)

    synthetic = [s for s in load_jsonl(args.synthetic) if s.get("commands")]
    real_all = load_jsonl(args.real)
    real = [s for s in real_all if s.get("commands")]
    real_dedup = dedup_by_template(real)

    res = {"n_real_total": len(real_all), "n_real_interactive": len(real),
           "n_real_templates": len(real_dedup), "n_synthetic": len(synthetic)}
    res["structure"] = {"synthetic": corpus_stats(synthetic), "real": corpus_stats(real)}
    res["tactics"] = {"synthetic": tactic_shares(synthetic), "real": tactic_shares(real),
                      "real_dedup": tactic_shares(real_dedup)}

    syn_v = Counter(v for s in synthetic for v in session_verbs(s))
    real_v = Counter(v for s in real for v in session_verbs(s))
    res["verbs"] = {
        "top_synthetic": syn_v.most_common(10),
        "top_real": real_v.most_common(10),
        "jaccard": len(set(syn_v) & set(real_v)) / len(set(syn_v) | set(real_v)),
        "real_mass_covered_by_synthetic_vocab": sum(n for v, n in real_v.items() if v in
syn_v) / sum(real_v.values()),
        "jsd_vs_real": jsd_with_ci(synthetic, real, rng),
        "jsd_vs_real_dedup": jsd_with_ci(synthetic, real_dedup, rng),
    }

    res["c2st"] = {
        "normalized_commands": c2st_auc(synthetic, real_dedup,
                                        lambda s: " ".join(normalize_cmd(c) for c in
s["commands"])),
        "verbs_only": c2st_auc(synthetic, real_dedup, lambda s: "
".join(session_verbs(s))),
    }
```

```
    conditions = sorted({(s["attacker_model"], s["honeypot_backend"]) for s in synthetic})
    res["per_condition"] = []
    for model, backend in conditions:
        sub = [s for s in synthetic if (s["attacker_model"], s["honeypot_backend"]) ==
(model, backend)]
        point, lo, hi = jsd_with_ci(sub, real_dedup, rng)
        shares = tactic_shares(sub)
        res["per_condition"].append({
            "attacker": model, "backend": backend, "n": len(sub),
            "sessions_with_chat_leak": sum(any(has_chat_leak(c) for c in x["commands"]) for
x in sub),
            "jsd_vs_real_dedup": [point, lo, hi],
            "tactics_matching_real": sum(1 for t in TACTICS if (shares[t] > 0) ==
(res["tactics"]["real_dedup"][t] > 0)),
        })

    out = Path(args.out_dir)
    out.mkdir(exist_ok=True)
    (out / "domain_shift.json").write_text(json.dumps(res, indent=2), encoding="utf-8")
    (out / "domain_shift.md").write_text(render_md(res), encoding="utf-8")
    print(render_md(res))


def render_md(r):
    s, rl = r["structure"]["synthetic"], r["structure"]["real"]
    L = ["# Label-free domain-shift analysis: synthetic vs. real\n",
         f"Real: {r['n_real_total']:,} sessions, {r['n_real_interactive']:,} with >=1
command, "
         f"{r['n_real_templates']} distinct normalized templates. Synthetic:
{r['n_synthetic']} sessions.\n",
         "## Table 4a: Corpus structure\n",
         "| Metric | Synthetic | Real (interactive) |", "| :--- | ---: | ---: |",
         f"| Sessions | {s['sessions']} | {rl['sessions']:,} |",
         f"| Distinct behavior templates | {s['distinct_templates']} |
{rl['distinct_templates']} |",
         f"| Share of top-5 templates | {fmt_pct(s['top5_template_share'])} |
{fmt_pct(rl['top5_template_share'])} |",
         f"| Median commands / session | {s['median_cmds_per_session']} |
{rl['median_cmds_per_session']} |",
         f"| Mean chars / command | {s['mean_chars_per_cmd']:.1f} |
{rl['mean_chars_per_cmd']:.1f} |",
         f"| Commands with chaining/pipes | {fmt_pct(s['pct_cmds_chained'])} |
{fmt_pct(rl['pct_cmds_chained'])} |",
         "\n## Table 4b: Share of sessions touching each ATT&CK tactic\n",
         "| Tactic | Synthetic | Real | Real (template-dedup) |", "| :--- | ---: | ---: | -
--: |"]
    for t in TACTICS:
        L.append(f"| {t} | {fmt_pct(r['tactics']['synthetic'][t])} |
{fmt_pct(r['tactics']['real'][t])} | "
                 f"{fmt_pct(r['tactics']['real_dedup'][t])} |")
    v = r["verbs"]
    L += ["\n## Table 4c: Command-verb distribution\n",
          f"- JSD(synthetic, real) = {v['jsd_vs_real'][0]:.3f} (95% bootstrap CI
{v['jsd_vs_real'][1]:.3f}-{v['jsd_vs_real'][2]:.3f}); "
          f"vs. template-dedup real = {v['jsd_vs_real_dedup'][0]:.3f}
({v['jsd_vs_real_dedup'][1]:.3f}-{v['jsd_vs_real_dedup'][2]:.3f}). "
          "Range 0 (identical) to 1 (disjoint).",
          f"- Verb vocabulary Jaccard = {v['jaccard']:.3f}; synthetic vocabulary covers "
          f"{fmt_pct(v['real_mass_covered_by_synthetic_vocab'])} of real verb
occurrences.",
          f"- Top synthetic verbs: {', '.join(f'{w} ({n})' for w, n in
v['top_synthetic'])}",
          f"- Top real verbs: {', '.join(f'{w} ({n})' for w, n in v['top_real'])}",
          "\n## Table 4d: Classifier two-sample test (synthetic vs. template-dedup
real)\n",
          "| Features | AUC (mean +/- sd, 5x5 CV) |", "| :--- | ---: |"]
    for name, (m, sd) in r["c2st"].items():
        L.append(f"| {name.replace('_', ' ')} | {m:.3f} +/- {sd:.3f} |")
    L += ["\nAUC 0.5 = indistinguishable; 1.0 = perfectly separable.\n",
```

```
          "## Table 4e: Per attacker x backend distance to real (template-dedup)\n",
          "| Attacker | Backend | n | JSD vs real [95% CI] | Tactics present/absent as in
real (of 9) | Sessions with chat-format leak |",
          "| :--- | :--- | ---: | :--- | ---: | ---: |"]
    for c in r["per_condition"]:
        p, lo, hi = c["jsd_vs_real_dedup"]
        L.append(f"| {c['attacker']} | {c['backend']} | {c['n']} | {p:.3f} [{lo:.3f},
{hi:.3f}] | "
                 f"{c['tactics_matching_real']} | {c['sessions_with_chat_leak']} |")
    L.append("\nLeaked chat-format tokens and the reasoning text after them are stripped
before all "
             "analyses; commands the model concatenated without separators cannot be
recovered.")
    return "\n".join(L) + "\n"


if __name__ == "__main__":
    main()
```

Blind annotation sheets (make_annotation_sheet.py)

```
"""Build blind annotation sheets: all distinct real templates (>=1 command) + all synthetic
sessions."""
import argparse, csv, os, random
from collections import defaultdict
from session_features import load_jsonl, strip_chat_leak, template_key

MAXCMD, MAXLEN = 30, 200


def render(cmds):
    # Drop the synthetic attackers' "DONE" stop token: it is not a shell command and would
identify the source.
    cmds = [strip_chat_leak(c).strip().replace("\r", " ").replace("\n", " ") for c in cmds]
    while cmds and cmds[-1] == "DONE":
        cmds.pop()
    cmds = [c for c in cmds if c]
    shown = [c if len(c) <= MAXLEN else c[:MAXLEN] + "..." for c in cmds[:MAXCMD]]
    s = " ;; ".join(shown)
    if len(cmds) > MAXCMD:
        s += f" ;; [+{len(cmds) - MAXCMD} more]"
    return len(cmds), s


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--real", default="normalized/real_organic.jsonl")
    ap.add_argument("--synthetic", default="normalized/all_synthetic_120.jsonl")
    ap.add_argument("--out", default="annotation")
    ap.add_argument("--seed", type=int, default=42)
    a = ap.parse_args()

    groups = defaultdict(list)
    for s in load_jsonl(a.real):
        if s.get("commands"):
            groups[template_key(s)].append(s)
    items = []
    for k, ss in groups.items():
        rep = min(ss, key=lambda s: s["session_id"])
        items.append((rep, "real", "", "", len(ss)))
    for s in load_jsonl(a.synthetic):
        items.append((s, "synthetic", s.get("attacker_model", ""),
s.get("honeypot_backend", ""), 1))
    items.sort(key=lambda t: t[0]["session_id"])
    random.Random(a.seed).shuffle(items)
```

```
    os.makedirs(a.out, exist_ok=True)
    rows, key = [], []
    for i, (s, src, model, be, n) in enumerate(items, 1):
        iid = f"A{i:03d}"
        nc, cmds = render(s["commands"])
        rows.append((iid, nc, cmds))
        key.append((iid, s["session_id"], src, model, be, n))
    for name in ("sheet_annotator1.csv", "sheet_annotator2.csv"):
        with open(os.path.join(a.out, name), "w", newline="", encoding="utf-8-sig") as f:
            w = csv.writer(f)
            w.writerow(["item_id", "n_commands", "commands", "label", "confidence",
"notes"])
            for r in rows:
                w.writerow([*r, "", "", ""])
    with open(os.path.join(a.out, "annotation_sheet.csv"), "w", newline="", encoding="utf-
8-sig") as f:
        w = csv.writer(f)
        w.writerow(["item_id", "commands"])
        for r in rows:
            w.writerow([r[0], r[2]])
    with open(os.path.join(a.out, "key.csv"), "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["item_id", "session_id", "source", "attacker_model",
"honeypot_backend", "template_size"])
        w.writerows(key)
    nr = sum(k[2] == "real" for k in key)
    print(f"{len(key)} items: {nr} real, {len(key) - nr} synthetic -> {a.out}/")


if __name__ == "__main__":
    main()
```

Agreement and final labels (agreement.py)

```
"""Inter-annotator agreement + final label table.
Usage: python agreement.py [--dir annotation] [--a1 F] [--a2 F] [--llm F] [--adj F] [--key
F]
                           [--real normalized/real_organic.jsonl] [--md
results/annotation_agreement.md]
"""
import argparse
import csv
import os
import sys
from collections import Counter

from sklearn.metrics import cohen_kappa_score

from session_features import load_jsonl, template_key

LABELS = ["recon", "credential_access", "privilege_escalation", "persistence",
          "malware_drop", "resource_hijacking", "defense_evasion", "other"]


def read_csv(path):
    with open(path, encoding="utf-8-sig", newline="") as f:
        return list(csv.DictReader(f))


def labels_of(path, name, bad):
    out = {}
    for r in read_csv(path):
        v = (r.get("label") or "").strip().lower().replace(" ", "_")
        if not v:
            continue
        if v not in LABELS:
```

```
            bad.append((name, r["item_id"], v))
            continue
        out[r["item_id"]] = v
    return out


def kappa(x, y, ids):
    ids = sorted(ids)
    if len(ids) < 2:
        return None
    a, b = [x[i] for i in ids], [y[i] for i in ids]
    if len(set(a) | set(b)) < 2:
        return 1.0 if a == b else None
    return cohen_kappa_score(a, b, labels=LABELS)


def fmt(k):
    return "n/a" if k is None else f"{k:.3f}"


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--dir", default="annotation")
    ap.add_argument("--a1")
    ap.add_argument("--a2")
    ap.add_argument("--llm")
    ap.add_argument("--adj")
    ap.add_argument("--key")
    ap.add_argument("--real", default="normalized/real_organic.jsonl")
    ap.add_argument("--md", default="results/annotation_agreement.md")
    ap.add_argument("--out-labels")
    ap.add_argument("--out-disagree")
    a = ap.parse_args()
    D = a.dir

    def p(v, n):
        return v or os.path.join(D, n)

    a1p, a2p, keyp = p(a.a1, "sheet_annotator1.csv"), p(a.a2, "sheet_annotator2.csv"),
p(a.key, "key.csv")
    llmp, adjp = p(a.llm, "labels_llm.csv"), p(a.adj, "adjudicated.csv")
    for f in (a1p, a2p, keyp):
        if not os.path.exists(f):
            sys.exit(f"Missing {f}. Run make_annotation_sheet.py first.")
    bad = []
    l1, l2 = labels_of(a1p, "annotator1", bad), labels_of(a2p, "annotator2", bad)
    if not l1 or not l2:
        sys.exit(f"Sheets are still blank (annotator1 labels: {len(l1)}, annotator2:
{len(l2)}). "
                 f"Fill the 'label' column in {a1p} and {a2p} (see annotation/RUBRIC.md),
then rerun.")
    key = {r["item_id"]: r for r in read_csv(keyp)}
    both = sorted(set(l1) & set(l2))
    if len(both) < 2:
        sys.exit("Fewer than 2 items labeled by both annotators; finish labeling first.")
    llm = labels_of(llmp, "llm", bad) if os.path.exists(llmp) else {}
    adj = labels_of(adjp, "adjudicated", bad) if os.path.exists(adjp) else {}

    L = ["# Annotation agreement\n",
         f"Items labeled by both annotators: {len(both)} of {len(key)} "
         f"(annotator1 {len(l1)}, annotator2 {len(l2)}).\n"]
    if bad:
        L.append(f"Invalid labels ignored ({len(bad)}): " + ", ".join(f"{n}:{i}={v}" for n,
i, v in bad[:20]) + "\n")
    L.append("## Cohen's kappa (annotator1 vs annotator2)\n")
    L.append("| subset | n | raw agreement | kappa |\n|---|---|---|---|")
    for name, f in (("all", lambda i: True), ("real", lambda i: key[i]["source"] ==
"real"),
                    ("synthetic", lambda i: key[i]["source"] == "synthetic")):
```

```
        ids = [i for i in both if f(i)]
        agr = sum(l1[i] == l2[i] for i in ids) / len(ids) if ids else None
        L.append(f"| {name} | {len(ids)} | {fmt(agr)} | {fmt(kappa(l1, l2, ids))} |")
    cm = Counter((l1[i], l2[i]) for i in both)
    used = [lab for lab in LABELS if any(lab in k for k in cm)]
    L.append("\n## Confusion (rows annotator1, columns annotator2)\n")
    L.append("| a1 / a2 | " + " | ".join(used) + " |\n|---|" + "---|" * len(used))
    for r in used:
        L.append(f"| {r} | " + " | ".join(str(cm[(r, c)]) for c in used) + " |")
    if llm:
        L.append("\n## LLM pre-labels vs humans (kappa)\n\n| pair | n | raw agreement |
kappa |\n|---|---|---|---|")
        for nm, h in (("LLM vs annotator1", l1), ("LLM vs annotator2", l2)):
            ids = sorted(set(llm) & set(h))
            agr = sum(llm[i] == h[i] for i in ids) / len(ids) if ids else None
            L.append(f"| {nm} | {len(ids)} | {fmt(agr)} | {fmt(kappa(llm, h, ids))} |")

    dis = [i for i in both if l1[i] != l2[i]]
    dpath = a.out_disagree or os.path.join(D, "disagreements.csv")
    rows1, rows2 = read_csv(a1p), read_csv(a2p)
    n1 = {r["item_id"]: (r.get("notes") or "").strip() for r in rows1}
    n2 = {r["item_id"]: (r.get("notes") or "").strip() for r in rows2}
    cmds = {r["item_id"]: r["commands"] for r in rows1}
    with open(dpath, "w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["item_id", "label_annotator1", "label_annotator2", "notes1", "notes2",
"commands", "label_final"])
        for i in dis:
            w.writerow([i, l1[i], l2[i], n1.get(i, ""), n2.get(i, ""), cmds.get(i, ""),
adj.get(i, "")])

    # final per-item labels: agreed, else adjudicated, else excluded
    final, n_agree, n_adj, excluded = {}, 0, 0, []
    for i in key:
        if i in l1 and i in l2 and l1[i] == l2[i]:
            final[i] = l1[i]
            n_agree += 1
        elif i in adj:
            final[i] = adj[i]
            n_adj += 1
        else:
            excluded.append(i)

    # broadcast real item labels to every real session sharing the template
    rows = []
    n_real = 0
    if os.path.exists(a.real):
        real = load_jsonl(a.real)
        by_sid = {s["session_id"]: s for s in real}
        tkey_label = {}
        for i, k in key.items():
            if k["source"] == "real" and i in final and k["session_id"] in by_sid:
                tkey_label[template_key(by_sid[k["session_id"]])] = final[i]
        for s in real:
            if s.get("commands"):
                lab = tkey_label.get(template_key(s))
                if lab:
                    rows.append((s["session_id"], lab, "real"))
                    n_real += 1
    else:
        print(f"WARNING: {a.real} not found; real sessions not broadcast", file=sys.stderr)
    for i, k in key.items():
        if k["source"] == "synthetic" and i in final:
            rows.append((k["session_id"], final[i], "synthetic"))
    lpath = a.out_labels or os.path.join(D, "labels_final.csv")
    with open(lpath, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["session_id", "label", "source"])
        w.writerows(rows)
```

```
    L.append("\n## Final labels\n")
    L.append(f"- Items agreed: {n_agree}; adjudicated: {n_adj}; excluded (disagreement not
adjudicated, "
             f"or unlabeled by an annotator): {len(excluded)} of {len(key)} items "
             f"({len(dis)} disagreements).")
    L.append(f"- Real sessions labeled via template broadcast: {n_real}; "
             f"synthetic: {sum(r[2] == 'synthetic' for r in rows)}.")
    for src in ("real", "synthetic"):
        c = Counter(r[1] for r in rows if r[2] == src)
        L.append(f"- {src} label counts: " + ", ".join(f"{lab}={c[lab]}" for lab in LABELS
if c[lab]))
    os.makedirs(os.path.dirname(a.md) or ".", exist_ok=True)
    with open(a.md, "w", encoding="utf-8") as f:
        f.write("\n".join(L) + "\n")
    print("\n".join(L))
    print(f"\nWrote {a.md}, {dpath}, {lpath}")


if __name__ == "__main__":
    main()
```

Four-quadrant transfer experiment (train_eval_classifier.py)

```
"""
4-quadrant transfer experiment: synthetic vs real training data -> synthetic vs real test
data.

Q1 synth->synth (StratifiedKFold), Q2 real->real (GroupKFold by command template),
Q3 synth->real, Q4 real+synth->real (same folds as Q2).

Labels: rule_label() (same function for both domains) unless --real-labels /
--synthetic-labels CSVs (session_id,label) are supplied. Rule labels are built from
the same keywords as the features -> PLUMBING CHECK only, not a paper result.

Usage:
  python train_eval_classifier.py --synthetic normalized/all_synthetic_120.jsonl \
      --real normalized/real_organic.jsonl [--real-labels CSV] [--synthetic-labels CSV]
"""
import argparse
import csv
import json
import os
from collections import Counter

import numpy as np
from scipy.sparse import hstack, csr_matrix
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import GroupKFold, StratifiedKFold

from session_features import load_jsonl, normalize_cmd, session_tactics, template_key

LABELS = ["recon", "credential_access", "privilege_escalation", "persistence",
          "malware_drop", "resource_hijacking", "defense_evasion", "other"]
SEED = 42


def rule_label(s):
    t = session_tactics(s)
    if "resource_hijacking" in t:
        return "resource_hijacking"
    if "ingress_tool_transfer" in t:
        return "malware_drop"
    if "persistence" in t or "account_manipulation" in t:
```

```
        return "persistence"
    if "privilege_escalation" in t:
        return "privilege_escalation"
    if "credential_access" in t:
        return "credential_access"
    if "defense_evasion" in t:
        return "defense_evasion"
    if "discovery" in t:
        return "recon"
    return "other"


def load_labels(path):
    with open(path, encoding="utf-8-sig") as f:
        return {r["session_id"]: r["label"] for r in csv.DictReader(f)}


def text_of(sessions):
    return [" ".join(normalize_cmd(c) for c in s["commands"]) for s in sessions]


def numeric(sessions):
    rows = []
    for s in sessions:
        cmds = s["commands"]
        lens = [len(normalize_cmd(c)) for c in cmds]
        chained = [any(x in c for x in ("&&", ";", "|")) for c in cmds]
        rows.append([len(cmds), float(np.mean(lens)), float(np.mean(chained))])
    return np.array(rows)


def fit_predict(train, test, ytr):
    vec = TfidfVectorizer(token_pattern=r"[^\s]+", max_features=2000)
    Xtr = hstack([vec.fit_transform(text_of(train)), csr_matrix(numeric(train))]).tocsr()
    Xte = hstack([vec.transform(text_of(test)), csr_matrix(numeric(test))]).tocsr()
    clf = RandomForestClassifier(n_estimators=300, class_weight="balanced",
                                 random_state=SEED, n_jobs=-1)
    clf.fit(Xtr, ytr)
    return list(clf.predict(Xte))


def majority(ytr, n):
    return [Counter(ytr).most_common(1)[0][0]] * n


def row(qid, train, test, yt, yp, ym, ntr):
    # Average F1 over classes that occur in the test set only; otherwise a predicted
    # class with zero support adds a 0 to the mean and fakes a drop.
    present = sorted(set(yt))
    return {"id": qid, "train": train, "test": test,
            "macro_f1": float(f1_score(yt, yp, labels=present, average="macro",
zero_division=0)),
            "accuracy": float(accuracy_score(yt, yp)),
            "predicted": dict(Counter(yp)),
            "majority_macro_f1": float(f1_score(yt, ym, labels=present, average="macro",
zero_division=0)),
            "majority_accuracy": float(accuracy_score(yt, ym)),
            "n_train": int(ntr), "n_test": int(len(yt))}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--synthetic", required=True)
    ap.add_argument("--real", required=True)
    ap.add_argument("--real-labels")
    ap.add_argument("--synthetic-labels")
    ap.add_argument("--label-source", default="human",
                    help="Provenance name reported when label CSVs are given, e.g. 'llm-
dual'.")
```

```
    ap.add_argument("--out-dir", default="results")
    a = ap.parse_args()

    syn = [s for s in load_jsonl(a.synthetic) if s.get("commands")]
    real = [s for s in load_jsonl(a.real) if s.get("commands")]
    for ss, path in ((syn, a.synthetic_labels), (real, a.real_labels)):
        hl = load_labels(path) if path else {}
        for s in ss:
            s["label"] = hl.get(s["session_id"]) or rule_label(s)
    src = a.label_source if (a.real_labels or a.synthetic_labels) else "rule"
    for s in real:
        s["tpl"] = template_key(s)
    ys = [s["label"] for s in syn]
    yr = [s["label"] for s in real]
    tpl_ids = {t: i for i, t in enumerate(dict.fromkeys(s["tpl"] for s in real))}
    groups = [tpl_ids[s["tpl"]] for s in real]
    print(f"synthetic n={len(syn)}, real n={len(real)}, templates={len(tpl_ids)}")

    # Q1
    k = max(2, min(5, min(Counter(ys).values())))
    yp, ym, yt = [], [], []
    for tr, te in StratifiedKFold(k, shuffle=True, random_state=SEED).split(syn, ys):
        ytr = [ys[i] for i in tr]
        yp += fit_predict([syn[i] for i in tr], [syn[i] for i in te], ytr)
        ym += majority(ytr, len(te))
        yt += [ys[i] for i in te]
    q1 = row("Q1", "synthetic", "synthetic", yt, yp, ym, len(syn) * (k - 1) // k)

    # Q2 / Q4 on identical template-grouped folds
    p2, p4, m2, yt2, ntr = [], [], [], [], []
    for tr, te in GroupKFold(5).split(real, yr, groups):
        a_ = [real[i] for i in tr]
        b_ = [real[i] for i in te]
        ytr = [yr[i] for i in tr]
        p2 += fit_predict(a_, b_, ytr)
        m2 += majority(ytr, len(te))
        p4 += fit_predict(a_ + syn, b_, ytr + ys)
        yt2 += [yr[i] for i in te]
        ntr.append(len(tr))
    n2 = int(np.mean(ntr))
    q2 = row("Q2", "real", "real", yt2, p2, m2, n2)
    q4 = row("Q4", "real+synthetic", "real", yt2, p4, m2, n2 + len(syn))

    # Q3
    p3 = fit_predict(syn, real, ys)
    m3 = majority(ys, len(real))
    q3 = row("Q3", "synthetic", "real", yr, p3, m3, len(syn))
    first = {}
    for i, s in enumerate(real):
        first.setdefault(s["tpl"], i)
    idx = list(first.values())
    q3d = row("Q3_dedup", "synthetic", "real_dedup", [yr[i] for i in idx],
              [p3[i] for i in idx], [m3[i] for i in idx], len(syn))
    f1c = f1_score(yr, p3, labels=LABELS, average=None, zero_division=0)
    q3pc = {l: float(f) for l, f in zip(LABELS, f1c)}

    dc = {"synthetic": Counter(ys), "real": Counter(yr),
          "real_dedup": Counter(yr[i] for i in idx)}
    dist = {d: {l: int(c.get(l, 0)) for l in LABELS} for d, c in dc.items()}
    quads = [q1, q2, q3, q3d, q4]
    os.makedirs(a.out_dir, exist_ok=True)
    with open(os.path.join(a.out_dir, "table3_transfer.json"), "w") as f:
        json.dump({"label_source": src, "label_distribution": dist, "quadrants": quads,
                   "q3_per_class_f1": q3pc}, f, indent=2)

    md = ["# Table 3: transfer between synthetic and real training data", ""]
    if src == "rule":
        md += ["> **PLUMBING CHECK - labels derived from same keywords as features.**
Labels come "
```

```
                "from `rule_label()` (ATT&CK-tactic keyword rules in `session_features.py`),
the "
                "same keywords that drive the TF-IDF features. These numbers are NOT a paper
"
                "result; re-run with `--real-labels` / `--synthetic-labels` human labels.",
""]
    md += [f"Label source: **{src}**. Sessions with >=1 command only. TF-IDF (normalized "
           "commands) + 3 numeric features, RandomForest(300, balanced, seed 42). "
           f"Q1: stratified {k}-fold; Q2/Q4: GroupKFold(5) by command template; Q3_dedup:
one "
           "session per template. CV metrics pooled over folds. Majority baseline predicts
the "
           "training-set majority class.", "",
           "| Quadrant | Train | Test | n_train | n_test | Macro-F1 | Accuracy | "
           "Majority macro-F1 | Majority acc |",
           "|---|---|---|---|---|---|---|---|---|"]
    for q in quads:
        md.append(f"| {q['id']} | {q['train']} | {q['test']} | {q['n_train']} |
{q['n_test']} | "
                  f"{q['macro_f1']:.3f} | {q['accuracy']:.3f} |
{q['majority_macro_f1']:.3f} | "
                  f"{q['majority_accuracy']:.3f} |")
    md += ["", "## Label distribution", "",
           "| Label | Synthetic | Real (sessions) | Real (dedup templates) |", "|---|---|--
-|---|"]
    for l in LABELS:
        md.append(f"| {l} | {dist['synthetic'][l]} | {dist['real'][l]} |
{dist['real_dedup'][l]} |")
    md.append("| **total** | " + " | ".join(str(sum(dist[d].values())) for d in dist) + "
|")
    md += ["", "## Q3 per-class F1 (synthetic -> real, session-weighted)", "",
           "| Label | F1 |", "|---|---|"]
    md += [f"| {l} | {q3pc[l]:.3f} |" for l in LABELS]
    md += ["", "## Predicted label counts per quadrant", "",
           "| Quadrant | " + " | ".join(LABELS) + " |", "|---|" + "---|" * len(LABELS)]
    md += [f"| {q['id']} | " + " | ".join(str(q["predicted"].get(l, 0)) for l in LABELS) +
" |"
           for q in quads]
    md += ["", "Macro-F1 averages only over classes present in the test set. Real labels
are "
           f"broadcast from templates, so the effective real sample size is {len(idx)}
templates."]
    with open(os.path.join(a.out_dir, "table3_transfer.md"), "w") as f:
        f.write("\n".join(md) + "\n")
    print("\n".join(md))


if __name__ == "__main__":
    main()
```

Appendix 11: Achievements

Table A11.1: Project achievements and outputs.

Achievement	Detail
Research paper	Prepared in Springer LNCS/LNNS format for the International Conference on Intelligent Communication, Control and Devices (ICICCD 2026), UPES Dehradun; proceedings in Springer's Lecture Notes in Networks and Systems
Open-source release	Full pipeline, 120 synthetic session logs, parser, analysis scripts and the annotation kit published on GitHub: https://github.com/parthkansal823/Honeypot_RedTeaming-paper
Reproducible results	Every number in the paper and this report is written to a file by a released script
Negative result reported honestly	A clear "no" to the transfer question, with the evidence and the limitations stated
A documented parsing pitfall	Showed that a native-Cowrie parser silently reads zero sessions from the CyberLab export
A documented contamination problem	Measured and mitigated chat-format token leakage from one attacker model
Reusable label-free test	A cheap way to check synthetic security data before spending money on labels
Engineering artefacts	Editable draw.io sources for all 20 diagrams, a scripted chart pipeline and a scripted Word build for this report
This report	A full engineering report with 70+ tables, 40+ figures, 10 algorithms and 12 equations, built reproducibly from Markdown

Paper status

[Paper submission status: to be filled in, for example "Submitted to ICICCD 2026 on DD/MM/2026, paper ID ____".]

Authors

Keshav, Anshul Sharma, Parth Kansal and Virain Nayar, Department of Computer Science and Engineering, Chandigarh University, Punjab, India.